



机器狗手势识别

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
- 九、运行示例

目录

一、实验前置准备 (0.5学时)

1.1实验描述

1.2实验目的

1.3实验建议

1.4实验环境

1.5背景知识1 YOLOv5介绍

1.6背景知识2 LabelImg介绍

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

1.1 实验描述

本实验的目标是使用YOLOv5模型对各种手势进行识别，机器狗根据手势做出相应动作。YOLOv5是革命性的“*You Only Look Once*” 物体检测模型的第五次迭代，旨在实时提供高速、高精度的结果。在实验过程中，我们收集大量手势数据集，对预训练权重进行模型训练和调优，并最终转换模型格式，将其在昇腾开发板上进行推理。通过海康工业相机和华为昇腾开发板结合，实现实时手势检测。再进行不同的手势，完成机器狗相应动作。

目录

一、实验前置准备 (0.5学时)

1.1实验描述

1.2实验目的

1.3实验建议

1.4实验环境

1.5背景知识1 YOLOv5介绍

1.6背景知识2 LabelImg介绍

1.7实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

1.2 实验目的

- 了解神经网络的基本原理
- 了解实现机器狗手势识别的原理和方法
- 掌握数据集的标注方法
- 了解onnx模型的简单分析方法
- 掌握如何将onnx模型转换为om模型的方法
- 学会如何将模型应用于机器狗上
- 如何使用华为云modelarts进行模型训练

目录

一、实验前置准备 (0.5学时)

1.1实验描述

1.2实验目的

1.3实验建议

1.4实验环境

1.5背景知识1 YOLOv5介绍

1.6背景知识2 LabelImg介绍

1.7实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

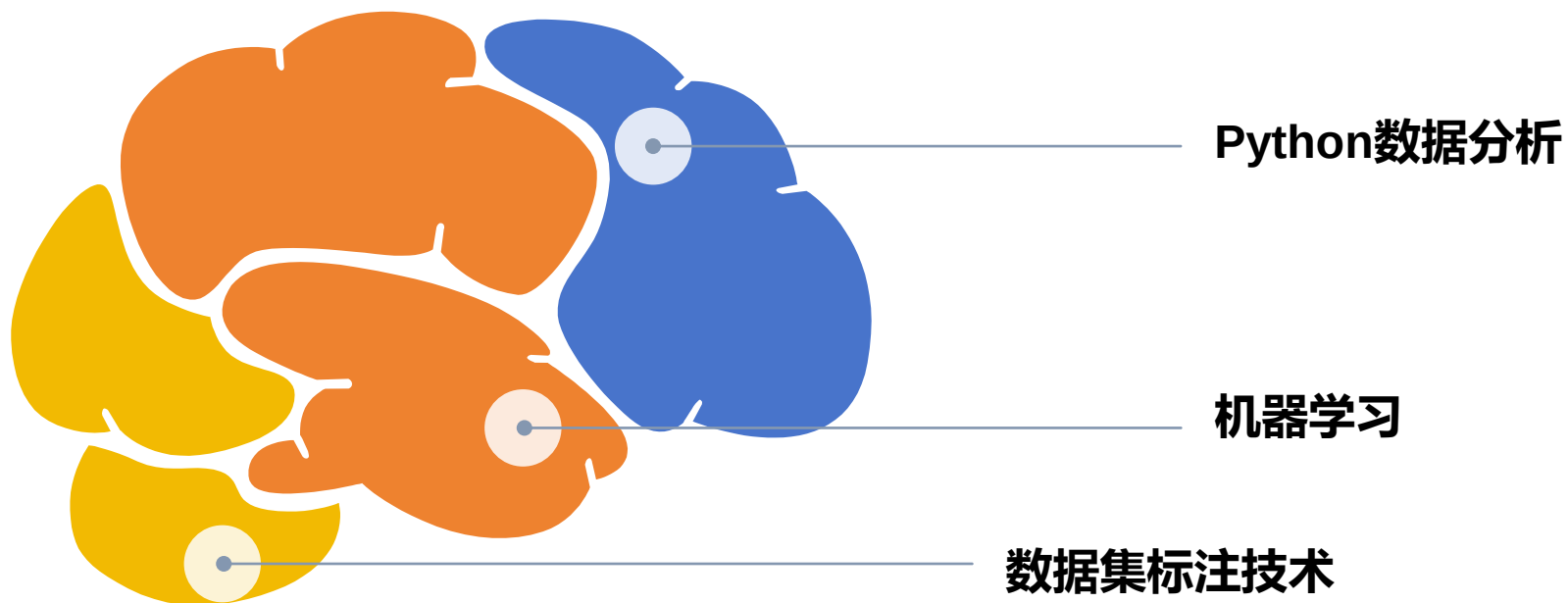
七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

1.3 实验建议

本实验主要是在YOLOv5上用Python编程，模型转换过程可以在自己的PC上进行，程序执行则需要在华为昇腾Atlas 200I DK开发板（Linux）上执行。实验中未注明的执行地方的，一律在开发板上执行。



目录

一、实验前置准备 (0.5学时)

1.1实验描述

1.2实验目的

1.3实验建议

1.4实验环境

1.5背景知识1 YOLOv5介绍

1.6背景知识2 LabelImg介绍

1.7实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

1.4实验环境——实验硬件设备

本实验需要的准备的硬件如下：（详见章节3.1、3.4）

1



机器狗

2



PC

3



昇腾开发板

4



读卡器

5



摄像头

1.4实验环境——实验软件环境

本实验需要的准备的软件环境如下：（详见章节3.2、3.3）



目录

一、实验前置准备 (0.5学时)

1.1实验描述

1.2实验目的

1.3实验建议

1.4实验环境

1.5背景知识1 YOLOv5介绍

1.6背景知识2 LabelImg介绍

1.7实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

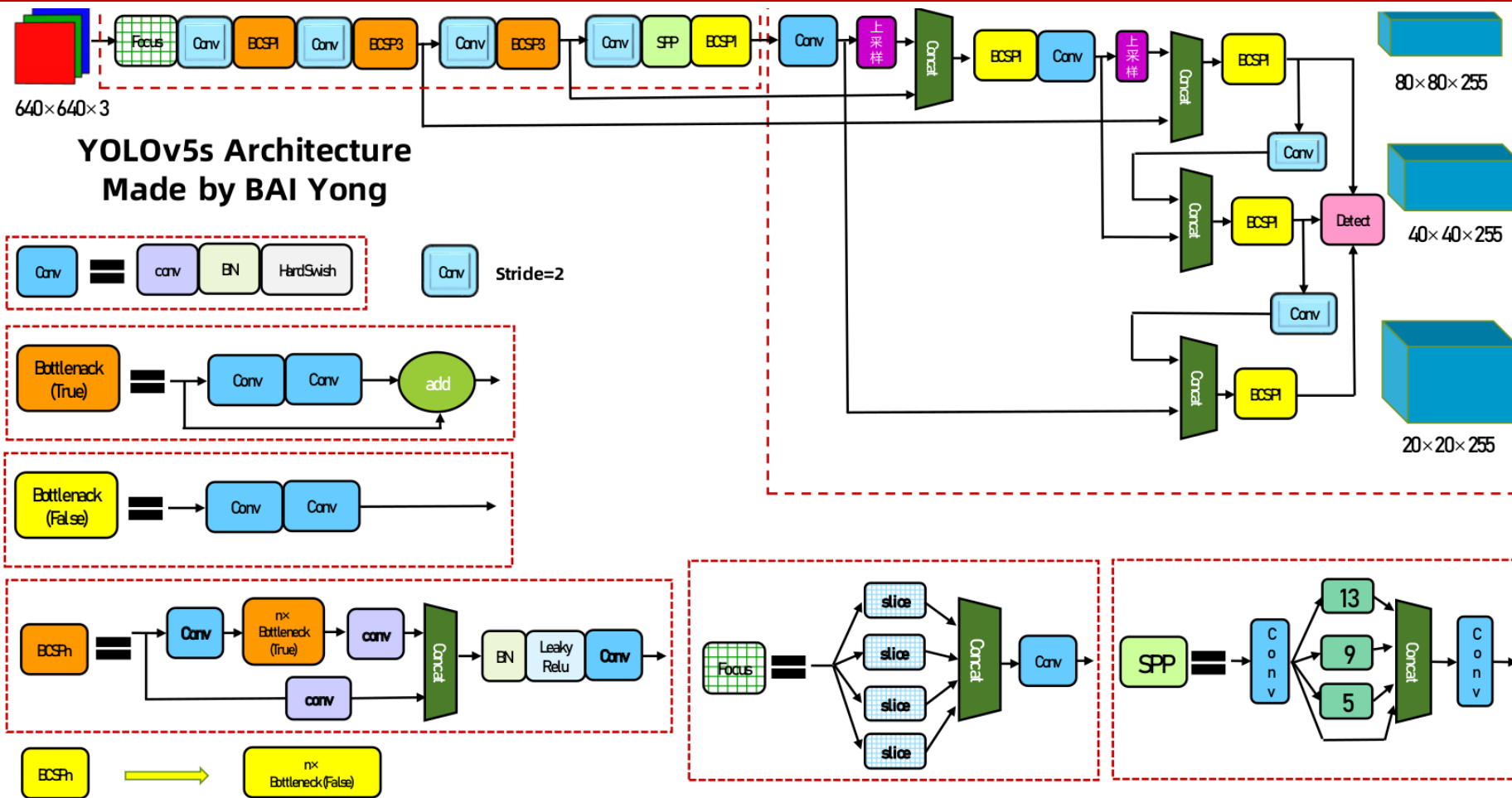
六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

1.5 背景知识1 YOLOv5介绍



YOLOv5是一个计算机视觉模式设计的对象检测任务，它是YOLO系列的升级版，

YOLO系列以其实时物体检测能力而著称。

目录

一、实验前置准备 (0.5学时)

1.1实验描述

1.2实验目的

1.3实验建议

1.4实验环境

1.5背景知识1 YOLOv5介绍

1.6背景知识2 LabelImg介绍

1.7实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

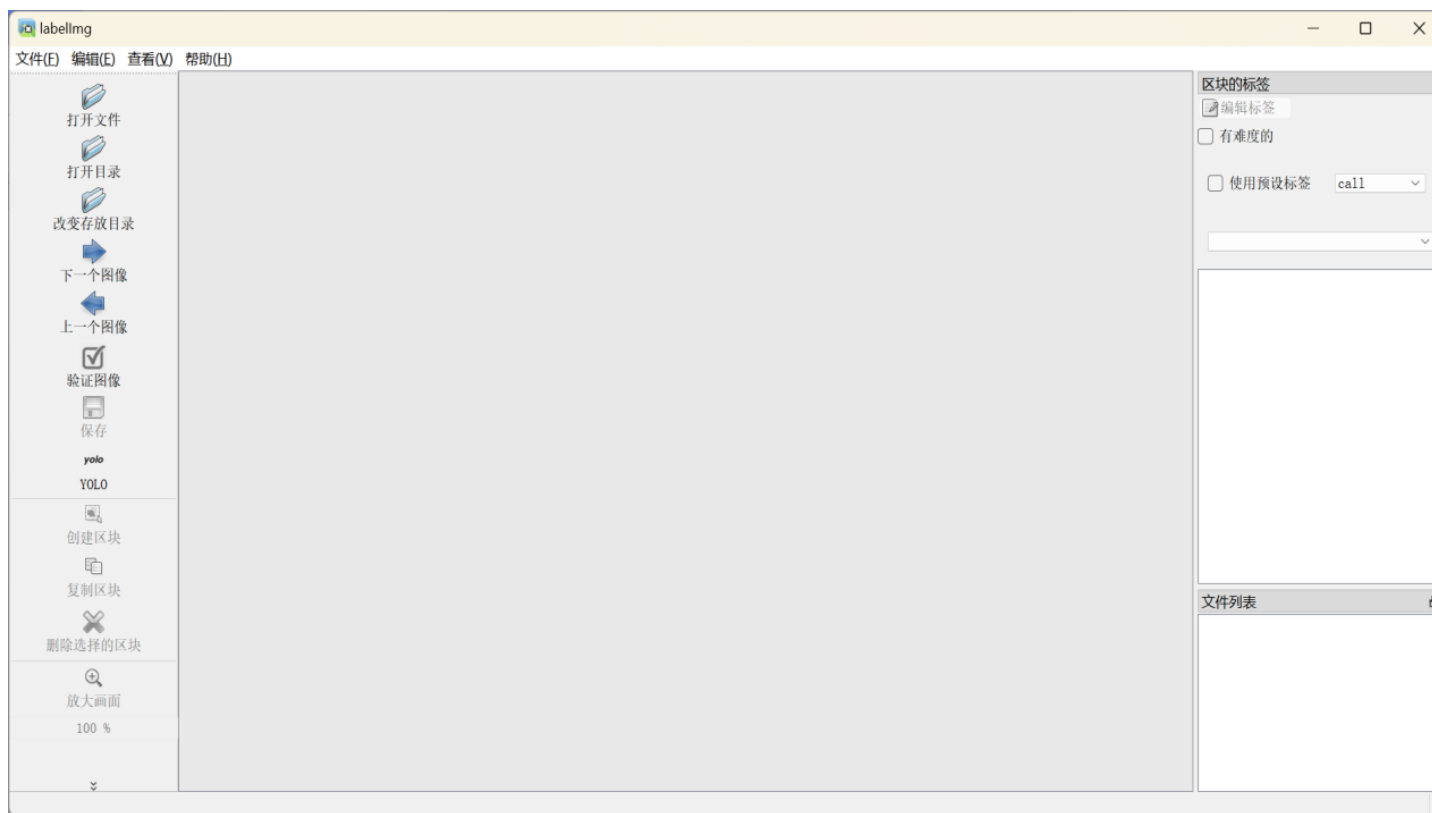
七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

1.6 背景知识2 labelImg介绍

labelImg是一款开源的图像标注工具，它主要用于标注目标检测、语义分割和图像分类等深度学习中需要的数据集。通过使用labelImg，用户可以快速、准确地为图片中的目标添加标注信息，从而建立数据集。



目录

一、实验前置准备 (0.5学时)

1.1实验描述

1.2实验目的

1.3实验建议

1.4实验环境

1.5背景知识1 YOLOv5介绍

1.6背景知识2 LabelImg介绍

1.7实验道具讲解

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

1.7实验道具讲解



fist



one



peace



rock



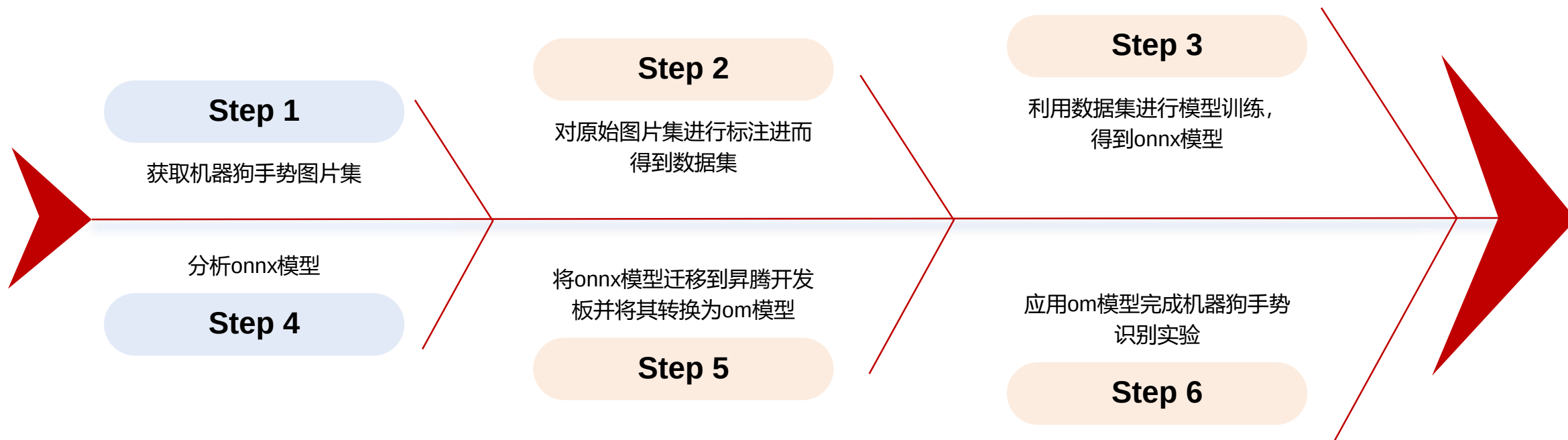
three2

当机器狗摄像头识别到fist时，机器狗会执行“趴下”动作；
当机器狗摄像头识别到one时，机器狗会执行“扭身跳”动作；
当机器狗摄像头识别到peace时，机器狗会执行“扭身体”动作；
当机器狗摄像头识别到rock时，机器狗会执行“太空步”动作；
当机器狗摄像头识别到three2时，机器狗会执行“打招呼”动作。

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
- 九、运行示例

2.实验过程描述



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

3.1 硬件环境准备

3.2 软件环境 (YOLO训练)

3.2 安装和配置CUDA

3.3 开发板环境 (机器狗网址配置)

四、工程文件目录

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

3.1 硬件环境准备

(1) SD卡中系统烧录

向SD卡中烧录镜像的步骤如下：

- ① 将SD卡插入读卡器，再将读卡器插到PC上。



3.1 硬件环境准备

(1) SD卡中系统烧录

②在浏览器中进入Ascend官网，选择“产品”->“开发者套件”



3.1 硬件环境准备

(1) SD卡中系统烧录

③点击“资源”，进入资源下载界面



3.1 硬件环境准备

(1) SD卡中系统烧录

④选择“制卡工具下载”，下载完成后进行安装。

寻你所需，玩转AI开发_

快速入门



准备

请提前做好下列物品，或查看 [硬件准备清单](#)，快速上手不卡顿

- MicroSD卡（推荐64GB）及读卡器（可选）
- 获取制卡工具，一键烧录镜像完成开发环境准备

[↓ 制卡工具下载](#) [↓ 校验文件下载](#)



入门

提供入门 Atlas 200I DK A2 的全流程指导，帮助你30分钟内轻松完成从启动开发者套件到运行首样例的过程

[▶ 快速开始（课程）](#) [📖 快速开始（文档）](#)



参考工具

提供Atlas 200I DK A2 进一步开发所需的参考工具及配套资源，可按需下载

模型适配工具

使能开发者0代码完成数据标注、模型训练、模型部署到昇腾开发者套件

[↓ 软件包下载](#) [↓ 校验文件下载](#)

配套Conda环境包

模型适配工具运行所必须的配套conda依赖环境，需要提前安装conda并导入conda环境包

[↓ 软件包下载](#) [↓ 校验文件下载](#)

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Page 24

 HUAWEI

3.1 硬件环境准备

(1) SD卡中系统烧录

⑤打开镜像烧录工具Ascend AI Devkit Imager，选择Atlas 200I DK A2



3.1 硬件环境准备

(1) SD卡中系统烧录

⑥打开“配置网络信息”，选择“Type-C”选项卡。可以看到Type-C接口默认IP为192.168.0.2。选择默认IP。

镜像网络信息配置

×

以下为镜像默认IP信息，您可以根据PC/路由器IP信息重新填写网络信息

网口信息

ETH0

ETH1

Type-C

☐ 自动获取IP地址

☒ 使用下面的IP地址

IP地址

192 · 168 · 0 · 2

子网掩码

255 · 255 · 255 · 0

默认网关

· · ·

首选DNS服务器

8 · 8 · 8 · 8

备用DNS服务器

114 · 114 · 114 · 114

确定

取消

3.1 硬件环境准备

(1) SD卡中系统烧录

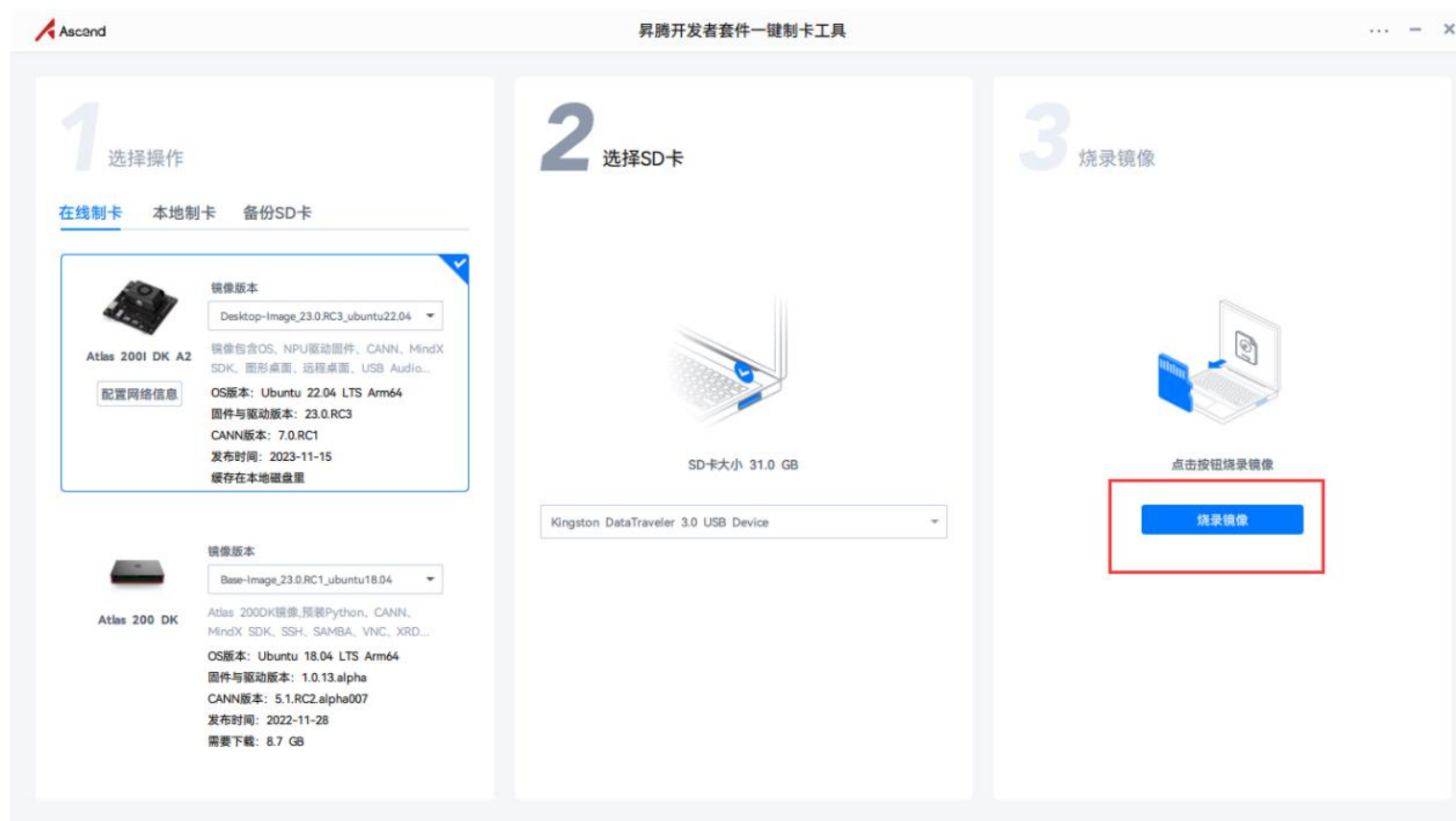
⑦选择要烧录的SD卡。



3.1 硬件环境准备

(1) SD卡中系统烧录

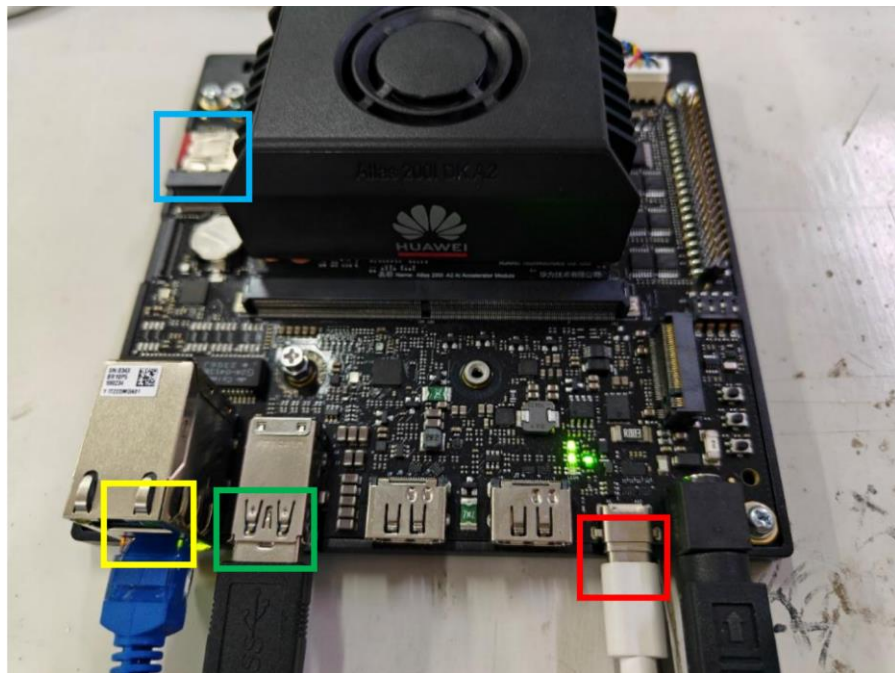
⑧点击“烧录镜像”，等待镜像烧录完成。



3.1 硬件环境准备

(2) 开发板连接

将烧录好系统的SD卡插入到开发板中（下方蓝色框中），昇腾开发板和PC机通过Type-C线连接，将Type-C线和电源线连接到开发板（下方红色框中），并将海康工业相机和机器狗的USB线连接到开发板（下方绿色框中）。同时可以连接网线（下方黄色框中），让开发版连接互联网，方便环境依赖包的下载。连接方式如下图所示。



3.1 硬件环境准备

(3) PC端远程连接开发板

当开发者套件通过Type-C接口和PC连接时，如何在PC将接口 IP地址修改为和开发者套件接口 IP地址同一个网段（如开发者套件Type-C 接口为192.168.0.2， PC对应USB或Type-C接口为192.168.0.101）， 并使用SSH工具远程登录开发者套件。

1) 安装Windows的USB网卡驱动（以Windows 10系统为例）

①在PC上右键单击“此电脑”， 选择“更多” → “管理”， 进入“计算机管理” 界面。

②在“计算机管理” 操作界面中选择“设备管理器→“其他设备”， 如图所示， RNDIS为未识别状态。

注意：如果未出现RNDIS， 请按以下方法排查：

确保开发者套件Type-C接口和PC已通过连接线正常连接。

更换具备数据传输能力的Type-C数据线（一般手机充电线拥有数据传输能力）， 继续查看是否出现RNDIS。

3.1 硬件环境准备

(3) PC端远程连接开发板

如果尝试现场所有Type-C数据线，仍无法出现RNDIS，则考虑使用 RJ45网线连接PC和开发者套件的以太网口的方式登录开发者套件。

③右键单击“RNDIS”，选择“更新驱动程序(P)”。

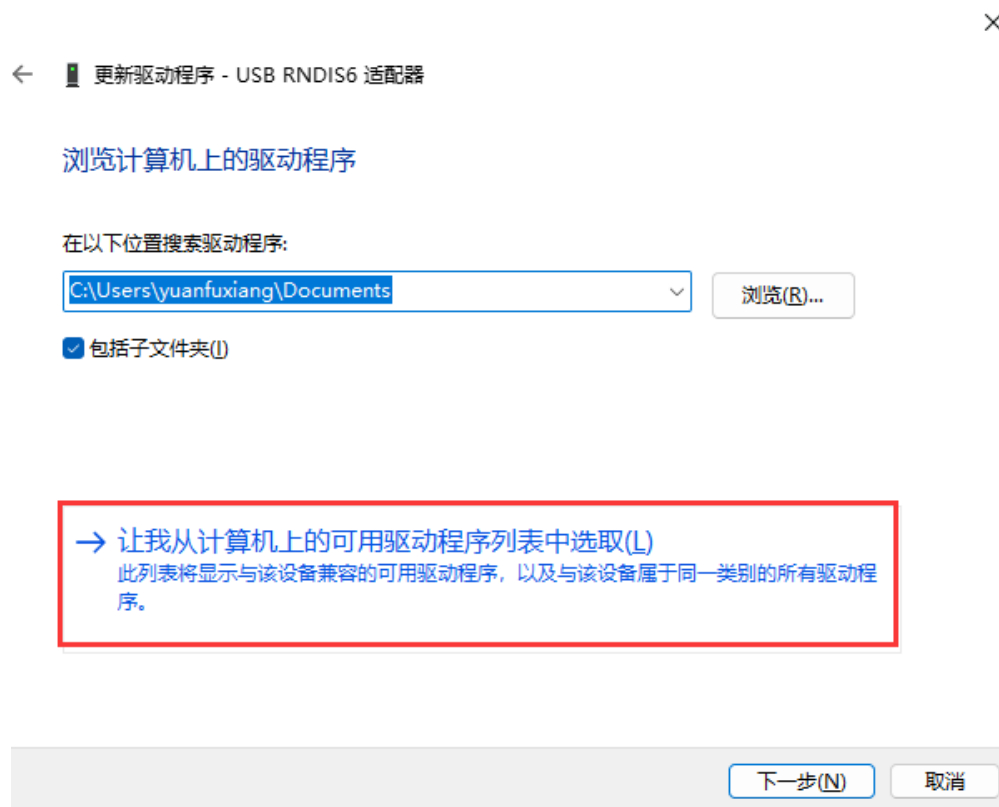
④在弹出的“更新驱动程序”→“RNDIS窗口”中选择“浏览我的计算机以查找驱动程序软件(R)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

⑤然后选择“让我从计算机上的可用驱动程序列表中选择(L)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

⑥在“常见硬件类型”列表中选择“网络适配器”，单击“下一页(N)”。



3.1 硬件环境准备

(3) PC端远程连接开发板

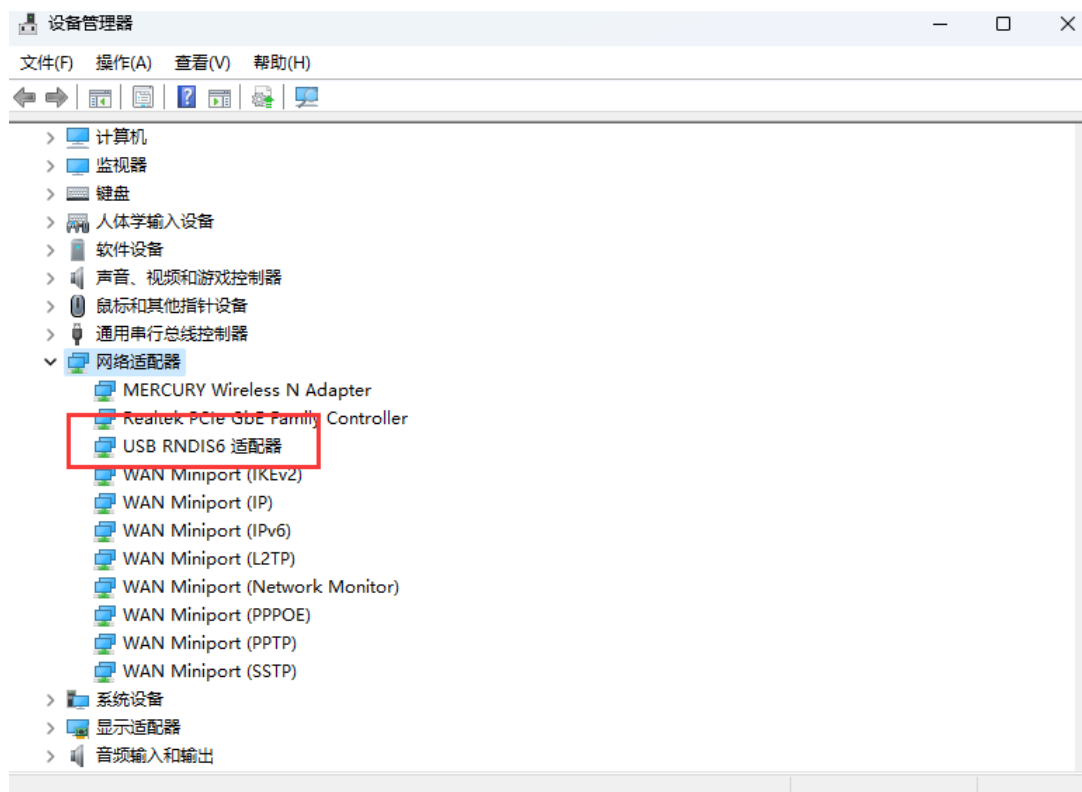
⑦在“选择要为此硬件安装的设备驱动程序”界面中选择“Microsoft”厂商的“USB RNDIS6适配器”。



3.1 硬件环境准备

(3) PC端远程连接开发板

- ⑧单击“下一页”，在弹出的“更新驱动程序警告”窗口选择“是”。
- ⑨返回“设备管理器”→“网络适配器”，可看到已经正常显示了USB RNDIS6适配器的驱动。



3.1 硬件环境准备

(4) 配置PC接口IP地址

本步骤以 Windows 10 系统为例。

①在 PC 上打开“开始”，单击“设置”按钮，进入“Windows 设置”界面。

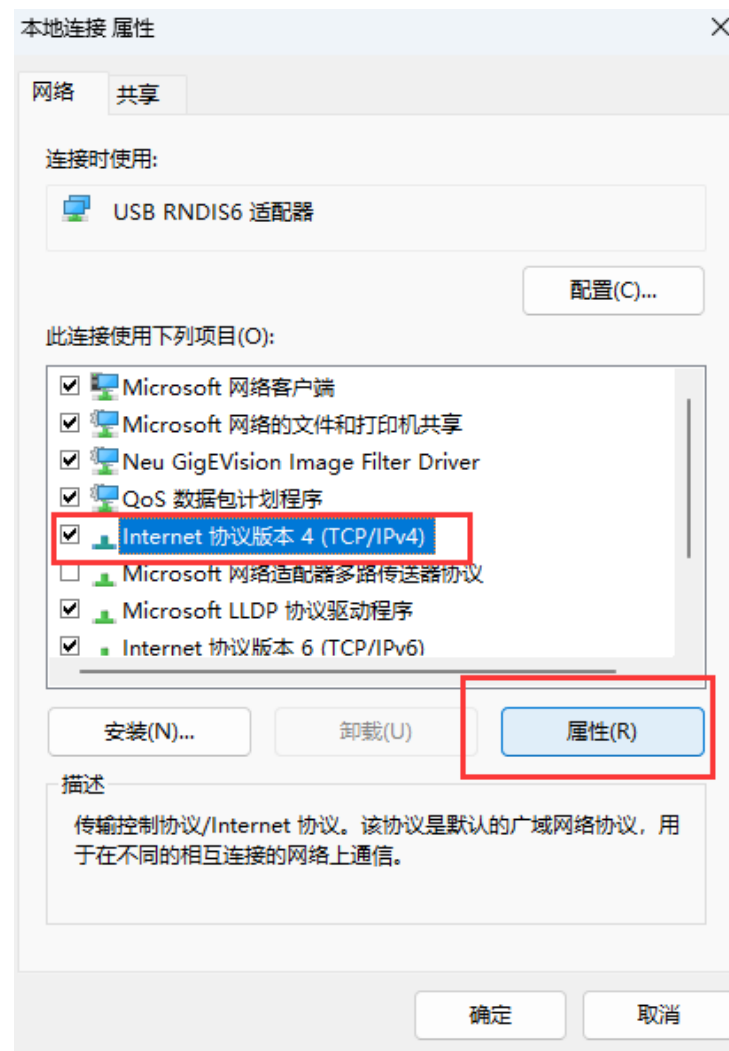
②选择“网络和 Internet”，单击“更改适配器选项”。

③鼠标右键单击“本地连接”后鼠标左键单击“属性”进入“本地连接 属性”界面（使用 Type-C 接口连接时一般为“本地连接 x”，x 为数字，以实际显示的数字为准。

3.1 硬件环境准备

(4) 配置PC接口IP地址

④选择 “Internet协议版本4 (TCP/IPv4) ” ， 单击 “属性” 。



3.1 硬件环境准备

(4) 配置PC接口IP地址

⑤勾选“使用下面的IP地址”选项，填写IP地址（图示以192.168.0.101为例）和子网掩码，默认网关与DNS服务器地址为空，单击“确定”保存。

Internet 协议版本 4 (TCP/IPv4) 属性

常规

如果网络支持此功能，则可以获取自动指派的 IP 设置。否则，你需要从网络系统管理员处获得适当的 IP 设置。

☐ 自动获得 IP 地址(O)

☒ 使用下面的 IP 地址(S):

IP 地址(I): 192 . 168 . 0 . 101

子网掩码(U): 255 . 255 . 255 . 0

默认网关(D): . . .

☐ 自动获得 DNS 服务器地址(B)

☒ 使用下面的 DNS 服务器地址(E):

首选 DNS 服务器(P): . . .

备用 DNS 服务器(A): . . .

☐ 退出时验证设置(L)

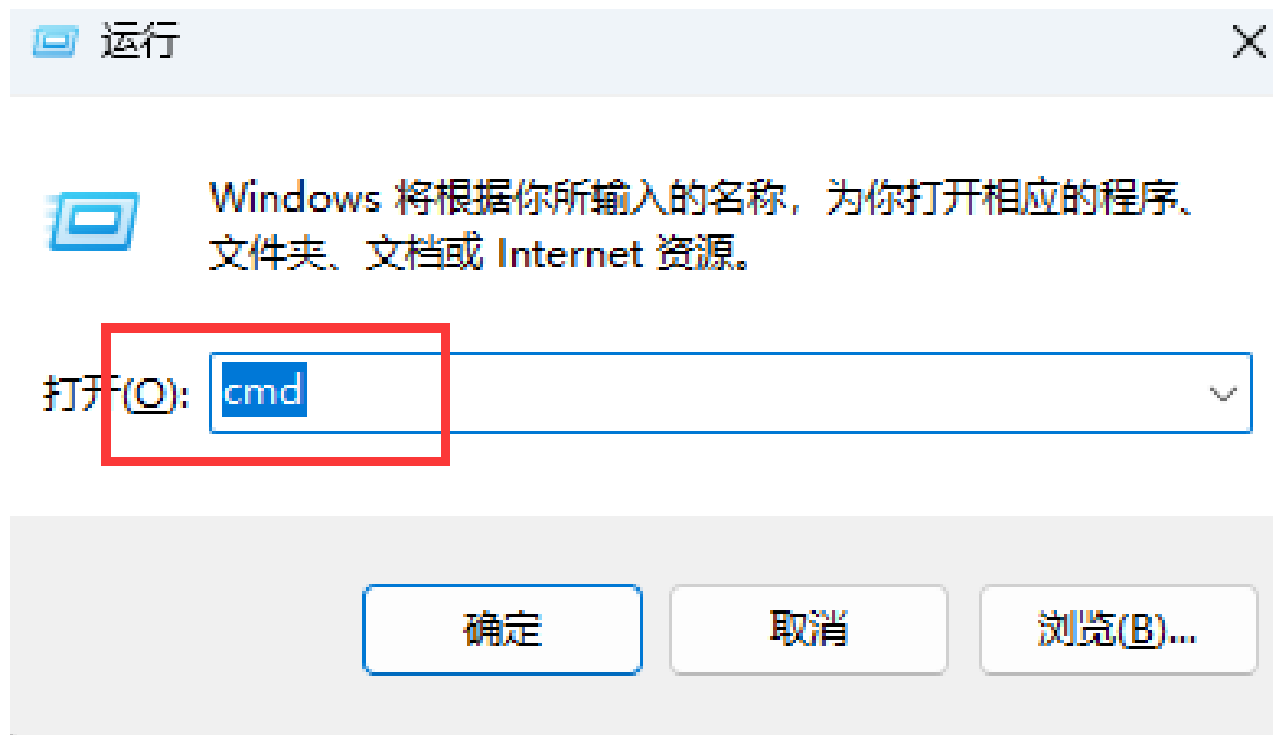
高级(V)...

确定 取消

3.1 硬件环境准备

(4) 配置PC接口IP地址

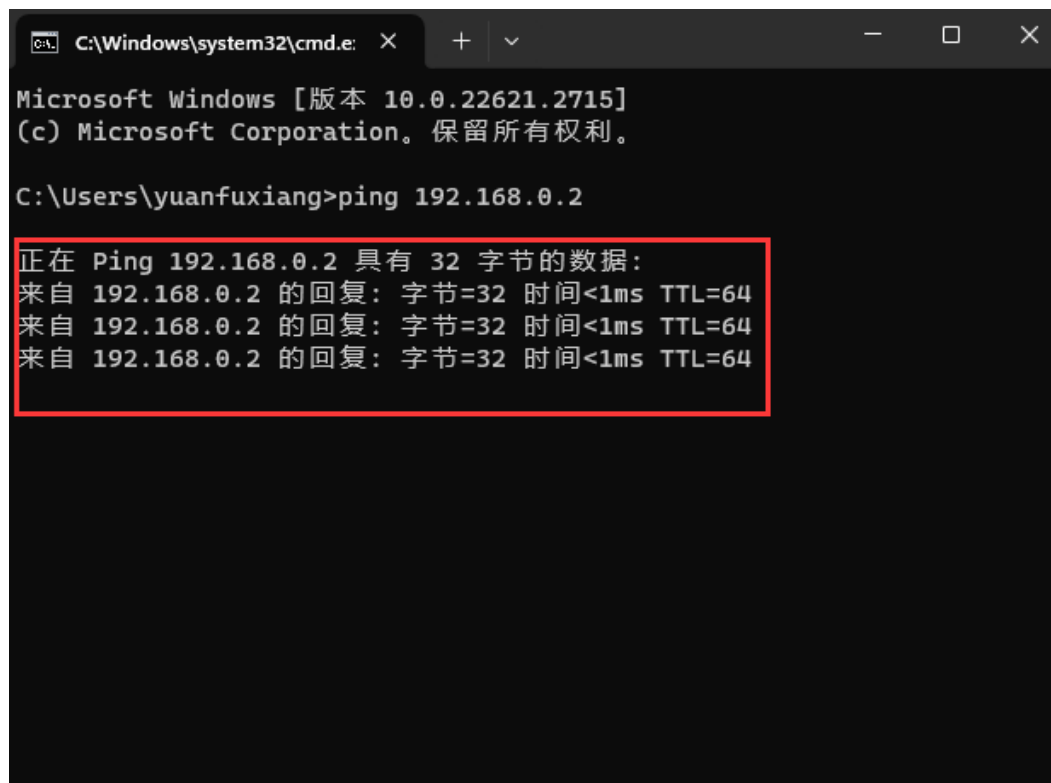
⑥使用快捷键 “Win+R” ， 在运行窗口输入cmd进入命令行窗口。



3.1 硬件环境准备

(4) 配置PC接口IP地址

⑦输入ping 192.168.0.2命令，查看是否能够访问开发板IP。



```
C:\Windows\system32\cmd.exe X + - □ X

Microsoft Windows [版本 10.0.22621.2715]
(c) Microsoft Corporation。保留所有权利。

C:\Users\yuanfuxiang>ping 192.168.0.2

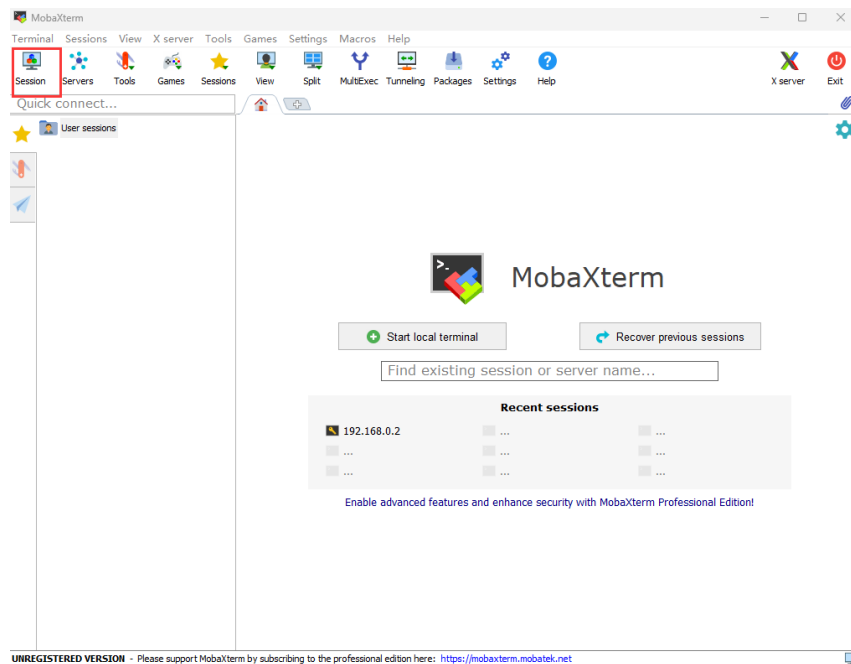
正在 Ping 192.168.0.2 具有 32 字节的数据:
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
来自 192.168.0.2 的回复: 字节=32 时间<1ms TTL=64
```

3.1 硬件环境准备

(5) 使用ssh远程登录开发板

本文使用MobaXterm工具在PC上远程登录开发板，单击下载链接获取MobaXterm软件压缩包，解压获得“MobaXterm_Personal_22.2.exe”。

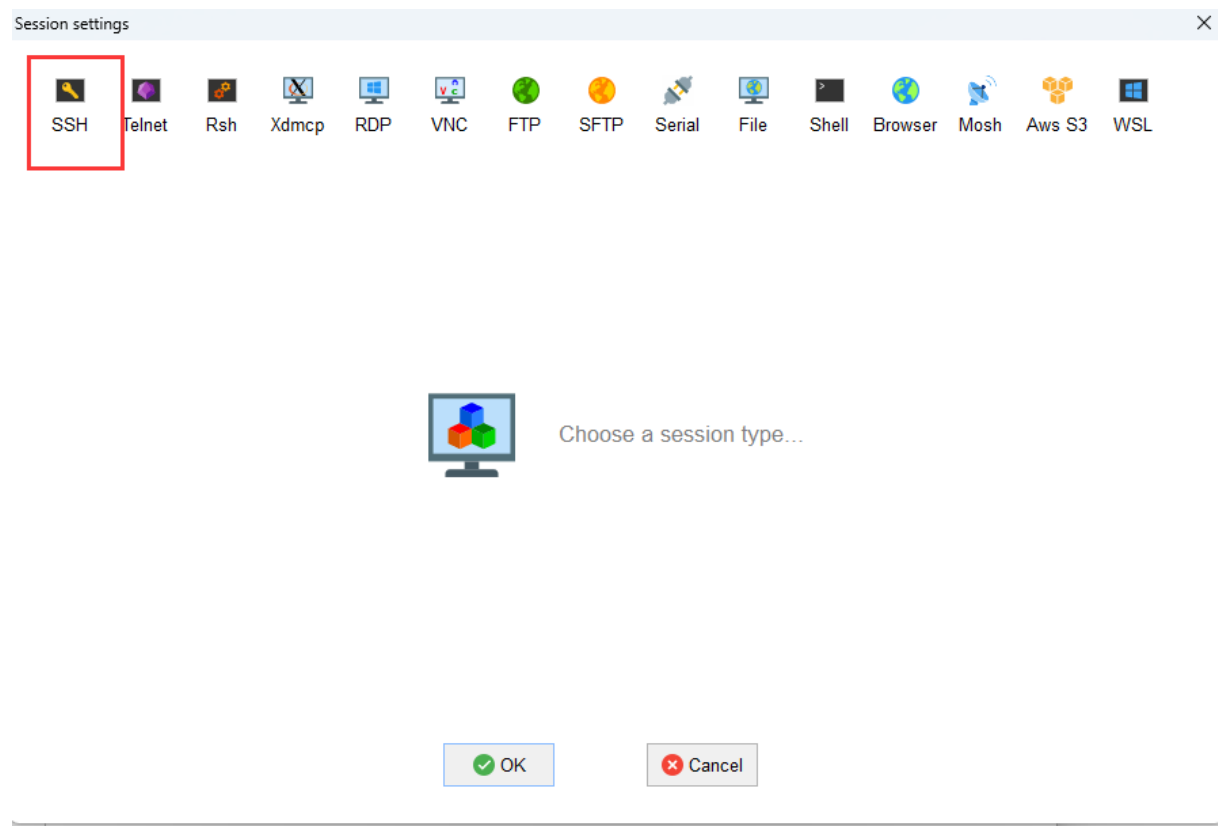
- ①双击“MobaXterm_Personal_22.2.exe”启动SSH登录工具。
- ②单击左上方的“Session”进入界面。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

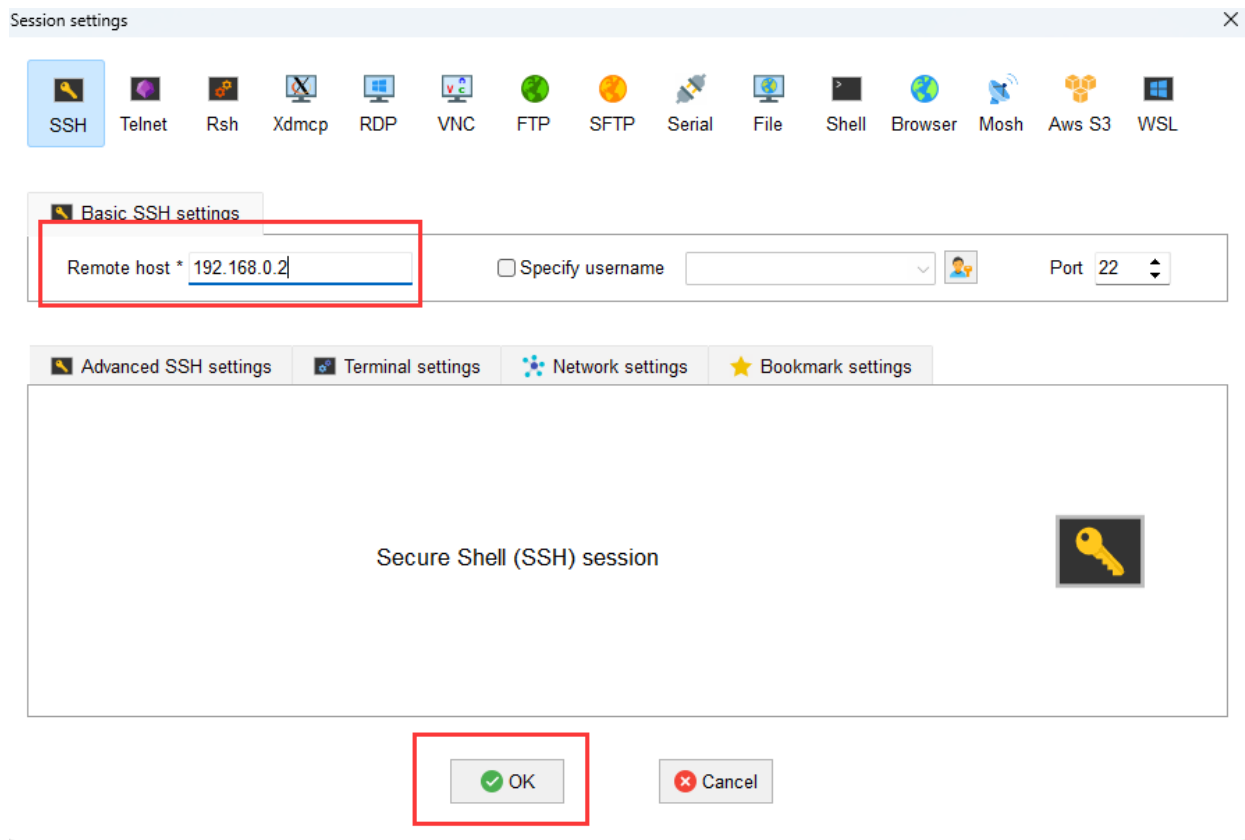
③单击左上方的“SSH”进入 SSH 连接配置界面



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

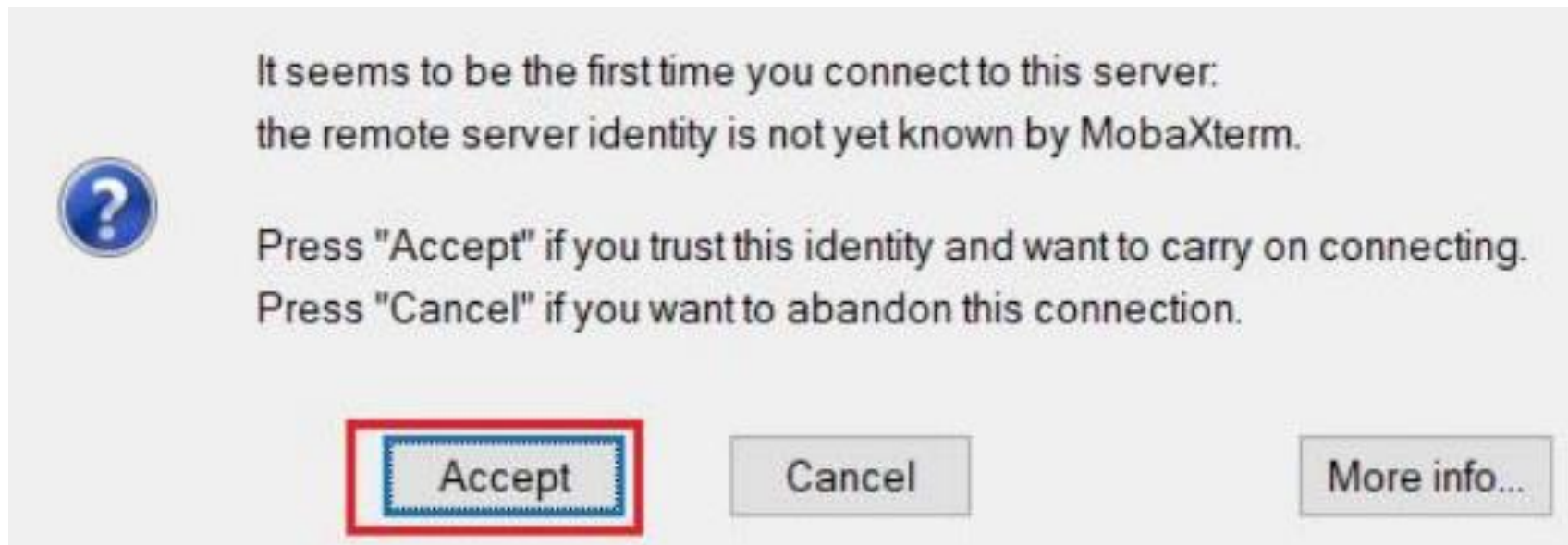
④根据硬件连接方式填写开发者套件连接PC的接口实际IP地址（一键制卡中配置的IP地址）。图示以192.168.0.2为例，请根据实际修改。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

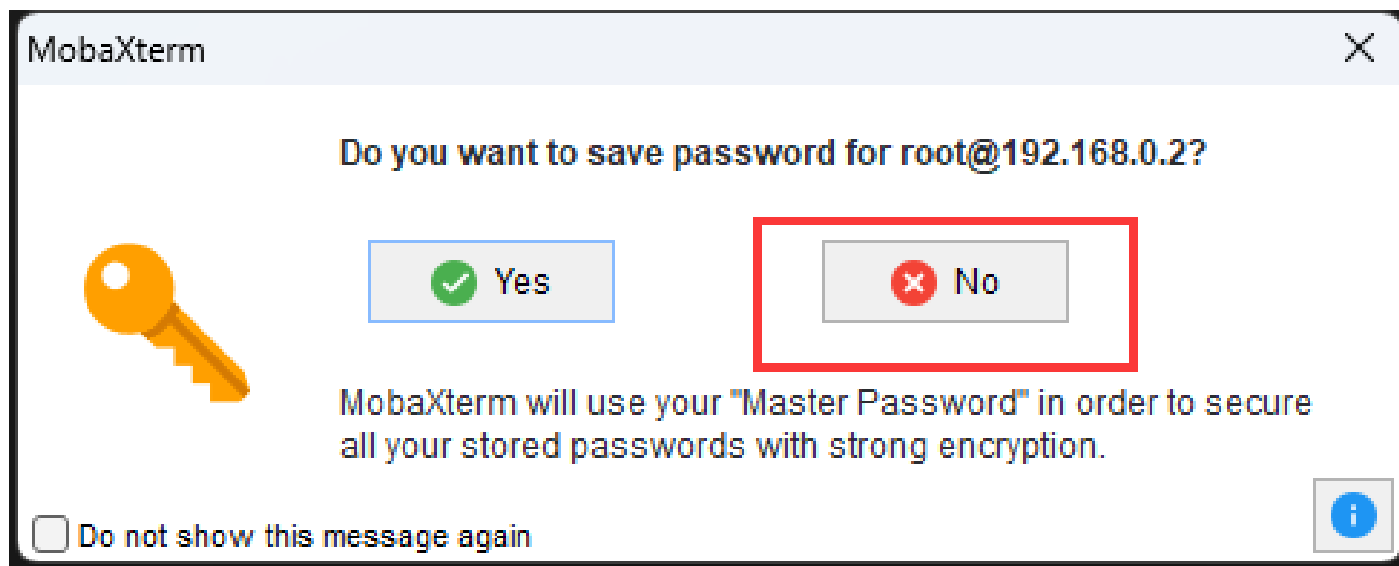
⑤单击“OK”按钮，首次连接开发者套件时，SSH工具提示是否信任连接的设备，单击“Accept”。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

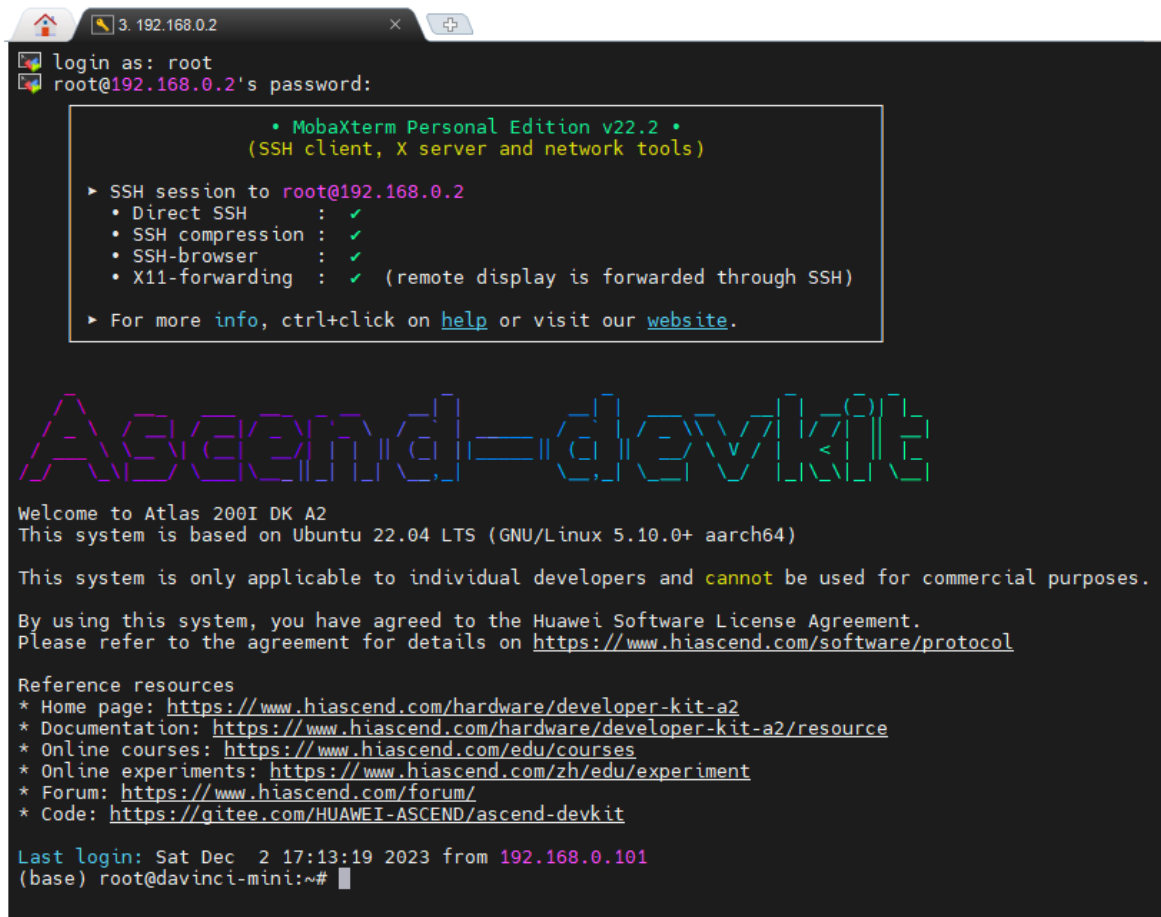
⑥进入远程登录界面后，输入 root 用户名登录密码（默认为 Mind@123）登录开发者套件，请修改默认密码，并妥善保管新密码。SSH 工具界面会出现保存密码提示，可以单击 “No”，不保存密码直接登录开发者套件。



3.1 硬件环境准备

(5) 使用ssh远程登录开发板

⑦远程登录开发者套件成功界面如图所示。



```
login as: root
root@192.168.0.2's password:

• MobaXterm Personal Edition v22.2 •
  (SSH client, X server and network tools)

▶ SSH session to root@192.168.0.2
  • Direct SSH      : ✓
  • SSH compression : ✓
  • SSH-browser     : ✓
  • X11-forwarding  : ✓ (remote display is forwarded through SSH)

▶ For more info, ctrl+click on help or visit our website.

Ascend=devkit

Welcome to Atlas 200I DK A2
This system is based on Ubuntu 22.04 LTS (GNU/Linux 5.10.0+ aarch64)

This system is only applicable to individual developers and cannot be used for commercial purposes.

By using this system, you have agreed to the Huawei Software License Agreement.
Please refer to the agreement for details on https://www.hiascend.com/software/protocol

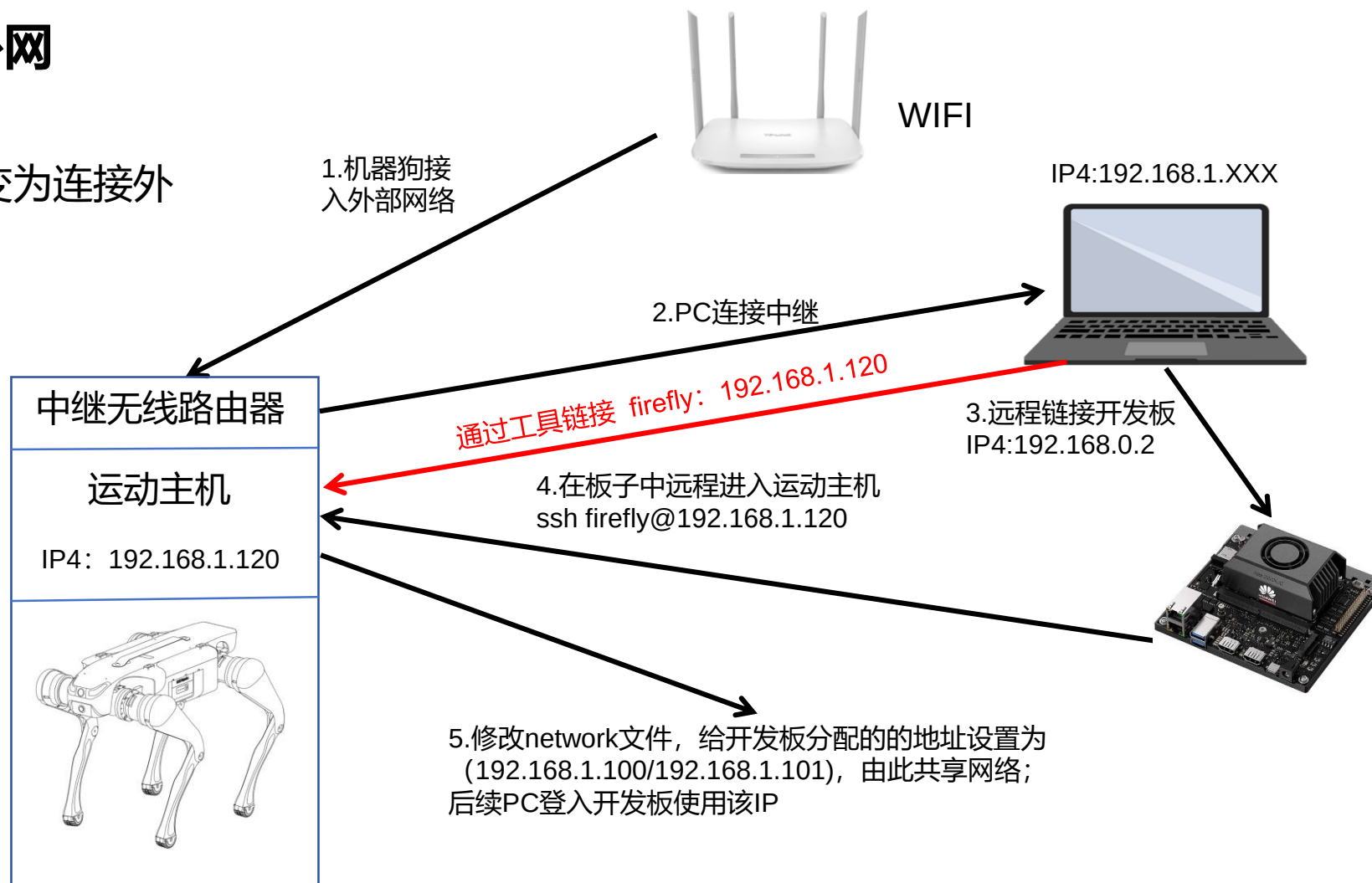
Reference resources
* Home page: https://www.hiascend.com/hardware/developer-kit-a2
* Documentation: https://www.hiascend.com/hardware/developer-kit-a2/resource
* Online courses: https://www.hiascend.com/edu/courses
* Online experiments: https://www.hiascend.com/zh/edu/experiment
* Forum: https://www.hiascend.com/forum/
* Code: https://gitee.com/HUAWEI-ASCEND/ascend-devkit

Last login: Sat Dec  2 17:13:19 2023 from 192.168.0.101
(base) root@davinci-mini:~#
```

3.1 硬件环境准备

(6) 机器狗中继连接外网

此步骤是为了让单机环境变为连接外部网络的环境



3.1 硬件环境准备

(6) 机器狗中继连接外网

注：本教程主要为暂时没有外置无线网卡的用户提供连接外网方案，教程以中继手机热点为例进行说明，中继无线路由器同理。

1、电脑连接 机器狗的wifi，点击电脑右下角的wifi图标，点击属性查看IPv4 DNS服务器的ip地址。如下图所示。

YSC-JYML-emzqzj
已连接，安全

属性①

断开连接

属性

SSID: YSC-JYML-emzqzj

协议: Wi-Fi 4 (802.11n)

安全类型: WPA2-个人

网络频带: 2.4 GHz

网络通道: 11

链接速度(接收/传输): 144/144 (Mbps)

IPv4 地址: 192.168.1.101

IPv4 DNS 服务器: 192.168.1.254 8.8.8.8②

制造商: Intel Corporation

描述: Intel(R) Wireless-AC 9560 160MHz

驱动程序版本: 22.200.0.6

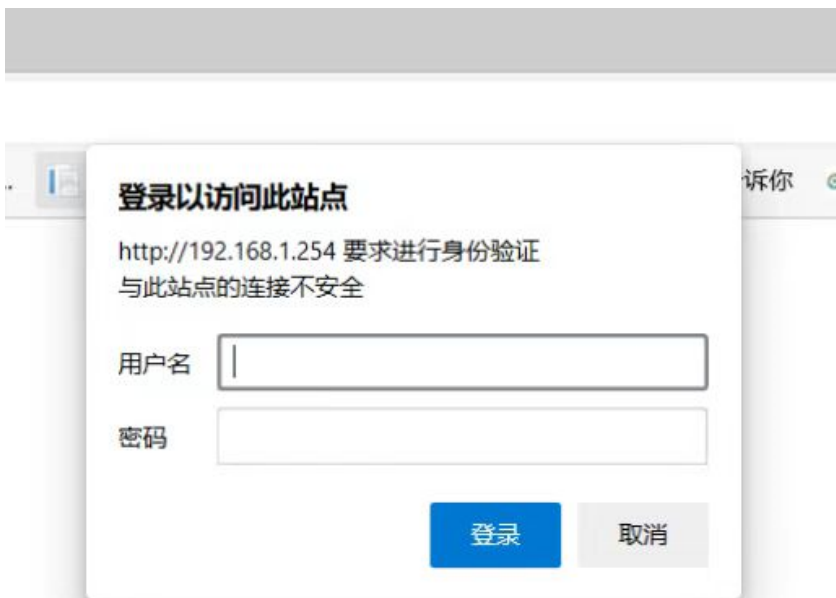
物理地址(MAC):

复制

3.1 硬件环境准备

(6) 机器狗中继连接外网

2、打开电脑浏览器，输入IPv4 DNS服务器的ip，默认登录用户和密码均为admin。
如下图所示。

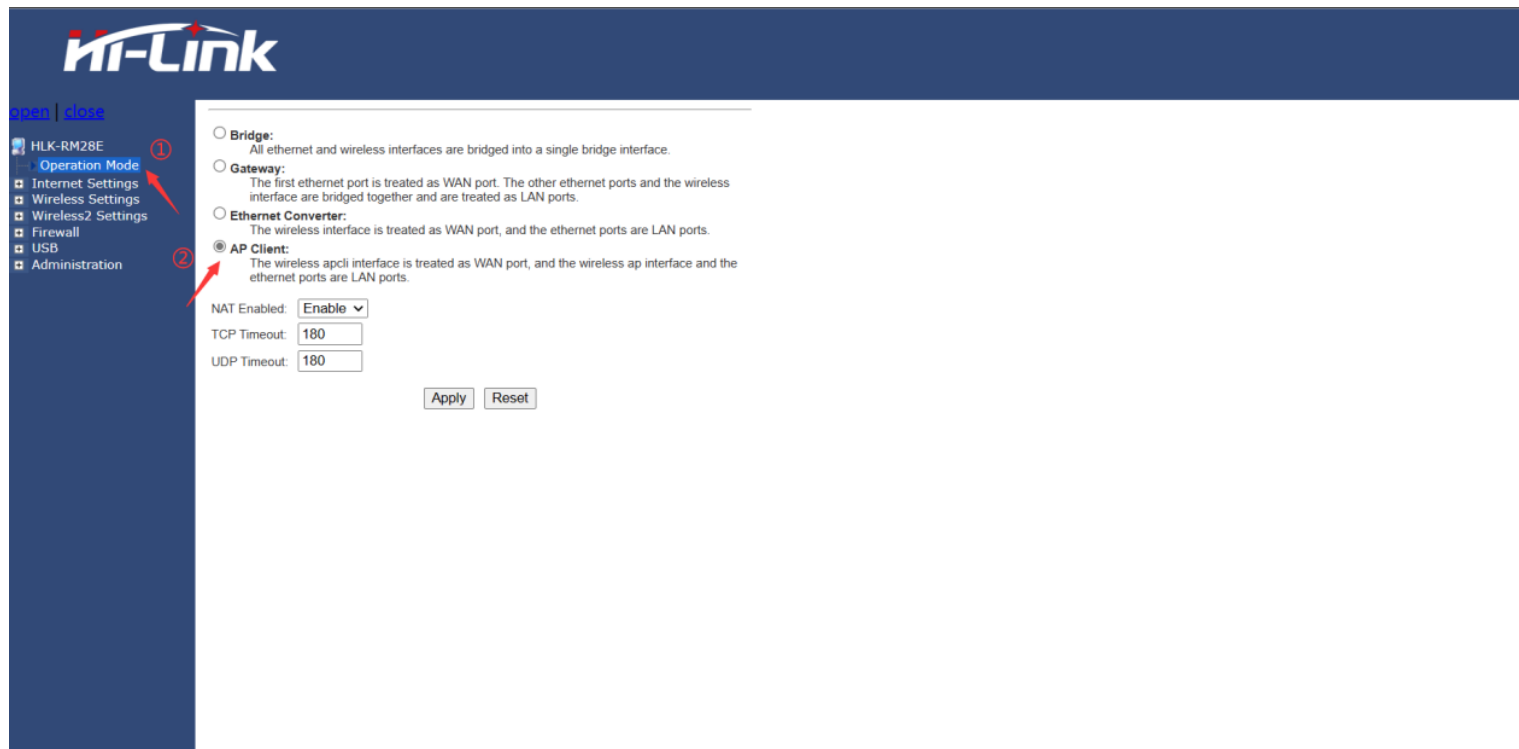


3.1 硬件环境准备

(6) 机器狗中继连接外网

3、打开手机热点。

4、进入Hi-link设置页面，点击Operation Mode，选择AP Client，最后点击Apply。如下图所示。



3.1 硬件环境准备

(6) 机器狗中继连接外网

5、展开Internet Settings，点击WAN，参照下图进行设置，最后点击Apply。



3.1 硬件环境准备

(6) 机器狗中继连接外网

6、展开Wireless Settings，点击AP Client，查看site Survey表中是否有自己的手机热点名称，如若没有点击SCAN进行扫描，在AP client Parameters表中参照下图进行设置，最后点击Apply。

HLK-RM28E

Operation Mode

Internet Settings

Wireless Settings

Basic

Advanced

Security

WDS

WPS

AP Client

Station List

Statistics

Wireless2 Settings

Firewall

USB

Administration

AP Client Feature

You could configure AP Client parameters here.

AP Client Parameters

SSID

HelloWorld

MAC Address (Optional)

Security Mode

WPA2PSK

Encryption Type

AES

Pass Phrase

Apply

Cancel

SCAN

Site Survey

Ch	SSID	BSSID	Security	Signal(%)	W-Moe	ExtCh	NT
1	weichao	f4 84 8d 97 88 fc	WPA1PSKWPA2PSK/AES	68	11b/g/n	ABOVE	In
1	weichao	f4 84 8d 97 8f c2	WPA1PSKWPA2PSK/AES	70	11b/g/n	ABOVE	In
1	KRN1303	10 5d dc 2f a4 20	WPA1PSKWPA2PSK/AES	50	11b/g/n	NONE	In
1	iTV-X1oT	aa 00 74 e6 e3 46	WPAPSK/TKIPAES	52	11b/g/n	NONE	In
1	jsscsc	ec 60 73 e4 fc 3e	WPA1PSKWPA2PSK/AES	39	11b/g/n	ABOVE	In
1		ee 60 73 74 fc 3e	WPA1PSKWPA2PSK/AES	39	11b/g/n	ABOVE	In
1	0xE997B9E5A587E69CBAE599A8E4BABA	a2 2e 55 9a b0 38	WPA1PSKWPA2PSK/AES	100	11b/g/n	NONE	In
1	ChinaNet-X1oT	9a 00 74 e6 e3 46	WPAPSK/TKIPAES	52	11b/g/n	NONE	In
1	weichao	f4 84 8d 97 8b ab	WPA1PSKWPA2PSK/AES	100	11b/g/n	ABOVE	In
3	Baiyuan	b8 f0 b9 53 b0 f8	WPA1PSKWPA2PSK/AES	100	11b/g/n	ABOVE	In
4	409-DT-2G	80 3f 5d a9 ac 7e	WPA2PSK/AES	65	11b/g/n	ABOVE	In
4	ChinaNet-wmwX	b0 b1 94 ef d2 c3	WPAPSK/TKIPAES	68	11b/g/n	NONE	In
5	ChinaNet-zu2d	14 30 04 00 5d 84	WPA1PSKWPA2PSK/TKIPAES	63	11b/g/n	NONE	In
6	ChinaNet-WrQ4	2c 1a 01 d1 ea 88	WPA1PSKWPA2PSK/TKIPAES	78	11b/g/n	NONE	In
6	weichao	f4 84 8d 97 8e cc	WPA1PSKWPA2PSK/AES	86	11b/g/n	BELOW	In
6		18 3c b7 37 74 99	WPA2PSK/AES	94	11b/g/n	NONE	In

① 输入手机的热点名称

② 可选

③ 输入热点加密类型

④ 输入手机的热点密码

3.1 硬件环境准备

(6) 机器狗中继连接外网

7、重启机器狗，注意中继时热点需一直处于开启状态，电脑连接机器狗的wifi，使用远程软件登录，此时机器狗处于连接外网状态。测试是否有网络，点击浏览器，搜索任意内容，如“百度学术”，能正常显示下图说明正常。



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

3.1 硬件环境准备

3.2 软件环境 (YOLO训练)

3.3 安装和配置CUDA

3.4 开发板环境 (机器狗网址配置)

四、工程文件目录

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

3.2 软件环境（YOLO训练）

首先安装软件YOLOv5步骤如下：

（1）激活虚拟环境

```
(base) C:\Users\lenovo>conda activate Lab2
(Lab2) C:\Users\lenovo>
(base) C:\Users\lenovo>conda activate Lab2
```

conda activate Lab2

（2）进入工程目录下，安装依赖

18 个文件 9 个目录 309,786 字节 500,259,495,936 可用字节	18 个文件 9 个目录
(Lab2) E:\Workspace\Project\yolov5>pip install -U -r requirements.txt	(Lab2) E:\Workspace\Pro
18 个文件 9 个目录 309,786 字节 500,259,495,936 可用字节	18 个文件 9 个目录

pip install -r requirments.txt

（3）验证安装成功

```
Successfully installed MarkupSafe-2.1.3 Pillow-10.0.0 PyYAML-6.0.1 absl-py-1.4.0 asttokens-2.2.1 backcall-0.2.0 cachetocSucc
ls-5.3.1 certifi-2023.7.22 charset-normalizer-3.2.0 colorama-0.4.6 contourpy-1.1.0 cycler-0.11.0 decorator-5.1.1 executils-5
ng-1.2.0 filelock-3.12.2 fonttools-4.41.1 gitdb-4.0.10 gitpython-3.1.32 google-auth-2.22.0 google-auth-oauthlib-1.0.0 grng-1
pcio-1.56.2 idna-3.4 importlib-metadata-6.8.0 importlib-resources-6.0.0 ipython-8.12.2 jedi-0.18.2 jinja2-3.1.2 kiwisolvpcid
er-1.4.4 markdown-3.4.4 matplotlib-3.7.2 matplotlib-inline-0.1.6 mpmath-1.3.0 networkx-3.1 numpy-1.24.4 oauthlib-3.2.2 der-1
pencv-python-4.8.0.74 packaging-23.1 pandas-2.0.3 parso-0.8.3 pickleshare-0.7.5 prompt-toolkit-3.0.39 protobuf-4.23.4 pspend
util-5.9.5 pure-eval-0.2.2 pyasn1-0.5.0 pyasn1-modules-0.3.0 pygments-2.15.1 pyparsing-3.0.9 python-dateutil-2.8.2 pytz-util
2023.3 requests-2.31.0 requests-oauthlib-1.3.1 rsa-4.9 scipy-1.10.1 seaborn-0.12.2 six-1.16.0 smmap-5.0.0 stack-data-0.6.2023
.2 sympy-1.12 tensorboard-2.13.0 tensorboard-data-server-0.7.1 thop-0.1.1.post2209072238 torch-2.0.1 torchvision-0.15.2 .2 s
tqdm-4.65.0 traitlets-5.9.0 typing-extensions-4.7.1 tzdata-2023.3 urllib3-1.26.16 wcwidth-0.2.6 werkzeug-2.3.6 zipp-3.16.1 tqdm
.2
Successfully installed MarkupSafe-2.1.3 Pillow-10.0.0 PyYAML-6.0.1 absl-py-1.4.0 asttokens-2.2.1 backcall-0.2.0 cachetocSucc
ls-5.3.1 certifi-2023.7.22 charset-normalizer-3.2.0 colorama-0.4.6 contourpy-1.1.0 cycler-0.11.0 decorator-5.1.1 executils-5
```

3.2 软件环境 (YOLO训练)

注意: PyTorch可能被替换为CPU版, 训练速度很慢
配置虚拟环境如下:

(1) 新建一个脚本查看是否为gpu版本

```
import torch
print(torch.__version__)
print(torch.cuda.is_available())
```

3 x

C:\Users\15210\.conda\envs\yolov5-2.2.2+cu121
True

```
import torch
print(torch.__version__)
print(torch.cuda.is_available())
```

```
yolov5-7.0 D:\YSW\人工智能-机械狗\手势识别
1 import torch
2 print(torch.__version__)
3
4 print("Is CUDA available:", torch.cuda.is_available())
5 print("CUDA version:", torch.version.cuda)
```

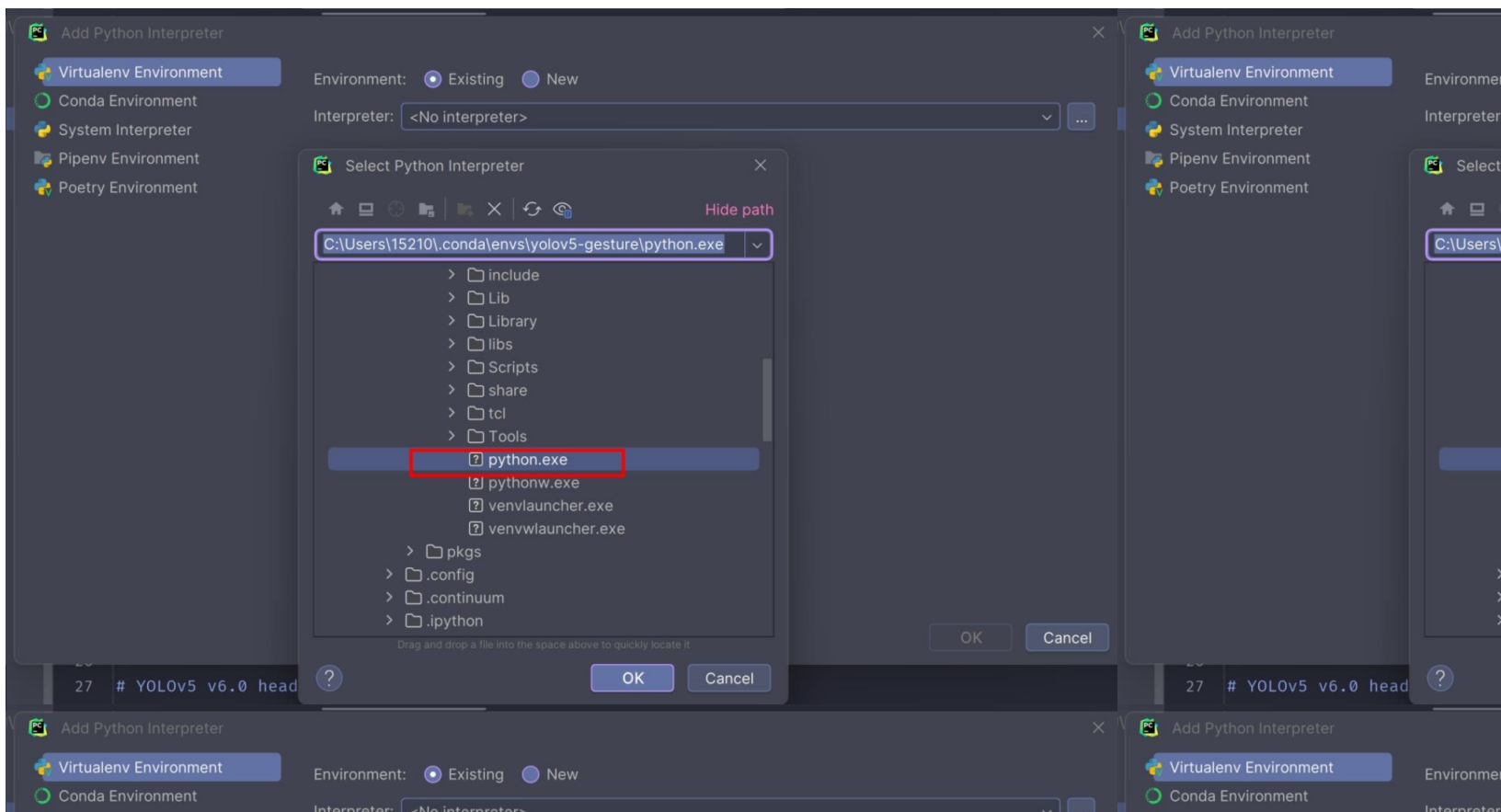
1 x

2.2.2+cpu
Is CUDA available: False
CUDA version: None

```
import torch
print(torch.__version__)
print("Is CUDA available", torch.cuda.is_ava
      ailable())
print("CUDA version: ", torch.version.cuda )
```

3.2 软件环境（YOLO训练）

（2）选择Add Local Interpreter——>Load Environments——>选择自己的环境



选择OK，虚拟环境配置成功。

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

3.1 硬件环境准备

3.2 软件环境 (YOLO训练)

3.3 安装和配置CUDA

3.4 开发板环境 (机器狗网址配置)

四、工程文件目录

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

3.3 安装和配置CUDA

Step1

1. 检查pytorch是否满足cuda的版本

Step2

1. 在官网下载对应的cuda和cuda Toolkit

Step3

1. 安装cuDNN对应的版本
2. 下载对应的cuDNN和cuda版本之后，解压，然后将cuDNN中的文件全部复制到cuda中对应的文件。

Step4

1. 在pycharm中测试

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

3.1 硬件环境准备

3.2 软件环境 (YOLO训练)

3.3 安装和配置CUDA

3.4 开发板环境 (机器狗网址配置)

四、工程文件目录

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

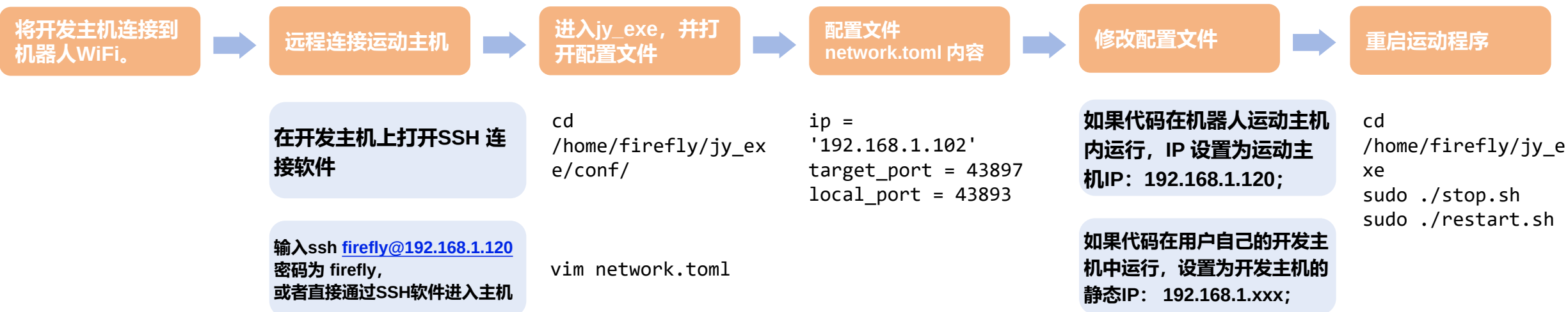
七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

3.4 开发板环境（机器狗网址配置）

用户可通过SSH 远程连接到运动主机。



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

4.1 PC

4.2 Ascend

4.3 ModelArts

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

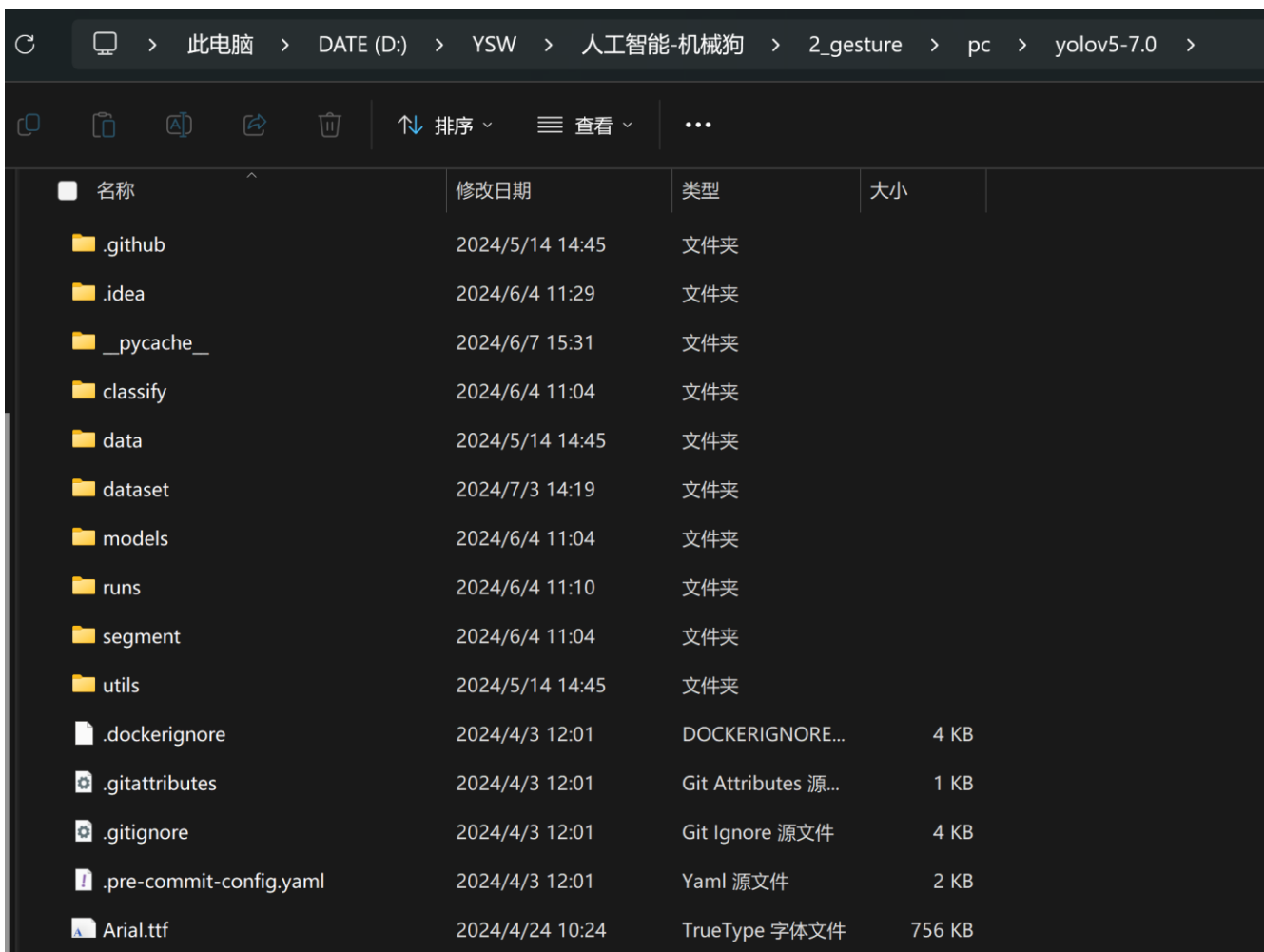
七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

4.1 PC

PC端yolov5的文件夹，
主要内容为dataset中存放的数据文件，
其他的地方不需要修改。
train.py是主要的训练文件



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

4.1 PC

4.2 Ascend

4.3 ModelArts

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

4.2 Ascend

本章节主要展示的是在开发板上所有的文件，以及每个文件所对应的作用。本章节主要展示开发板上所有的文件及其对应的作用，以便更好地理解整个项目的结构和功能。

```
(base) root@davinci-mini:/opt/lab/GestureRecognition# tree -I "__pycache__|*.pyc" -L 10
.
├── camera
│   ├── HKcamera.py
│   └── det_utils.py
├── command
│   ├── __init__.py
│   └── udp_command.py
├── gesture_main.py
├── gesture_models
│   ├── fusion_result.json
│   ├── gesture.om
│   ├── model.onnx
│   └── predefined_classes.txt
├── getAndSavePicture.py
├── robotstatuswatcher
│   ├── __init__.py
│   └── listener.py
├── sendcommand
│   ├── SendToCommand.py
│   ├── __init__.py
│   └── heartbeat.py
├── socketnetwork
│   ├── __init__.py
│   └── network_utils.py
├── speeds
│   ├── __init__.py
│   └── sportsspeed.py
└── threading_utils
    └── ThreadTemplates.py
```

4.2 Ascend开发板下文件目录

Command 文件夹

- ◆ **udp_command.py**: 该文件用于存放指令码。它定义了各种控制命令，这些命令通过UDP协议发送到机器狗，以实现对其精确控制。

SendCommand 文件夹

- ◆ **heartbeat.py**: 该文件用于发送心跳包。心跳包是一种定期发送的信号，用于检测系统或设备是否在线和正常工作。通过发送心跳包，可以确保机器狗与控制系统之间的通信连接是稳定和可靠的。
- ◆ **SendToCommand.py**: 该文件负责发送指令。它包含了一系列函数，用于将指令发送到机器狗，确保指令能够正确地传达和执行。

4.2 Ascend开发板下文件目录

SocketNetwork 文件夹

- ◆ **network_utils.py**: 该文件是与机器狗建立通信的脚本。它包含了各种网络通信的实用函数，用于初始化和维护与机器狗的通信连接，使得控制指令能够通过网络传输到机器狗。

Speeds 文件夹

- ◆ **sportspeed.py**: 该文件定义了机器狗运动的速度方程。通过调整速度参数，可以控制机器狗的运动速度，使其能够按照预定的轨迹和速度执行任务。

ThreadingUtils 文件夹

- ◆ **ThreadTemplates.py**: 该文件定义了线程模板。线程是并发执行的基本单位，通过使用线程，可以提高系统的响应速度和处理效率。该文件中的模板可以用于创建和管理多个线程，以实现并行处理

4.2 Ascend开发板下文件目录

robotstatuswatcher文件夹

- ◆ **listener.py**: 该文件定义了监听机器狗状态的函数。

gesture_models文件夹

- ◆ 该文件夹中存放了用来推理的模型文件。

camera文件夹

- ◆ • **HKcamera.py**: 该文件是调用摄像头的文件。

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

4.1 PC

4.2 Ascend

4.3 ModelArts

五、数据工程 (1学时)

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

4.3 ModelArts



yolov5与PC端相似，output用来存放训练好的模型。

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

5.1配置开发板的HK驱动

5.2编写数据采集脚本

5.3配置PC端数据标注环境

5.4标注数据集

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

5.1 配置开发板的HK驱动

Step 1

下载并安装MVS驱动文件（Linux）
海康威视官网下载链接：
<https://www.hikrobotics.com/cn/machinevision/service/download?module=0>

Step 2

打开MobaXterm
远程控制，进入开发板安装驱动指令：

```
dpkg -i MVS-3.0.1_aarch64_20240422.deb
```

Step 3

在代码中使用需要加入包的路径：

```
sys.path.append("/opt/MVS/Samples/aarch64/Python/MvImport")  
from MvCameraControl_class import*
```

Step 4

使用摄像头的代码

```
python HKcamera.py
```

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

5.1配置开发板的HK驱动

5.2编写数据采集脚本

5.3配置PC端数据标注环境

5.4标注数据集

六、模型本地化训练与推理 (1学时)

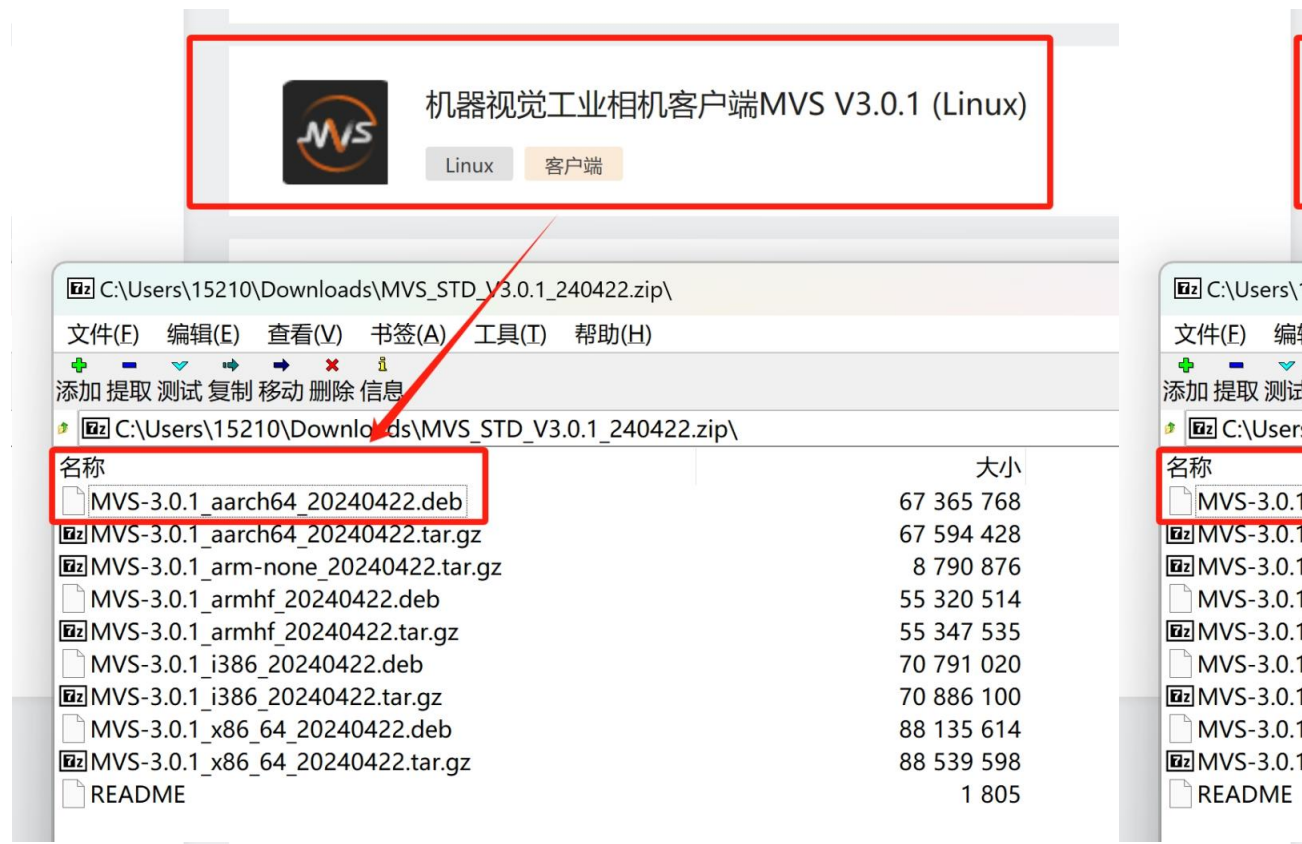
七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

5.1 配置开发板的HK驱动

下载并安装MVS驱动文件（Linux）



海康威视官网下载链接:

<https://www.hikrobotics.com/cn/machinevision/service/download?module=0>

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

5.1配置开发板的HK驱动

5.2编写数据采集脚本

5.3配置PC端数据标注环境

5.4标注数据集

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

5.2编写数据采集脚本

数据采集的方式是通过调用摄像头的SDK来获取图片信息，然后保存到开发板中，脚本名为getAndSavePicture.py。

```
1. import cv2
2. import time
3. import sys
4. sys.path.append('camera/')
5. from HKcamera import getImage
6. from det_utils import get_labels_from_txt, letterbox, scale_coords, nms, draw_bbox
7. from threading_utils.ThreadTemplates import *
8. from sendcommand import SendToCommand, heartbeat
9. from socketnetwork import network_utils
10. from command.udp_command import *
11. from speeds.sportspeed import *
12. from robotstatuswatcher.listener import *
13. from HandDistance.ridge_np import *
14. def save_image(img):
15.     image_name = f'train_images/demo/{time.time()}.jpg'
16.     cv2.imwrite(image_name, img)
17.     print(f'图像已保存为: {image_name}')
```

用于获取图像

用于目标检测模型的前后处理

导入定义的机器狗相关的包

线程的模板与基本轴指令的发送

发送简单指令与心跳包

通信函数

存放各种结构体与状态数据、指令

运动精准控制的基础版模型

监听机器狗状态

测量距离

5.2编写数据采集脚本

```
1. print("按 's' 保存图像，按 'q' 退出程序。")

2. # 主循环

3. #通过 getImage() 获取图像。

4. #使用 cv2.imshow 显示图像。

5. #使用 cv2.waitKey(1) 检测按键事件，如果按下 's'，则调用 save_image 保存图像；如果按下 'q'，则退出循环。

6. while True:

7.     img = getImage() # 获取图像

8.     cv2.imshow('Camera Feed', img) # 显示图像

9.     key = cv2.waitKey(1) & 0xFF # 检测按键事件

10.    if key == ord('s'):

11.        save_image(img) # 保存图像

12.    elif key == ord('q'):

13.        break # 退出循环

14. # 清理工作

15. cv2.destroyAllWindows()
```

目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

5.1配置开发板的HK驱动

5.2编写数据采集脚本

5.3配置PC端数据标注环境

5.4标注数据集

六、模型本地化训练与推理 (1学时)

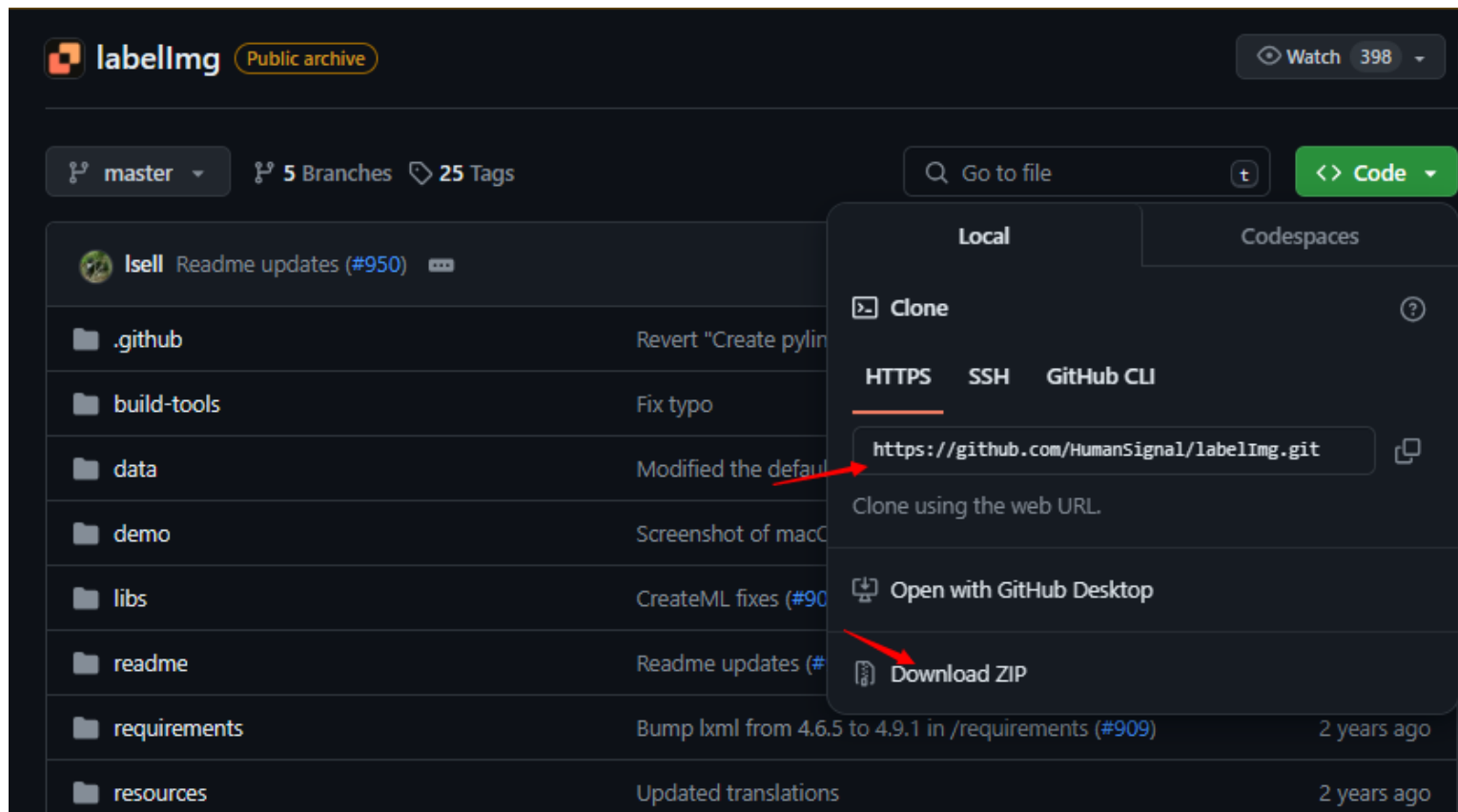
七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

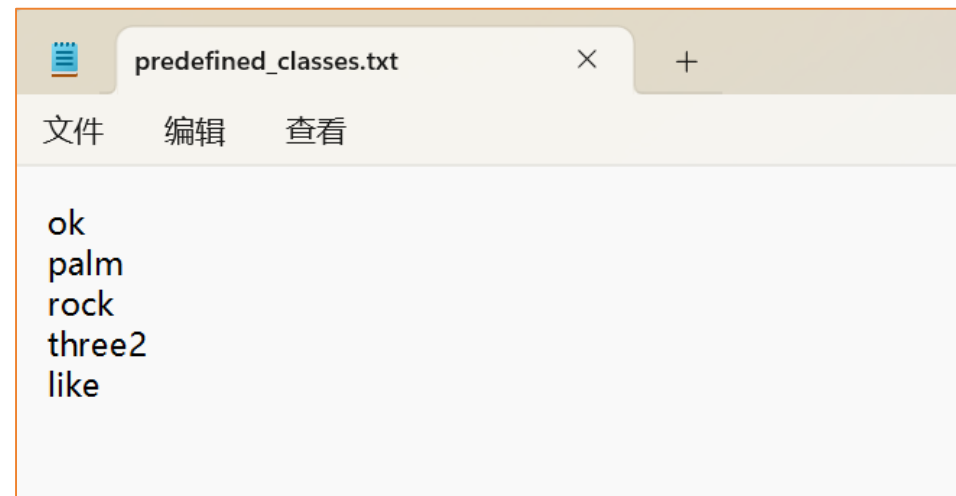
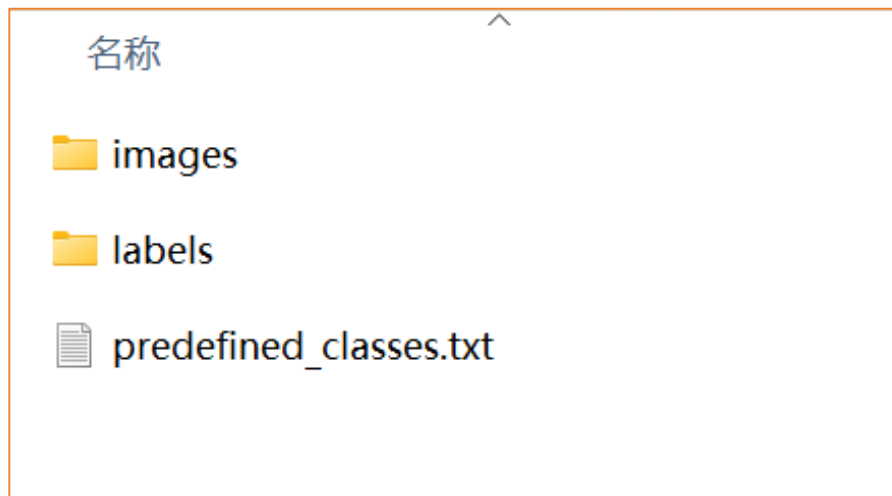
5.3配置PC端数据标注环境

下载数据标注工具labelimg: <https://github.com/HumanSignal/labelImg>



5.3配置PC端数据标注环境

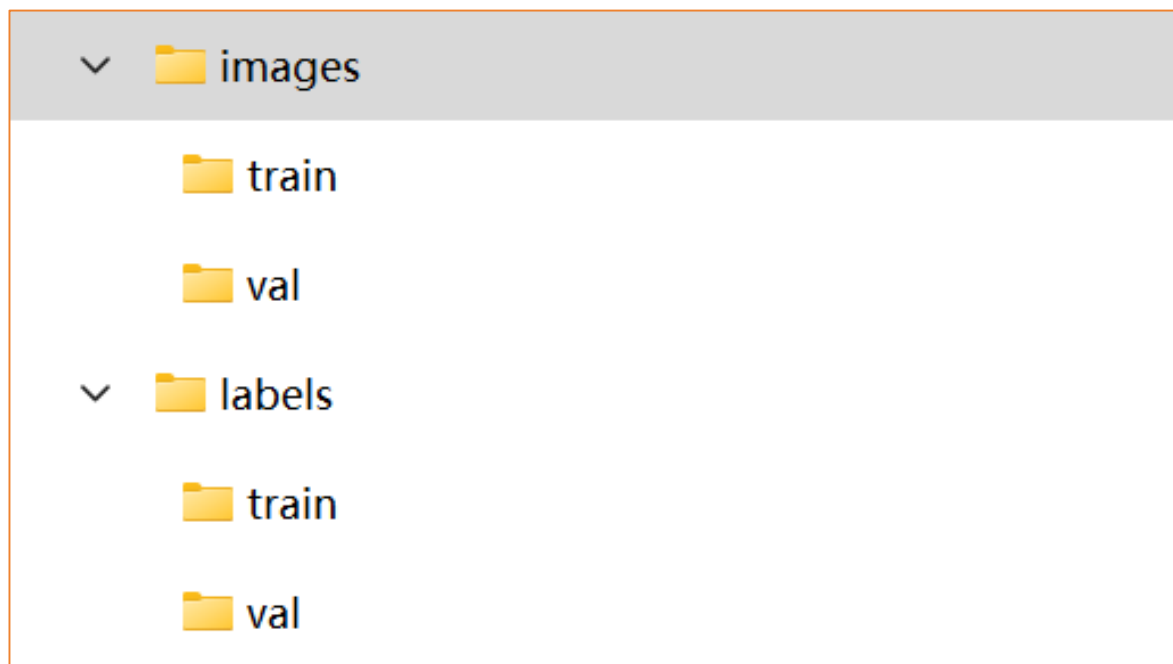
建立两个文件夹，分别是images用来存放训练集train的图片和验证集val的图片， labels用来存放标注完train中对应的图片数据标签和val中的对应的图片数据标签， 以及predefined_classes.txt存放标签类别。



/images/train对应labelImg中的存放目录，而/labels/train对应labelImg的改变存放目录，用于存放标注完图片的数据标签。

5.3配置PC端数据标注环境

/images/train对应labelImg中的存放目录，而/labels/train对应labelImg的改变存放目录，用于存放标注完图片的数据标签。



目录

一、实验前置准备 (0.5学时)

二、实验过程描述

三、实验环境准备 (0.5学时)

四、工程文件目录

五、数据工程 (1学时)

5.1配置开发板的HK驱动

5.2编写数据采集脚本

5.3配置PC端数据标注环境

5.4标注数据集

六、模型本地化训练与推理 (1学时)

七、在线ModelArts训练与模型转换及应用 (1学时)

八、逻辑实现 (4学时)

九、运行示例

5.4标注数据集

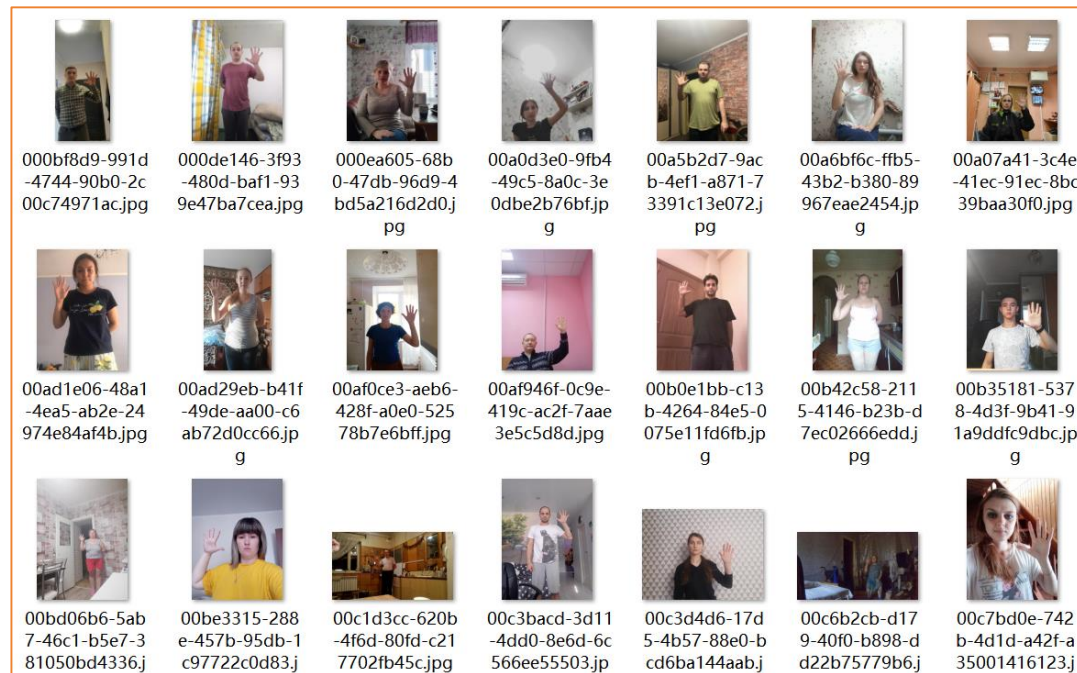
采集数据

使用labellmg进行数
据集标注

数据标定

数据预览

在代码高亮上下载已有不同类型图片，
选择其中几种手势，作为手势识别的数据
来源，以其中一类palm为例，可看到如下
图所示。



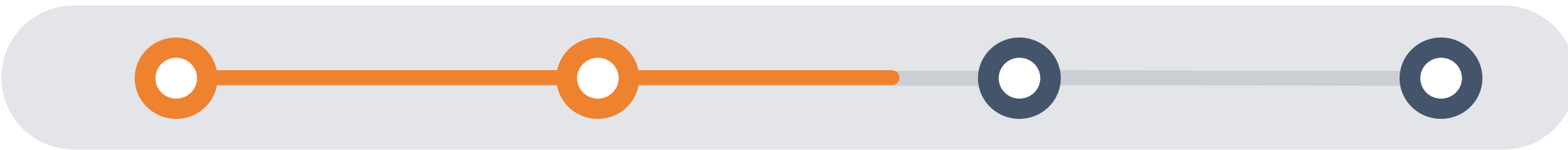
5.4标注数据集

采集数据

使用labelImg进行数
据集标注

数据标定

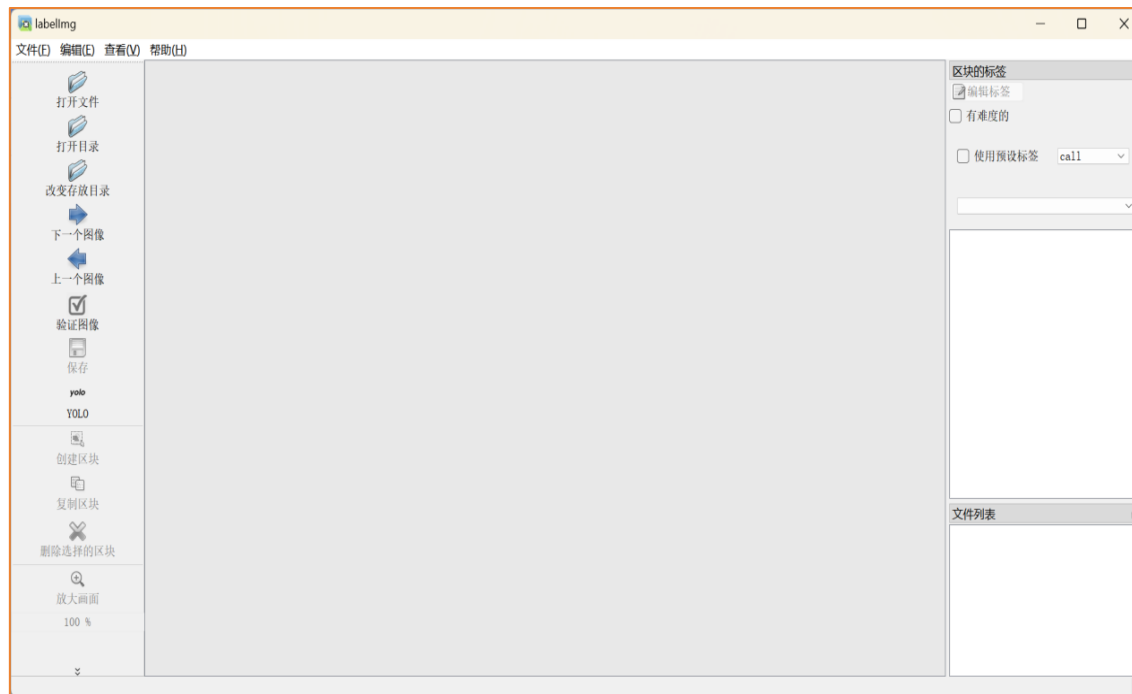
数据预览



1、打开anaconda命令行界面

2、同时在predefined_classes.txt中，设置所分类的标签名称；在data文件夹中分别新建images、label文件夹，同时在images文件夹中，新建train文件夹，里面存放训练集手势照片，与之对应应在label文件夹中，新建train文件夹，里面用来存放标注好的数据集信息；同时在images文件夹中，再新建val文件夹，用来存放验证集手势照片，与之对应应在label文件夹中，新建val文件夹，里面用来存放标注好的数据集信息

3、在“打开目录”中打开刚刚的data/images/train文件夹，在“改变存放目录”中更换保存路径为data/label/train文件夹



5.4标注数据集

采集数据

使用labelImg进行数
据集标注

数据标定

数据预览

下载关于labelImg的安装包后，进入labelImg-master所在文件夹地址

```
(base) PS E:\> cd E:\Desktop\亿生为\labelImg-master
(base) PS E:\Desktop\亿生为\labelImg-master> python labelImg.py
E:\Desktop\亿生为\labelImg-master\labelImg.py:73: DeprecationWarning: sipPyTypeDict() is deprecated, the extension modul
e should use sipPyTypeDictRef() instead
  class MainWindow(QMainWindow, WindowMixin):
[('palm', [(207, 279), (310, 279), (310, 406), (207, 406)], None, None, False)]
```

后输入python labelImg.py进入数据标注页面，

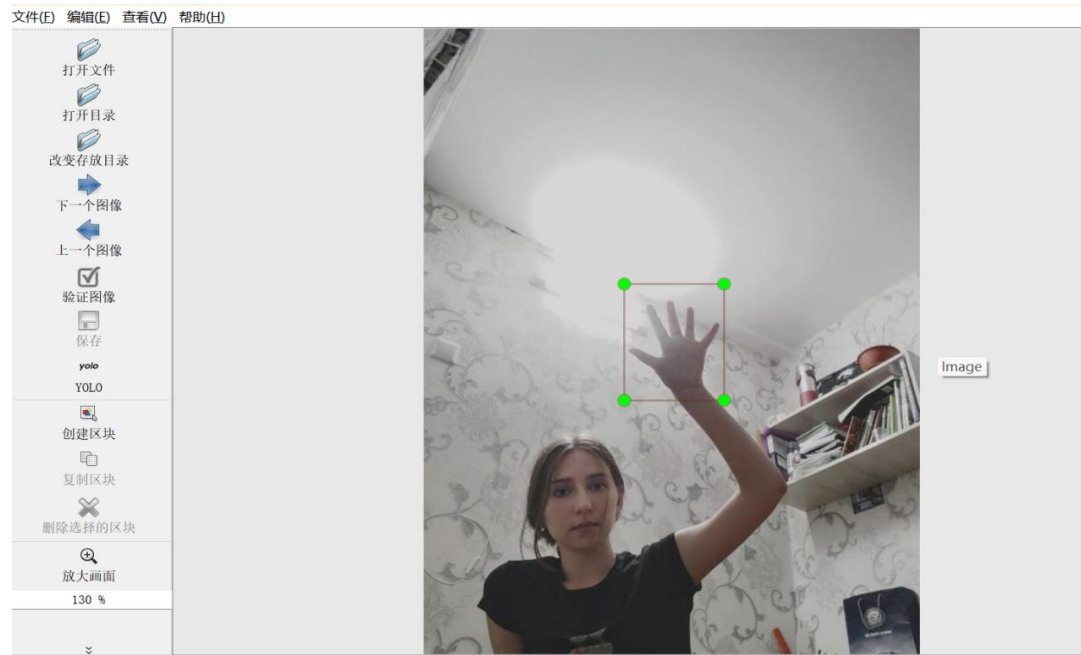
点击“创造区块”，然后出现十字架，进行拖动框住标注区域

然后输入标签，点击palm，再点击左侧的“保存”标签

在data/label/train里面可以看到我们的标签，在此介绍labelImg的三种标签格式：

- (1) PascalVOC标签格式，保存为xml文件
- (2) YOLO标签格式，保存为txt文件
- (3) CreateML标签格式，保存为json文件

点击“下一个图像”继续标注下一张



5.4标注数据集

采集数据

使用labelImg进行数
据集标注

数据标定

数据预览

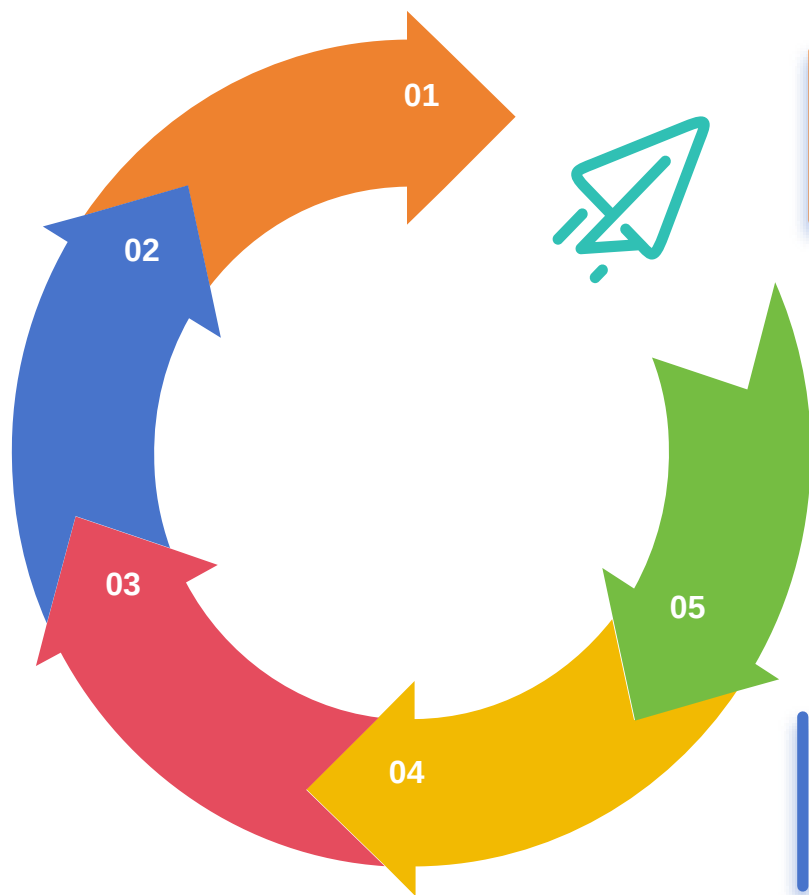
数据标注完成后，若标注图片是
`\labeldata\images\train` 中，则可到
`\labeldata\labels\train`中查看标签文件，`images`
中存放训练集train图，`labels`中存放训练集train
所对应的数据标签txt文件，可看到其文件名对
应图片名称，txt中的四列数据分别对应数据标
注区块各个含义，具体如下图所示。



目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
 - 6.1 训练流程
 - 6.2 训练参数详细介绍与修改
 - 6.3 模型训练
 - 6.4 模型测试
 - 6.5 模型转换
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
- 九、运行示例

6.1 训练流程



模型训练

- a.详细流程：调用Yolov5训练文件，读取制作的数据集，来训练我们的模型。
- b.实验注意点：超参数的设置，以及数据集文件结构。
- c.重点知识：Yolov5训练过程，超参数设置。
- d.完成本步骤后输出：在runs/train/exp文件夹下保存的pt模型。

开发板相机设置

- a.详细流程：打开工业相机保护盖，连接工业相机和昇腾开发板，调整HKcamera.py代码参数和补光灯亮度，使工业相机清晰成像。
- b.实验注意点：HKcamera.py代码内容调整后，保存后才能生效。
- c.重点知识：昇腾开发板上工业相机的调整。

模型测试

- a.详细流程：调用detect.py文件的参数，对模型进行验证，
- b.实验注意点：需要指定测试的图片和pt模型路径。
- c.重点知识：检测验证。
- d.完成本步骤后输出：在指定的目录中保存检测后的图片。

模型转换

- a.详细流程：在PC机上将.pt模型转换为.onnx模型；再在PC上将onnx模型转换为om模型。
- b.实验注意点：om模型转换命令参数。
- c.重点知识：Yolov5转换onnx模型流程；onnx模型转换为om模型的命令使用。
- d.完成本步骤后输出：可以在昇腾开发板上推理的.om模型。

开发板推理

- a.详细流程：编写昇腾开发板推理.om模型的代码。
- b.实验注意点：代码中MindX SDK调用时需要方便拓展。
- c.重点知识：昇腾开发板推理.om的方法
- d.完成本步骤后输出：运行代码后会输出目标检测后的结果图。

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
 - 6.1 训练流程
 - 6.2 训练参数详细介绍与修改
 - 6.3 模型训练
 - 6.4 模型测试
 - 6.5 模型转换
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
- 九、运行示例

6.2训练参数详细介绍与修改

新建一个dataset文件夹，在里面除了存放数据之外再新建一个data.yaml，对数据集进行配置

```
train: dataset/images/train
```

```
val: dataset/images/val
```

```
test:
```

```
# number of classes
```

```
nc: 5
```

训练种类数

```
# class names
```

```
names:
```

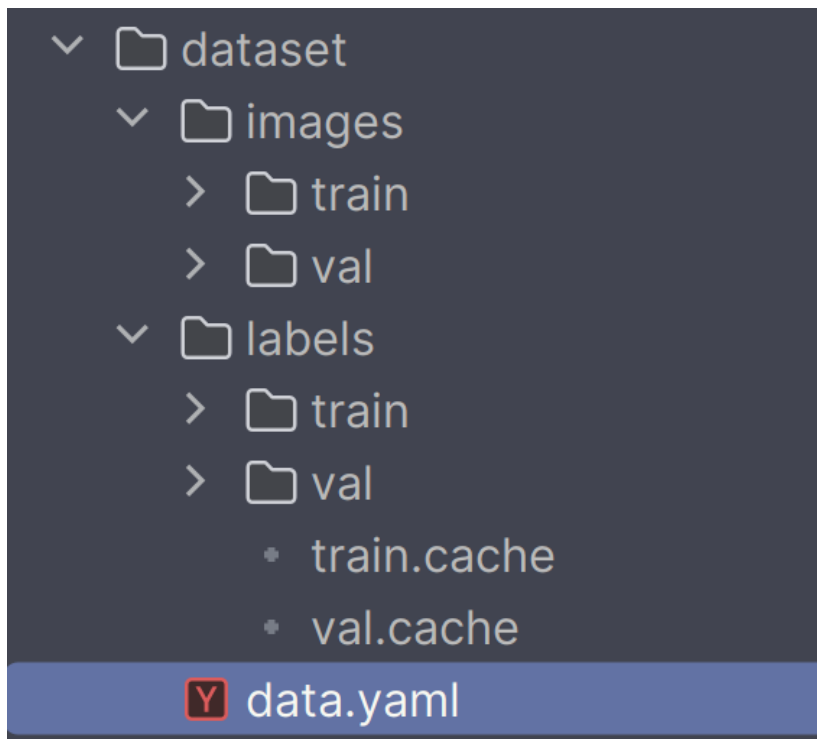
```
['fist', 'one', 'peace', 'rock', 'three2']
```

训练种类名称



6.2训练参数详细介绍与修改

整理数据集文件结构，确定和如下图示相同。并且labels下需要存放classes.txt文件。



6.2训练参数详细介绍与修改

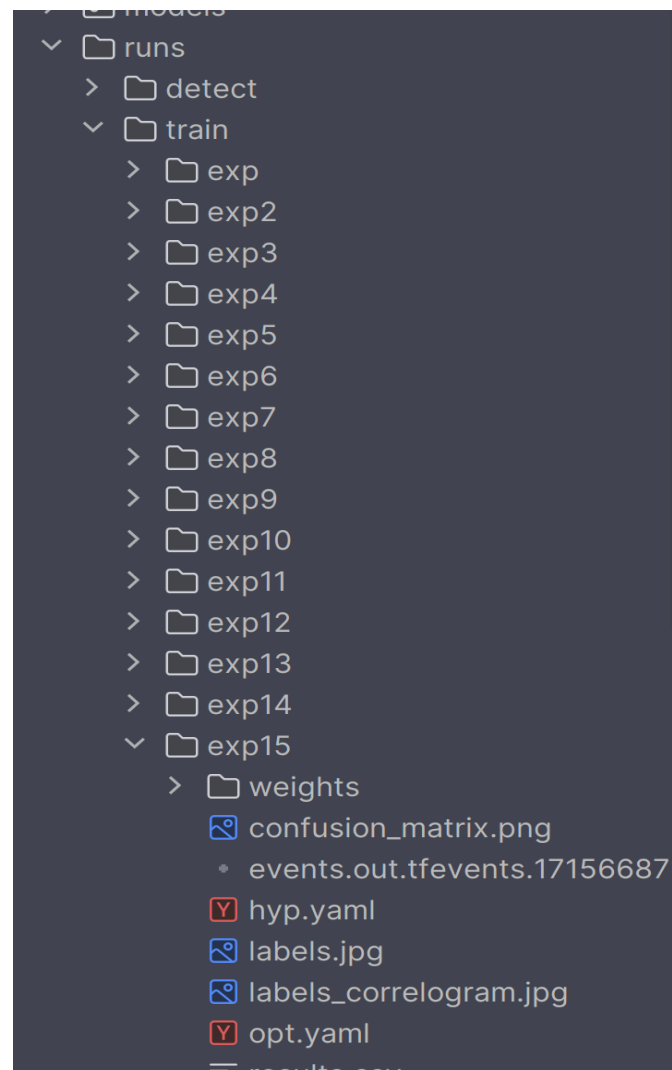
修改train.py文件的参数，其中主要包括--weights[预训练权重路径]、--cfg[模型结构配置]、--data[数据集配置文件]、--epochs[训练轮数]、--batch-size[批数据大小]。

本文所采取的参数如下：

```
python train.py --weights yolov5s.pt --cfg yolov5s.yaml --data dataset/data.yaml --epochs 100  
--batch-size 4
```


6.2训练参数详细介绍与修改

运行train.py,等到训练完成后就可以在runs文件下的train目录找到训练的结果



目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
 - 6.1 训练流程
 - 6.2 训练参数详细介绍与修改
 - 6.3 模型训练
 - 6.4 模型测试
 - 6.5 模型转换
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
- 九、运行示例

6.3模型训练

本文的参数选择:

```
python train.py --img 640 --batch 16 --epoch 500 --data dataset/data.yaml --cfg  
models/yolov5s.yaml --weights yolov5s.pt
```

```
2      -1 1      18816 models.common.C3      [64, 64, 1]  
3      -1 1      73984 models.common.Conv [64, 128, 3, 2]  
4      -1 2      115712 models.common.C3      [128, 128, 2]  
5      -1 1      295424 models.common.Conv [128, 256, 3, 2]  
6      -1 3      625152 models.common.C3      [256, 256, 3]  
7      -1 1      1180672 models.common.Conv [256, 512, 3, 2]  
8      -1 1      1182720 models.common.C3      [512, 512, 1]  
9      -1 1      656896 models.common.SPPF [512, 512, 5]  
10     -1 1      131584 models.common.Conv [512, 256, 1, 1]  
11     -1 1      0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']  
12     [-1, 6] 1      0 models.common.Concat [1]  
13     -1 1      361984 models.common.C3      [512, 256, 1, False]  
14     -1 1      33024 models.common.Conv [256, 128, 1, 1]  
15     -1 1      0 torch.nn.modules.upsampling.Upsample [None, 2, 'nearest']  
16     [-1, 4] 1      0 models.common.Concat [1]  
17     -1 1      90880 models.common.C3      [256, 128, 1, False]  
18     -1 1      147712 models.common.Conv [128, 128, 3, 2]  
19     [-1, 14] 1      0 models.common.Concat [1]  
20     -1 1      296448 models.common.C3      [256, 256, 1, False]  
21     -1 1      590336 models.common.Conv [256, 256, 3, 2]  
22     [-1, 10] 1      0 models.common.Concat [1]  
23     -1 1      1182720 models.common.C3      [512, 512, 1, False]  
24     [17, 20, 23] 1      26970 models.yolo.Detect [5, [[10, 13, 16, 30, 33, 23], [30, 61, 62,  
45, 59, 119], [116, 90, 156, 198, 373, 326]], [128, 256, 512]]  
YOLOv5s summary: 214 layers, 7033114 parameters, 7033114 gradients, 16.0 GFLOPs  
Transferred 342/349 items from yolov5s.pt  
AMP: checks passed  
optimizer: SGD(lr=0.01) with parameter groups 57 weight(decay=0.0), 60 weight(decay=0.0005), 60 bias  
train: Scanning D:\YSW\人工智能-机械狗\2_gesture\pc\yolov5-7.0\dataset\labels\train...: 0% | 0/5 00:00
```

```
AttributeError: FreeTypeFont object has no attribute 'getsize'  
Class      Images  Instances  P      R      mAP50  mAP50-95: 100% | 1/1 00:00  
all        2          1        0      0      0      0  
Epoch      GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size  
3/499      1.23G    0.1275    0.0289    0.05135   8          640: 100% | 1/1 00:00  
Class      Images  Instances  P      R      mAP50  mAP50-95: 100% | 1/1 00:00  
all        2          1        0      0      0      0  
Epoch      GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size  
4/499      1.23G    0.0808    0.02902   0.03415   8          640: 100% | 1/1 00:00  
Class      Images  Instances  P      R      mAP50  mAP50-95: 100% | 1/1 00:00  
all        2          1        0      0      0      0  
Epoch      GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size  
5/499      1.23G    0.1234    0.02895   0.04959   8          640: 100% | 1/1 00:00  
Class      Images  Instances  P      R      mAP50  mAP50-95: 100% | 1/1 00:00  
all        2          1        0      0      0      0  
Epoch      GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size  
6/499      1.23G    0.1158    0.03143   0.05045  10         640: 100% | 1/1 00:00  
Class      Images  Instances  P      R      mAP50  mAP50-95: 100% | 1/1 00:00  
all        2          1        0      0      0      0  
Epoch      GPU_mem  box_loss  obj_loss  cls_loss  Instances  Size  
7/499      1.23G    0.1126    0.02856   0.05895   9          640: 100% | 1/1 00:00  
Class      Images  Instances  P      R      mAP50  mAP50-95: 100% | 1/1 00:00  
all        2          1        0      0      0      0
```

训练完成之后在runs\train\exp17的文件夹下可以看到训练的文件

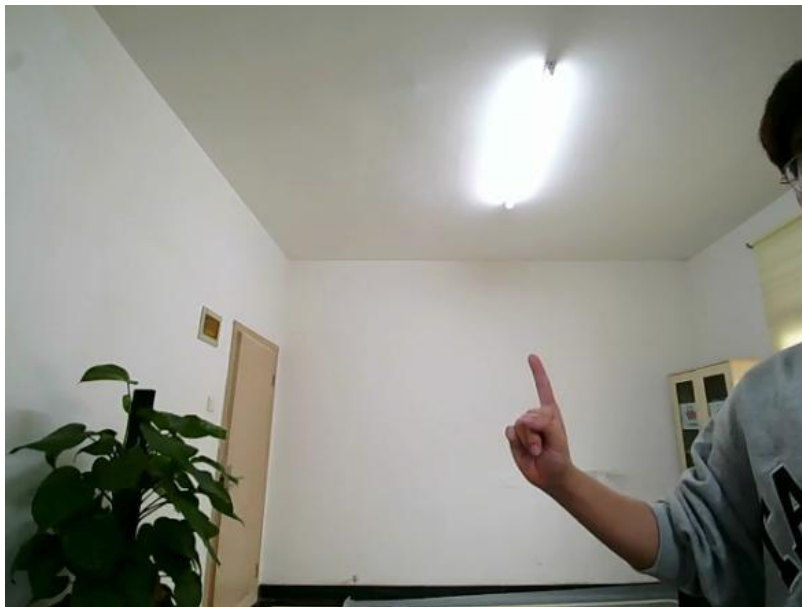
Results saved to runs\train\exp17

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
 - 6.1 训练流程
 - 6.2 训练参数详细介绍与修改
 - 6.3 模型训练
 - 6.4 模型测试
 - 6.5 模型转换
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
- 九、运行示例

6.4模型测试

首先在yolov5-7.0\runs\train\exp15\weight\1.png中放入测试图片



修改detect.py文件的参数，对模型进行验证，其中主要包括--weights[训练权重路径]、--source[测试图片目录]、--data[数据集配置文件]

6.4模型测试

本文所采取的参数如下：

```
python detect.py --weights runs/train/exp15/weights/best.pt --source "D:/YSW/人工智能-机械狗/2_gesture/pc/yolov5-7.0/runs/train/exp15/weights/1.jpg" --data dataset/data.yaml
```

运行成功之后

```
(yolov5-gesture) PS D:\YSW\人工智能-机械狗\2_gesture\pc\yolov5-7.0> python detect.py --weights runs/train/exp15/weights/best.pt --source "D:/YSW/人工智能-机械狗/2_gesture/pc/yolov5-7.0/runs/train/exp15/weights/1.jpg" --data dataset/data.yaml
detect: weights=['runs/train/exp15/weights/best.pt'], source=D:/YSW/-/2_gesture/pc/yolov5-7.0/runs/train/exp15/weights/1.jpg, data=dataset/data.yaml, imgsz=[640, 640], conf_thres=0.25, iou_thres=0.45, max_det=1000, device=, view_img=False, save_txt=False, save_conf=False, save_crop=False, nosave=False, classes=None, agnostic_nms=False, augment=False, visualize=False, update=False, project=runs\detect, name=exp, exist_ok=False, line_thickness=3, hide_labels=False, hide_conf=False, half=False, dnn=False, vid_stride=1
YOLOv5 2024-4-3 Python-3.9.19 torch-2.2.2+cu121 CUDA:0 (NVIDIA GeForce RTX 3050 Ti Laptop GPU, 4096MiB)

Fusing layers...
YOLOv5s summary: 157 layers, 7061368 parameters, 0 gradients, 15.9 GFLOPs
image 1/1 D:\YSW\-\2_gesture\pc\yolov5-7.0\runs\train\exp15\weights\1.jpg: 480x640 1 one, 80.2ms
Speed: 0.0ms pre-process, 80.2ms inference, 170.1ms NMS per image at shape (1, 3, 640, 640)
Results saved to runs\detect\exp16
```

6.4模型测试

结果保存在了下面路径 runs\detect\exp16



可以看到已经手势识别成功。

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
 - 6.1 训练流程
 - 6.2 训练参数详细介绍与修改
 - 6.3 模型训练
 - 6.4 模型测试
 - 6.5 模型转换
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
- 九、运行示例

6.5 模型转换

1. 首先将模型转换为onnx

`python export.py --weights runs/model.pt --include onnx`
输入之后会看到有

```
Export complete (1.5s)
Results saved to D:\YSW\-\2_gesture\pc\yolov5-7.0\runs
Detect:      python detect.py --weights runs\model.onnx
Validate:    python val.py --weights runs\model.onnx
PyTorch Hub: model = torch.hub.load('ultralytics/yolov5', 'custom', 'runs\model.onnx')
Visualize:   https://netron.app
(yolov5-gesture) PS D:\YSW\人工智能-机械狗\2_gesture\pc\yolov5-7.0> |
```

将训练得到的ONNX模型文件，修改名字为model.onnx（方便区别），上传到开发板

```
(base) root@davinci-mini:/opt# ll
total 27888
drwxr-xr-x  2 root root    4096 Apr 12 17:10 ./
drwxr-xr-x 24 root root    4096 Apr  8 2022 ../
-rw-r--r--  1 root root 28545135 Apr 12 17:10 model.onnx
```

运行下面命令

`atc --model=model.onnx --framework=5 --output=model --soc_version=Ascend310B4`

若提示如下信息，则说明模型转化成功。

```
(base) root@davinci-mini:/opt# atc --model=model.onnx --framework=5 --output=model --soc_version=Ascend310B4
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

执行动作

后台输入动作序列

线程模板

建立速度模型

动作指令

创建指令头

发送心跳包

建立通信

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
 - 7.1 ModelArts介绍
 - 7.2 操作以及演示
 - 7.3 自定义训练教程——手势识别
- 八、逻辑实现 (学时)
- 九、运行示例

7.1 ModelArts介绍

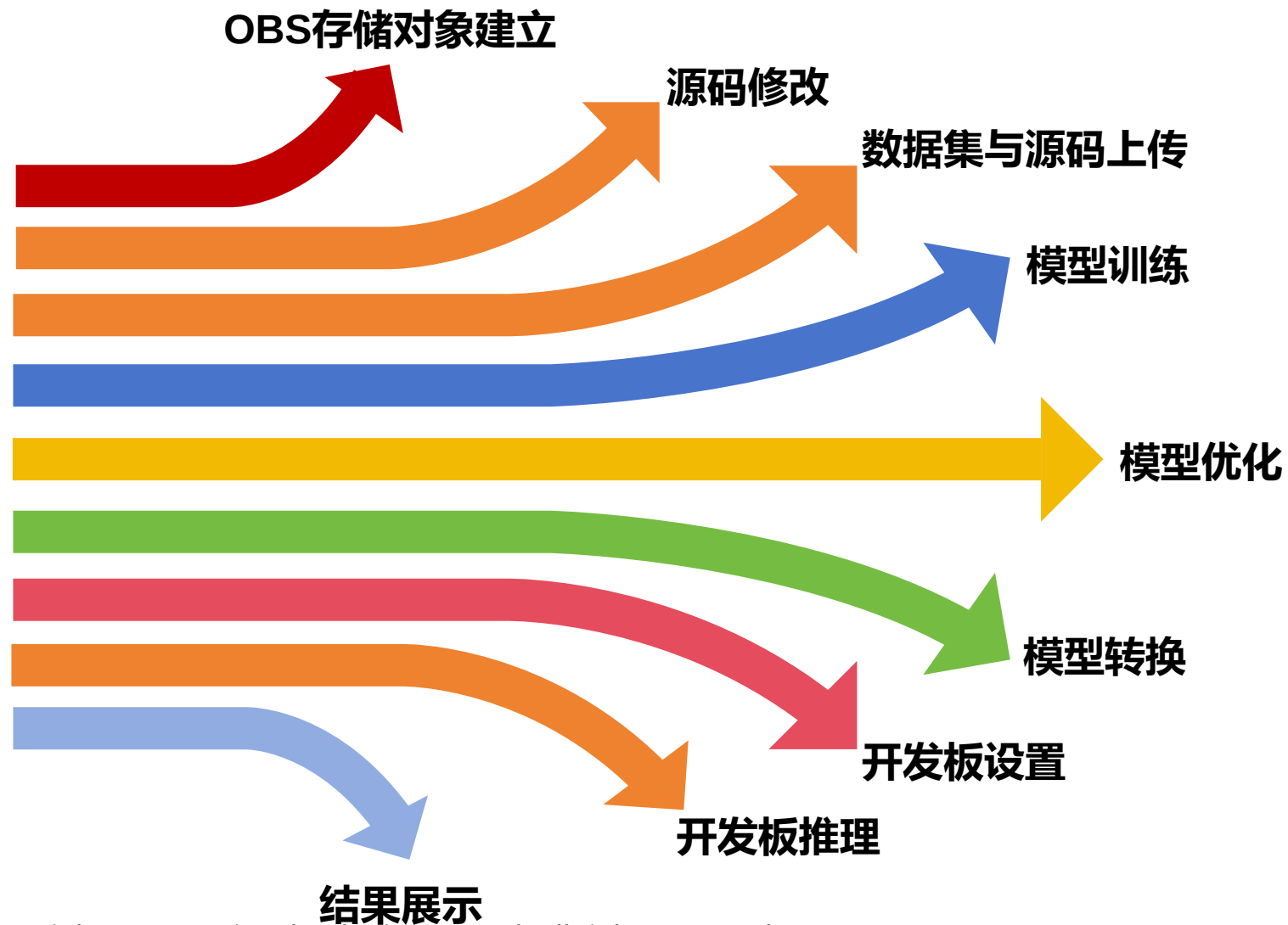
AI开发平台ModelArts是面向开发者的一站式AI开发平台，可快速创建和部署模型，管理全周期AI工作流，助力千行百业智能升级。



7.1 ModelArts介绍



7.1 ModelArts介绍



7.2 操作以及演示



详见实验手册

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
 - 7.1 ModelArts介绍
 - 7.2 操作以及演示
 - 7.3 自定义训练教程——手势识别
- 八、逻辑实现 (4学时)
- 九、运行示例

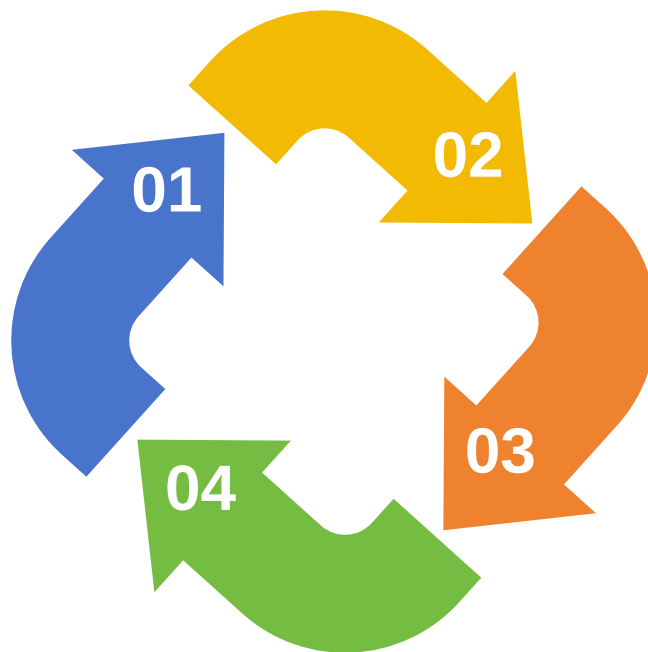
7.3 自定义训练教程——手势识别

使用预训练框架

Duo illum dolores erat ipsum euismod
ipsum consetetur magna.

文件上传OBS

Ipsum amet sit amet tation sanctus sed
sea consetetur ea.



模型转换

Et minim amet no et dolor et ut laoreet
odio sit erat.

训练作业

Ipsum diam et magna et et kasd labore
duis in elit.

7.3 自定义训练教程——手势识别

事先在OBS桶中，新建用于上传代码和模型存储的文件夹



7.3 自定义训练教程——手势识别

然后上传PC端的工程文件

桶列表 / test-dog-gesture / yolov5-7.0

华北-北京四 | 桶内对象总数: 353 | 存储总用量: 134.13 MB

上传 新建文件夹 下载 复制 更多

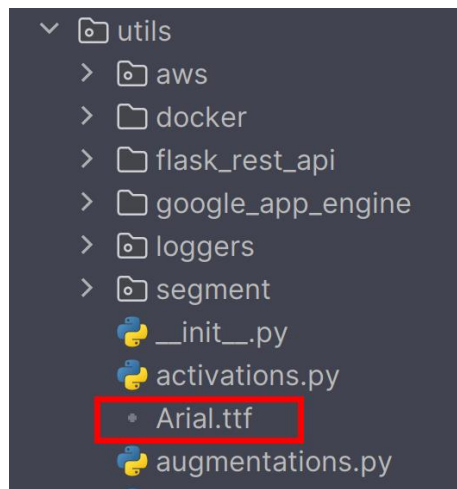
☐	对象名称	存储类别	大小
☐	.github	--	--
☐	.idea	--	--
☐	__pycache__	--	--
☐	classify	--	--
☐	data	--	--
☐	dataset	--	--
☐	models	--	--
☐	runs	--	--
☐	segment	--	--
☐	utils	--	--
☐	.dockerignore	标准存储	3.61 KB
☐	.gitattributes	标准存储	75 bytes



7.3 自定义训练教程——手势识别

其中我们需要做出如下修改：

1. 修改字体设置，防止在部署中因为网络原因报错



手动在浏览器的网址中输入：<https://ultralytics.com/assets/Arial.ttf>，下载字体文件，然后放在图中显示的目录下



7.3 自定义训练教程——手势识别

然后在general.py文件中做出如下修改，将字体设置为本地

```
def check_font(font=FONT, progress=False):
    # Download font to CONFIG_DIR if necessary
    font = Path(font)
    file = CONFIG_DIR / font.name
    local_font_path = Path('yolov5-7.0/utils') / font.name
    if local_font_path.exists():
        return local_font_path
    # if not font.exists() and not file.exists():
    #     url =
    f'https://ultralytics.com/assets/{font.name}'
    #     LOGGER.info(f'Downloading {url} to {file}...')
    #     torch.hub.download_url_to_file(url, str(file),
    progress=progress)
```



7.3 自定义训练教程——手势识别

2. 设置模型的保存路径

在train.py中新增如下代码：

```
general.py  train.py x  detect.py  data.yaml  export.py
381         opt : vars(opt),
382         # 'git': GIT_INFO, # {remote, branch, commit} if a git repo
383         'date': datetime.now().isoformat()
384
385         # Save last, best and delete
386         torch.save(ckpt, last)
387         if best_fitness == fi:
388             torch.save(ckpt, best)
389         if opt.save_period > 0 and epoch % opt.save_period == 0:
390             torch.save(ckpt, w / f'epoch{epoch}.pt')
391
392         # 新增
393         model_save_path = opt.train_url
394         print(model_save_path)
395         save_name = os.path.join(model_save_path, 'model.pt')
396         torch.save(ckpt, save_name)
397
```

新增

```
model_save_path = opt.train_url
```

```
print(model_save_path)
```

```
save_name = os.path.join(model_save_path, 'model.pt')
```

```
torch.save(ckpt, save_name)
```



7.3 自定义训练教程——手势识别

再设置一个参数

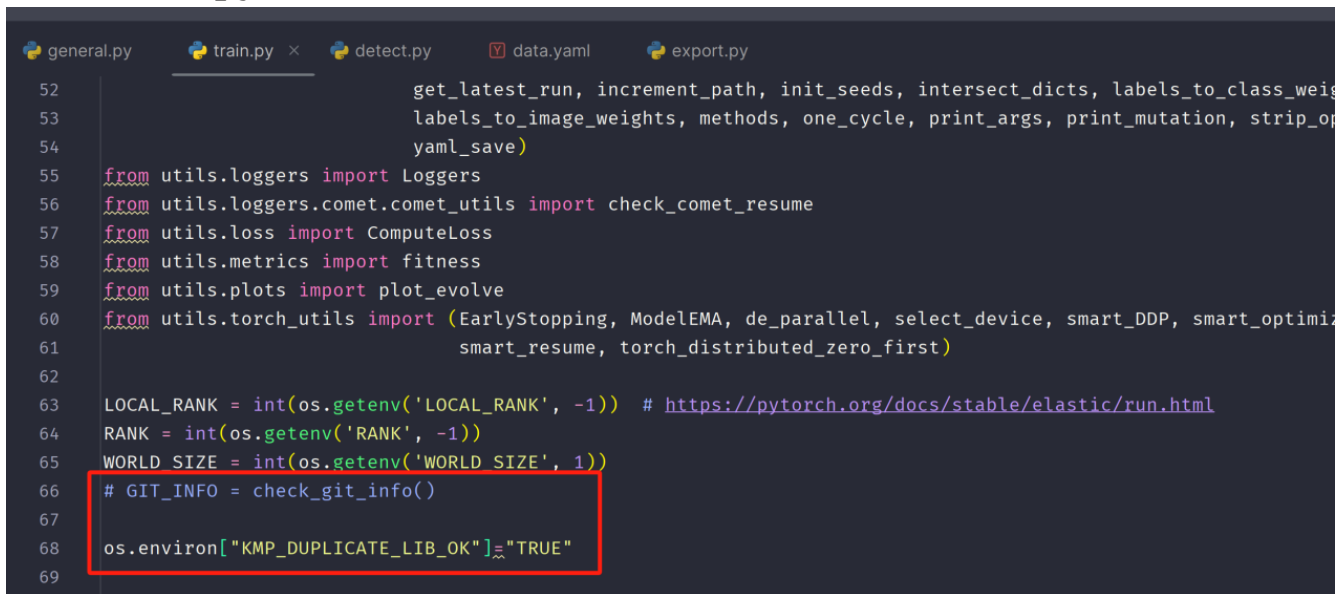
```
general.py  train.py  detect.py  data.yaml  export.py
441
442
443 def parse_opt(known=False):
444     parser = argparse.ArgumentParser()
445     parser.add_argument('--weights', type=str, default=ROOT / 'yolov5s.pt', help='initial weights path')
446     parser.add_argument('--cfg', type=str, default='', help='model.yaml path')
447     parser.add_argument('--data', type=str, default=ROOT / 'data/coco128.yaml', help='dataset.yaml path')
448     parser.add_argument('--train_url', type=str, default="weights", help='Train url for ModelArts')
449     parser.add_argument('--hyp', type=str, default=ROOT / 'data/hyps/hyp.scratch-low.yaml', help='hyperparameters path')
450     parser.add_argument('--epochs', type=int, default=100, help='total training epochs')
451     parser.add_argument('--batch-size', type=int, default=16, help='total batch size for all GPUs, -1 for autobatch')
452     parser.add_argument('--imgsz', '--img', '--img-size', type=int, default=640, help='train, val image size (pixels)')
453     parser.add_argument('--rect', action='store_true', help='rectangular training')
454     parser.add_argument('--resume', nargs='?', const=True, default=False, help='resume most recent training')
```

`parser.add_argument('--train_url', type=str, default="weights", help='Train url for ModelArts')`



7.3 自定义训练教程——手势识别

3.Git相关检测操作去除：在modelarts建立的环境中git相关操作可能因为网络原因导致脚本运行失败，git相关操作在本实验中无用处，可以去除，先找到train.py中的导包部分，再找红色圈出来的



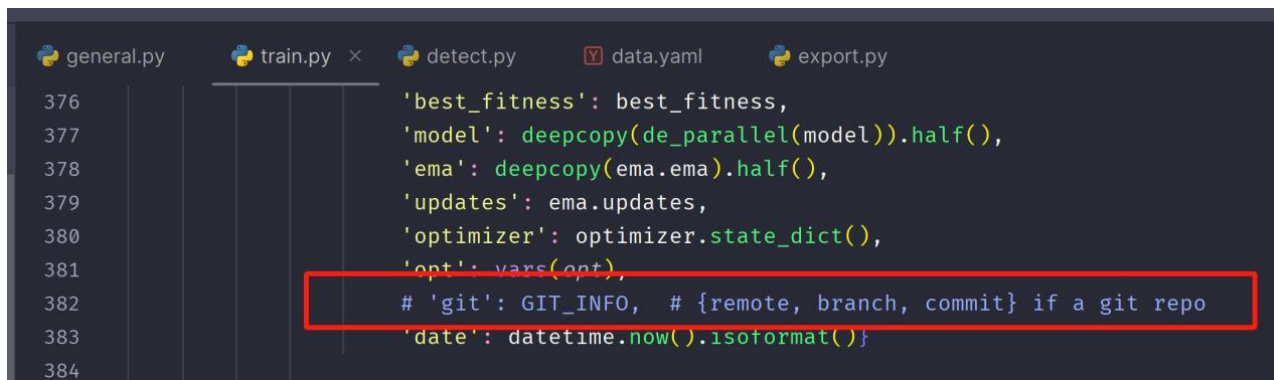
```
52         get_latest_run, increment_path, init_seeds, intersect_dicts, labels_to_class_weights,
53         labels_to_image_weights, methods, one_cycle, print_args, print_mutation, strip_optimizer)
54     yaml_save(yaml_path, yaml_data)
55     from utils.loggers import Loggers
56     from utils.loggers.comet.comet_utils import check_comet_resume
57     from utils.loss import ComputeLoss
58     from utils.metrics import fitness
59     from utils.plots import plot_evolve
60     from utils.torch_utils import (EarlyStopping, ModelEMA, de_parallel, select_device, smart_DDP, smart_optimizer,
61                                   smart_resume, torch_distributed_zero_first)
62
63     LOCAL_RANK = int(os.getenv('LOCAL_RANK', -1)) # https://pytorch.org/docs/stable/elastic/run.html
64     RANK = int(os.getenv('RANK', -1))
65     WORLD_SIZE = int(os.getenv('WORLD_SIZE', 1))
66     # GIT_INFO = check_git_info()
67
68     os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"
69
```

```
# GIT_INFO = check_git_info()
```

```
os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"
```



7.3 自定义训练教程——手势识别



```
376     'best_fitness': best_fitness,  
377     'model': deepcopy(de_parallel(model)).half(),  
378     'ema': deepcopy(ema.ema).half(),  
379     'updates': ema.updates,  
380     'optimizer': optimizer.state_dict(),  
381     'opt': vars(opt),  
382     # 'git': GIT_INFO, # {remote, branch, commit} if a git repo  
383     'date': datetime.now().isoformat()  
384
```

'git': GIT_INFO, # {remote, branch, commit} if a
git repo

注意：确保上面的操作都已完成，才可进行下面的操作。



7.3 自定义训练教程——手势识别

事先在OBS桶中，新建和上传代码、模型存储文件夹，
文件夹和桶名称如下所示：



上传ModelArts端的工程文件。



7.3 自定义训练教程——手势识别

进行训练之前，确保前面的所有步骤已经完成；在“训练管理”->“训练作业”，点击右上角的“创建训练作业”

名称ID	作业类型	状态	运行时长 (h...)	创建时间	算法	资源池	规格	节点 (个数)	作业优先级	描述	创建者	操作
job-779e-copy-b629 da038ab1-bcd7-49a4-bf67-57923...	训练作业	运行失败	00:01:13	2023/12/26 1...	--	--	[限时免费]GPU:	1	--	--	hw028137784	删除 重建 终止
job-779e bfa620c7-b536-4056-add1-471db9...	训练作业	运行失败	00:01:25	2023/12/26 1...	--	--	[限时免费]GPU:	1	--	--	hw028137784	删除 重建 终止
handwriting aef67c32-859b-419e-b5ce-d6a8c...	训练作业	已完成	00:03:23	2023/12/26 1...	--	--	[限时免费]GPU:	1	--	用于手写体识别	hw028137784	删除 重建 终止
workflow_created_job_342131b8-... c50eea26-3ce0-403b-81c6-20508...	训练作业	已完成	00:23:09	2023/12/22 1...	自动学习算法-物	--	[限时免费]GPU:	1	--	--	hw028137784	删除 重建 终止
workflow_created_job_7a00c30d-... e9f7e730-9bcc-4a11-9f00-20440a...	训练作业	已完成	00:00:49	2023/12/22 1...	自动学习算法-数	--	[限时免费]GPU:	1	--	--	hw028137784	删除 重建 终止



7.3 自定义训练教程——手势识别

新建一个训练作业，名称任意即可，同时按照下面配置，配置好路径和参数，在modelarts平台配置如下，训练作业名称：

★ 名称	job-a931-copy-7122	✓
描述	狗	1/256

自定义算法的启动文件和数据文件路径：

代码目录
不支持选择跨区域（Region）的OBS桶。只能选择对象存储服务（OBS）桶下的文件夹。

< test-dog-gesture

新建桶 >>

桶名称
☐ [模糊名称]
☐ [模糊名称]
☒ test-dog-gesture

名称	最后修改时间	类型	大小
返回上一级			
☒ yolov5-7.0	--	文件夹	--
☐ output	--	文件夹	--



7.3 自定义训练教程——手势识别

启动文件

不支持选择跨区域（Region）的OBS桶。请选择英文路径和英文命名文件。

test-dog-gesture / yolov5-7.0

请输入桶名称查询

新建桶 >>

桶名称

test-dog-gesture

新建文件夹

请输入名称查询

名称	最后修改时间	类型	大小
返回上一级			
tutorial.ipynb	2024/04/10 15:46:51 GM...	文件	53842 B
yolov5n.pt	2024/04/10 15:46:51 GM...	文件	4062133 B
train.py	2024/04/10 15:55:00 GM...	文件	33904 B
val.py	2024/04/10 15:46:51 GM...	文件	20282 B
setup.cfg	2024/04/10 15:46:51 GM...	文件	1686 B
requirements.txt	2024/04/10 15:46:51 GM...	文件	1541 B
yolov5s.pt	2024/04/10 15:46:51 GM...	文件	1480843...
utils	--	文件夹	--
segment	--	文件夹	--
runs	--	文件夹	--

上一页

下一页



7.3 自定义训练教程——手势识别

★ 创建方式

自定义算法

我的算法

我的订阅

?

★ 启动方式

预置框架

自定义

PyTorch

pytorch_1.8.0-cuda_10.2-py_3....

★ 代码目录

/test-dog-gesture/yolov5-7.0/

选择

?

★ 启动文件

/test-dog-gesture/yolov5-7.0/train.py

选择

?

本地代码目录

/home/ma-user/modelarts/user-job-dir

工作目录

/home/ma-user/modelarts/user-job-dir

×

选择



7.3 自定义训练教程——手势识别

输入参数:

输入 ?

weights

/test-dog-gesture/yolov5-7.0/yolov5s.pt

数据集

数据存储位置

获取方式

☒ 超参 ☐ 环境变量

--weights=/home/ma-user/modelarts/inputs/weights_0

data

/test-dog-gesture/yolov5-7.0/dataset/data.yaml

数据集

数据存储位置

获取方式

☒ 超参 ☐ 环境变量

--data=/home/ma-user/modelarts/inputs/data_1

cfg

/test-dog-gesture/yolov5-7.0/models/yolov5s.yaml

数据集

数据存储位置

获取方式

☒ 超参 ☐ 环境变量

--cfg=/home/ma-user/modelarts/inputs/cfg_2

输出参数:

输出 ?

train_url

/test-dog-gesture/output/

数据存储位置

获取方式

☒ 超参 ☐ 环境变量

--train_url=/home/ma-user/modelarts/outputs/train_url_0

预下载至本地目录 ?

☒ 不下载 ☐ 下载



7.3 自定义训练教程——手势识别

训练用到的超参数：

超参

img

=

640

🗑

batch

=

16

🗑

epoch

=

5

🗑

⊕ 增加超参

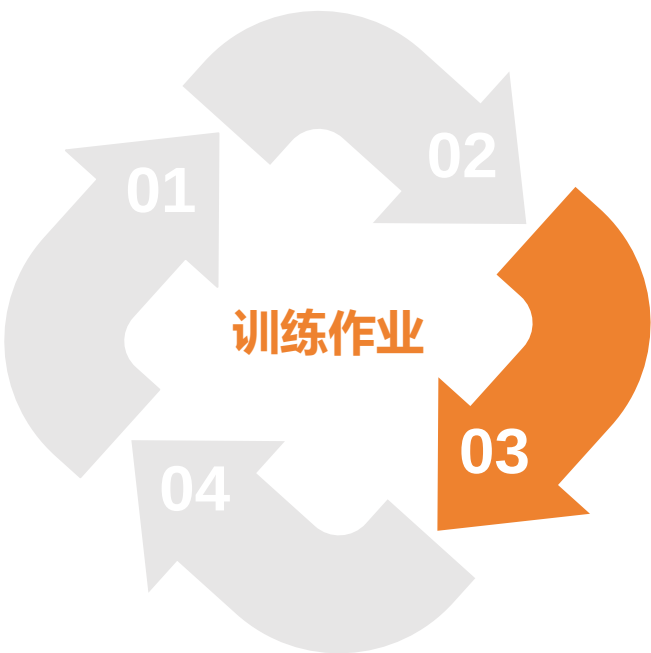
点击提交

信息确认

作业名	训练信息	价格
job-a931-copy-7122	算法	--
	预置镜像	PyTorch pytorch_1.8.0-cuda_10.2-py_3.7-ubuntu_18.04-x86_64
	代码目录	/test-dog-gesture/yolov5-7.0/
	启动文件	/test-dog-gesture/yolov5-7.0/train.py
	本地代码目录	/home/ma-user/modelarts/user-job-dir
	工作目录	/home/ma-user/modelarts/user-job-dir
	输入	weights=/test-dog-gesture/yolov5-7.0/yolov5s.pt data=/test-dog-gesture/yolov5-7.0/dataset/data.yaml cfg=/test-dog-gesture/yolov5-7.0/models/yolov5s.yaml
	输出	train_url=/test-dog-gesture/output/
	超参	img=640 batch=16 epoch=5
	规格	[限时免费] GPU: 1*NVIDIA-V100(32GB) CPU: 8 核 64GB 780GB
计算节点个数		1

确定

取消



7.3 自定义训练教程——手势识别

等待完成

< | job-a931-copy-7122

作业ID2d1d21f1-404c-415a-86ac-ba3a4b5e729c

作业状态

✔ 已完成

创建时间2024/04/10 17:12:09 GMT+08:00

运行时长00:01:18

描述狗

预置镜像PyTorch | pytorch_1.8.0-cuda_10.2-py_3.7-ubuntu_18.04-x86_64

算法名称--

代码目录/test-dog-gesture/yolov5-7.0/

编辑代码

启动文件/test-dog-gesture/yolov5-7.0/train.py

运行用户ID1102

本地代码目录/home/ma-user/modelarts/user-job-dir

工作目录/home/ma-user/modelarts/user-job-dir

事件日志资源占用情况评估结果标签

2024/04/10 17:12:09 — 2024/04/10 17:12:09

事件类型	事件信息
✔ 正常	训练作业运行成功。
✔ 正常	[Job: modelarts-job-2d1d21f1-404c-415a-86ac-ba3a4b5e729c]
✔ 正常	[worker-0][耗时: 1.032秒] 训练输出 (参数名称: train_u
✔ 正常	[worker-0] 训练输出 (参数名称: train_u
✔ 正常	[worker-0] 训练结束。退出码0。
✔ 正常	[worker-0] GPU进程启动
✔ 正常	[worker-0][耗时: 8.730秒] Python依赖包
✔ 正常	[worker-0] 正在安装Python依赖包。导入
✔ 正常	[worker-0] 训练开始。
✔ 正常	训练作业开始运行。

10 总条数: 38 < 1 2 3 4 > 跳转 页

01 02 03 04

训练作业

Copyright © Huawei Technologies Co., Ltd. All rights reserved.

Page 124

HUAWEI

7.3 自定义训练教程——手势识别

运行成功，然后在模型的保存路径中找到模型文件

← → ↑ 桶列表 / test-dog-gesture / **output**

📍 华北-北京四 | 桶内对象总数: 353 | 存储总用量: 134.13 MB

📶 上传 📁 新建文件夹 ⬇️ 下载 📄 复制 更多 ▾

☐	对象名称 ▾	存储类别 ▾	大小 ▾
☐	 model.pt	标准存储	54.24 MB



7.3 自定义训练教程——手势识别

在obs中将模型文件下载，然后上传项目文件中的runs目录下，然后运行：

← → ↑ 桶列表 / test-dog-gesture / output

📍 华北-北京四 | 桶内对象总数: 353 | 存储总用量: 134.13 MB

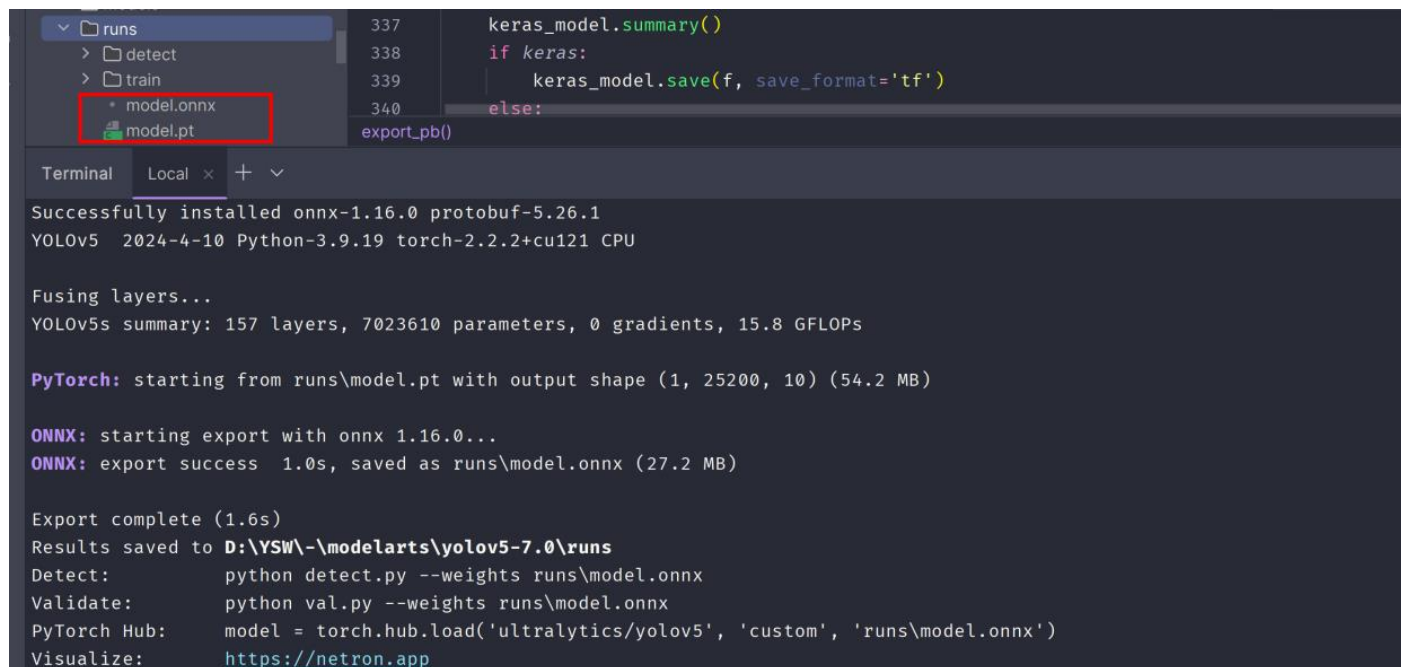
📶 上传 📁 新建文件夹 ⬇️ 下载 📋 复制 更多 ▾

☐	对象名称 ▾	存储类别 ▾	大小 ▾
☐	 model.pt	标准存储	54.24 MB

```
python export.py --weights runs/model.pt --include onnx
```



7.3 自定义训练教程——手势识别



The screenshot shows a code editor with a file explorer on the left. The file explorer shows a directory structure with 'runs' containing 'detect', 'train', and 'model.onnx' (highlighted with a red box). The code editor shows the following Python code:

```
337 keras_model.summary()  
338 if keras:  
339     keras_model.save(f, save_format='tf')  
340 else:  
    export_pb()
```

The terminal window shows the following output:

```
Successfully installed onnx-1.16.0 protobuf-5.26.1  
YOLOv5 2024-4-10 Python-3.9.19 torch-2.2.2+cu121 CPU  
  
Fusing layers...  
YOLOv5s summary: 157 layers, 7023610 parameters, 0 gradients, 15.8 GFLOPs  
  
PyTorch: starting from runs\model.pt with output shape (1, 25200, 10) (54.2 MB)  
  
ONNX: starting export with onnx 1.16.0...  
ONNX: export success 1.0s, saved as runs\model.onnx (27.2 MB)  
  
Export complete (1.6s)  
Results saved to D:\YSW\-\modelarts\yolov5-7.0\runs  
Detect: python detect.py --weights runs\model.onnx  
Validate: python val.py --weights runs\model.onnx  
PyTorch Hub: model = torch.hub.load('ultralytics/yolov5', 'custom', 'runs\model.onnx')  
Visualize: https://netron.app
```



7.3 自定义训练教程——手势识别

在runs目录中成功生成了model.onnx

将ONNX模型文件上传到开发板

```
(base) root@davinci-mini:/opt# ll
total 27888
drwxr-xr-x  2 root root    4096 Apr 12 17:10 ./
drwxr-xr-x 24 root root    4096 Apr  8 2022 ../
-rw-r--r--  1 root root 28545135 Apr 12 17:10 model.onnx
```

运行下面命令

```
atc --model=model.onnx --framework=5 --output=model --
soc_version=Ascend310B4
```



7.3 自定义训练教程——手势识别

显示转换成功即可

```
(base) root@davinci-mini:/opt# atc --model=model.onnx --framework=5 --output=model --soc_version=Ascend310B4
ATC start working now, please wait for a moment.
...
ATC run success, welcome to the next use.
```

```
(base) root@davinci-mini:/opt# ll
total 42844
drwxr-xr-x  2 root root    4096 Apr 12 17:26 ./
drwxr-xr-x 24 root root    4096 Apr  8  2022 ../
-rw-r-----  1 root root    3022 Apr 12 17:26 fusion_result.json
-rw-----  1 root root 15307128 Apr 12 17:26 model.om
-rw-r--r--  1 root root 28545135 Apr 12 17:10 model.onnx
```

可以看到已经有.om模型文件输出。

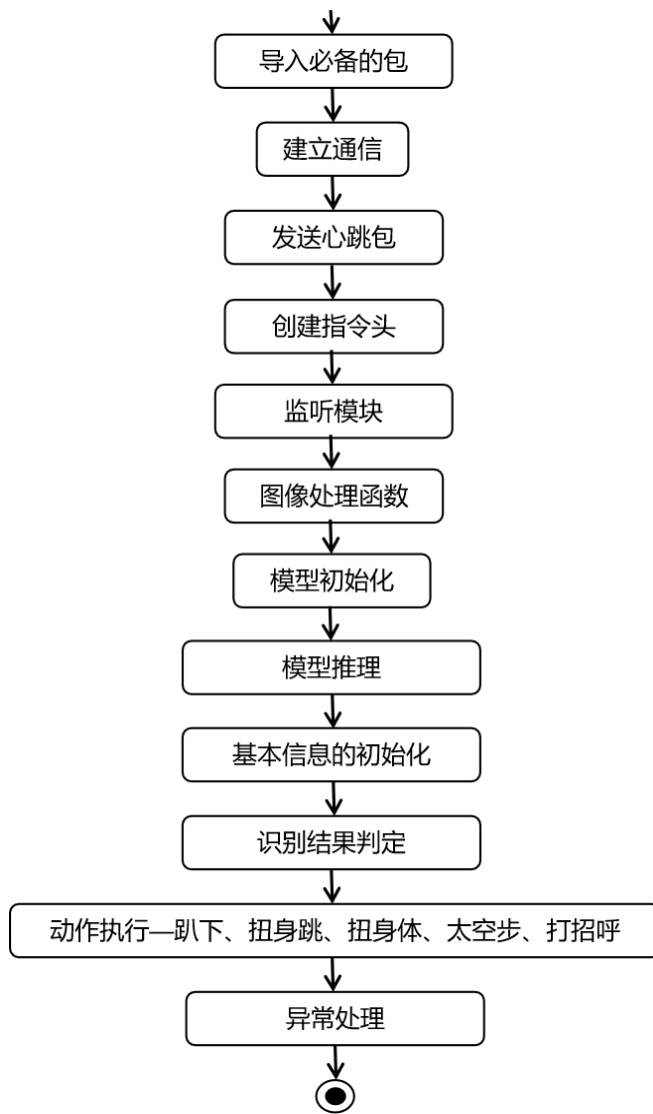


目录

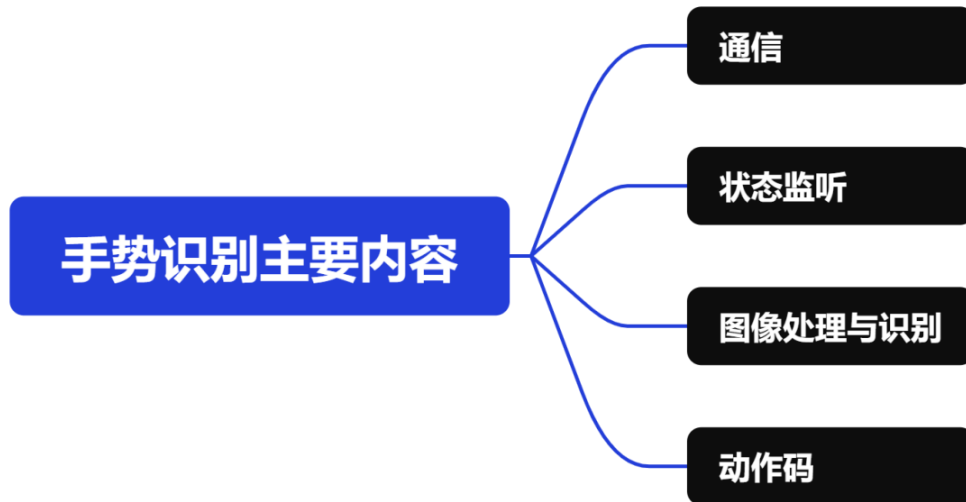
- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 模型初始化
 - 8.8 模型推理
 - 8.9 基本信息的初始化
 - 8.10 识别结果判定
 - 8.11 动作执行
 - 8.12 异常处理
- 九、运行示例

整体流程

手势识别的整体代码的流程图



8.1 导入必备的包



导入手势识别所用到的库，主要包括图像处理、深度学习、推理接口、时间处理、日志记录、类型注解、系统路径设置、相机图像获取、模型前后处理函数，以及与机器狗相关的特定功能模块。具体如下：（其中监听的函数包，放在了文件夹的`robotstatuswatcher/listener.py`中，后文给出解释

8.1 导入必备的包

导入手势识别所用到的库，主要包括图像处理、深度学习、推理接口、时间处理、日志记录、类型注解、系统路径设置、相机图像获取、模型前后处理函数，以及与机器狗相关的特定功能模块。具体如下：

```
1. import cv2 # 图片处理三方库，用于对图片进行前后处理
2. import numpy as np # 用于对多维数组进行计算
3. import torch # 深度学习运算框架，此处主要用来处理数据
4. from mindx.sdk import Tensor # mxVision 中的 Tensor 数据结构
5. from mindx.sdk import base # mxVision 推理接口
6. import time
7. import logging
8. from typing import *
9. import sys
10. sys.path.append('camera/')
11. from HKcamera import getImage
12. from det_utils import get_labels_from_txt, letterbox, scale_coords, nms, draw_bbox # 模型前后处理相关函数
13.
14. # 导入定义的机器狗相关的包
15. from threading_utils.ThreadTemplates import * # 线程的模板与基本轴指令的发送
16. from sendcommand import SendToCommand, heartbeat # 发送简单指令与心跳包
17. from socketnetwork import network_utils # 通信函数
18. from command.udp_command import * # 存放各种结构体与状态数据、指令
19. from speeds.sportspeed import * # 运动精准控制的基础版模型
20. from robotstatuswatcher.listener import * # 监听机器狗状态
```

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 模型初始化
 - 8.8 模型推理
 - 8.9 基本信息的初始化
 - 8.10 识别结果判定
 - 8.11 动作执行
 - 8.12 异常处理
- 九、运行示例

8.2建立通信

简单控制第一步：建立通信链接，位于socketnetwork/network_utils.py，用于创建一个UDP套接字，并设置机器狗的IP地址和端口号作为目标地址。该通信链接是所有后续控制和数据传输的基础。不仅仅确保了指令传输的可靠性和及时性，还实现对机器狗的远程控制和监控以及为数据传输提供通道，代码如下：

```
1. import socket
2. from typing import *
3.
4. def setup_socket_and_address(dest_ip = '192.168.1.120', port=43893) -> Tuple[socket.socket, Tuple[str, int]]:
5.     # 创建UDP套接字
6.     sfd = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
7.
8.     # 设置目标地址
9.     target_address = (dest_ip, port)
10.    # print(target_address)
11.
12.    return sfd, target_address
```

表示使用IPv4地址

表示使用UDP协议

将传入的IP地址和端口号组合成一个元组，这个元组将用作套接字的目标地址。

目录

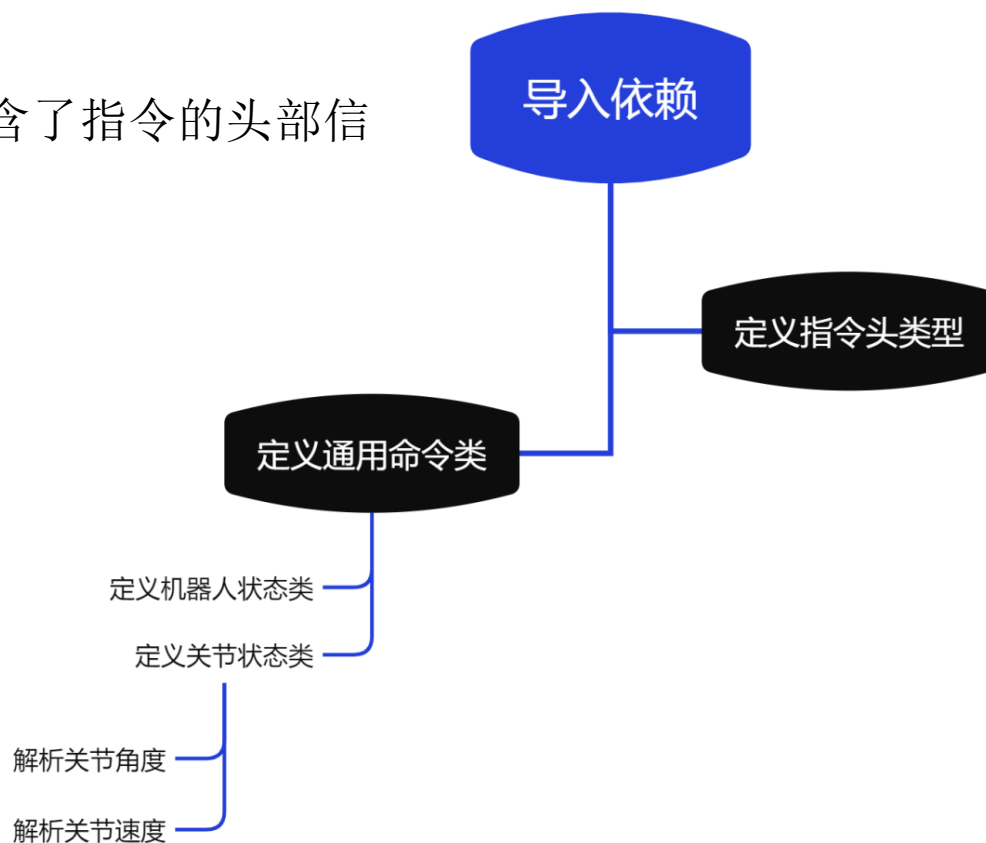
- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 模型初始化
 - 8.8 模型推理
 - 8.9 基本信息的初始化
 - 8.10 识别结果判定
 - 8.11 动作执行
 - 8.12 异常处理
- 九、运行示例

8.3 创建指令头

创建指令头主要为了构建通信协议，提高代码的可读性和可维护性以及支持复杂的指令操作。指令头包含了关键信息，如指令码、参数大小和类型，是通信协议的核心组成部分。通过定义指令头，可以构建和解析复杂的通信协议。

```
action_CommandHead = CommandHead()
```

创建了一个CommandHead对象，这个对象包含了指令的头部信息，如动作代码、序列号等。



8.3 创建指令头

类的继承结构

这些类都继承自CommandHead，它们共享一些基本属性和方法。CommandHead定义了基本信息，而他的子类定义了用于解析命令头部的方法和属性。

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. import ctypes
5. import struct
6. import inspect
7.
8. class CommandHead:
9.     def __init__(self, code=0, parameters_size=0, type_=0):
10.         self.code = code                # 指令码
11.         self.parameters_size = parameters_size    # 指令值
12.         self.type_ = type_              # 指令类型
13.
14. class Command:
15.     kDataSize = 256
```

8.3 创建指令头

```
1. class JointStateReceived(CommandHead):
2.     def __init__(self, data):
3.         """
4.
5.         1. 关节信息的头部信息为4字节命令字,4字节长度,4字节类型,后续为正文数据---(后续的所有都是类似)
6.
7.         2. 1字节(b),2字节无符号短整型(H),4字节的无符号整型(I),接着是12个8字节的双精度浮点数(12d)
8.
9.         3. code,parameters_size,type_的是继承于CommandHead头的,这样写是为了方便呈现出,数据都是基于CommandHead
   发送和接收的
10.        这样可以做到避免所有的复杂数据全部发在相同名字的Command中,而是重新定义一个基于CommandHead的结构体
11.
12.        4. "self.code, " 逗号是为了去除掉由 struct.unpack 操作产生的元组问题,可以去去掉逗号直接复制
13.
14.        """
15.        self.data = data          # 关节的状态数据, 全部接受, 再由code来决定分配给谁
16.        CommandHead.code, = struct.unpack('<I', self.data[:4])
```

→ 用于接收关节状态的原始数据, 并且解析出头部信息

8.3 创建指令头

```
1. class JointAngle(CommandHead):  
2.     def __init__(self, joint_state_received):  
3.         # 特定于关节角度的数据解析  
4.         *self.joint_angles, = struct.unpack('<12d', joint_state_received.data[12:])  
5.  
6. class JointSpeed(CommandHead):  
7.     def __init__(self, joint_state_received):  
8.         # 特定于关节速度的数据解析  
9.         *self.joint_speeds, = struct.unpack('<12d', joint_state_received.data[12:])
```

专门用于从关节状态数据中解析出关节的角度

用于解析关节的速度

8.3 创建指令头

RobotState 类

作用：解析机器人的整体状态，包括运动状态、步态信息、动作状态、雷达距离等。

解析过程：

首先解析命令头信息，包括命令码、参数大小和类型。

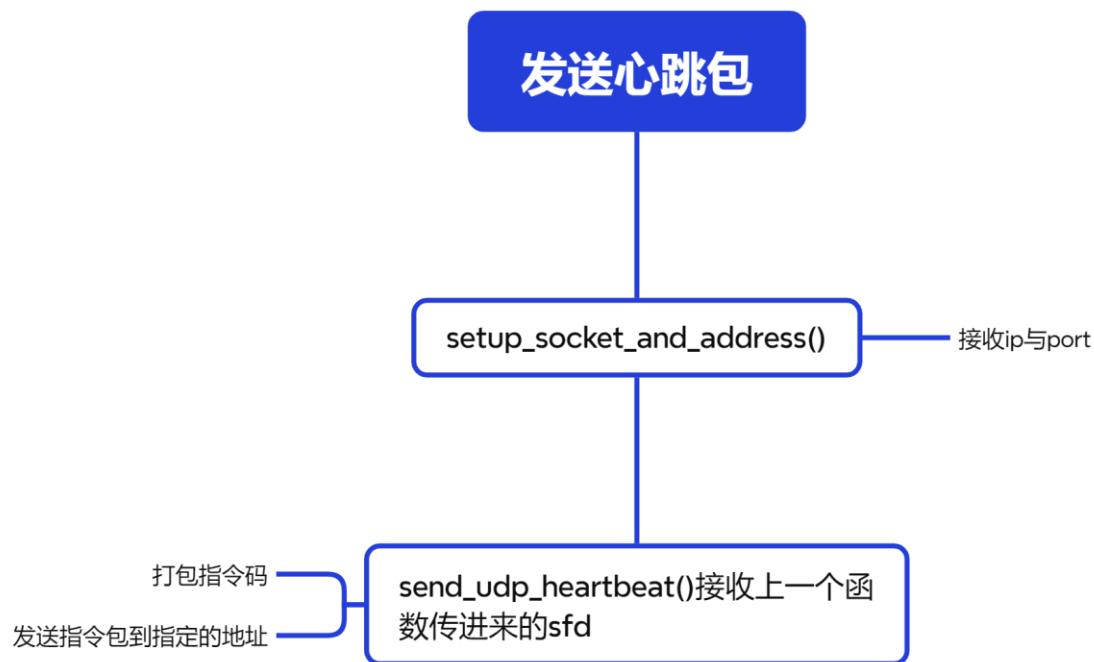
然后解析机器人的基本运动状态、步态信息、动作状态等。

```
1. class RobotState(CommandHead):
2.     def __init__(self, data):
3.
4.         CommandHead.code, = struct.unpack('<I', data[:4])
5.         CommandHead.parameters_size, = struct.unpack('<I', data[4:8])
6.         CommandHead.type_, = struct.unpack('<I', data[8:12])
7.         self.robot_basic_state, = struct.unpack('<I', data[12:16])           # 机器人基本运动状态
8.         self.robot_gait_state, = struct.unpack('<I', data[16:20])           # 机器人步态信息
9.         self.robot_motion_state, = struct.unpack('<i', data[176:180])       # 机器人动作状态
10.         self.distance_ahead, = struct.unpack('<d', data[-16:-8])           # 雷达前方的距离
11.         self.rear_distance, = struct.unpack('<d', data[-8:])
```

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 模型初始化
 - 8.8 模型推理
 - 8.9 基本信息的初始化
 - 8.10 识别结果判定
 - 8.11 动作执行
 - 8.12 异常处理
- 九、运行示例

8.4 发送心跳包



心跳包是一种周期性发送的信号，用于确保网络连接的活跃状态，防止因长时间无通信而被网络设备断开连接。这对于实时控制和数据传输至关重要。，确保系统在运行过程中不会因为通信问题而出现故障，提高系统的整体可靠性。发送心跳包代码位于sendcommand/heartbeat.py，代码如下：

8.4 发送心跳包

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3. import socket
4. import struct
5. import time
6.
7. def send_udp_heartbeat(sfd, target_address, code=0x21040001, parameters_size=0, type=0
, heartbeat_interval=0.25) -> None:
8.     # 打包心跳指令数据
9.     heartbeat_command = struct.pack('<III', code, parameters_size, type)
10.
11.     try:
12.         while True:
13.             # 记录发送前的时间
14.             start_time = time.time()
15.
16.             # 发送心跳包
17.             sfd.sendto(heartbeat_command, target_address)
18.
19.             # 发送后的延时
20.             time.sleep(max(0, heartbeat_interval - (time.time() - start_time)))
21.
22.     except KeyboardInterrupt:
23.         print("Heartbeat sending stopped by user.")
24.     finally:
25.         # 关闭socket
26.         sfd.close()
```

8.4 发送心跳包

启动心跳线程, 代码如下:

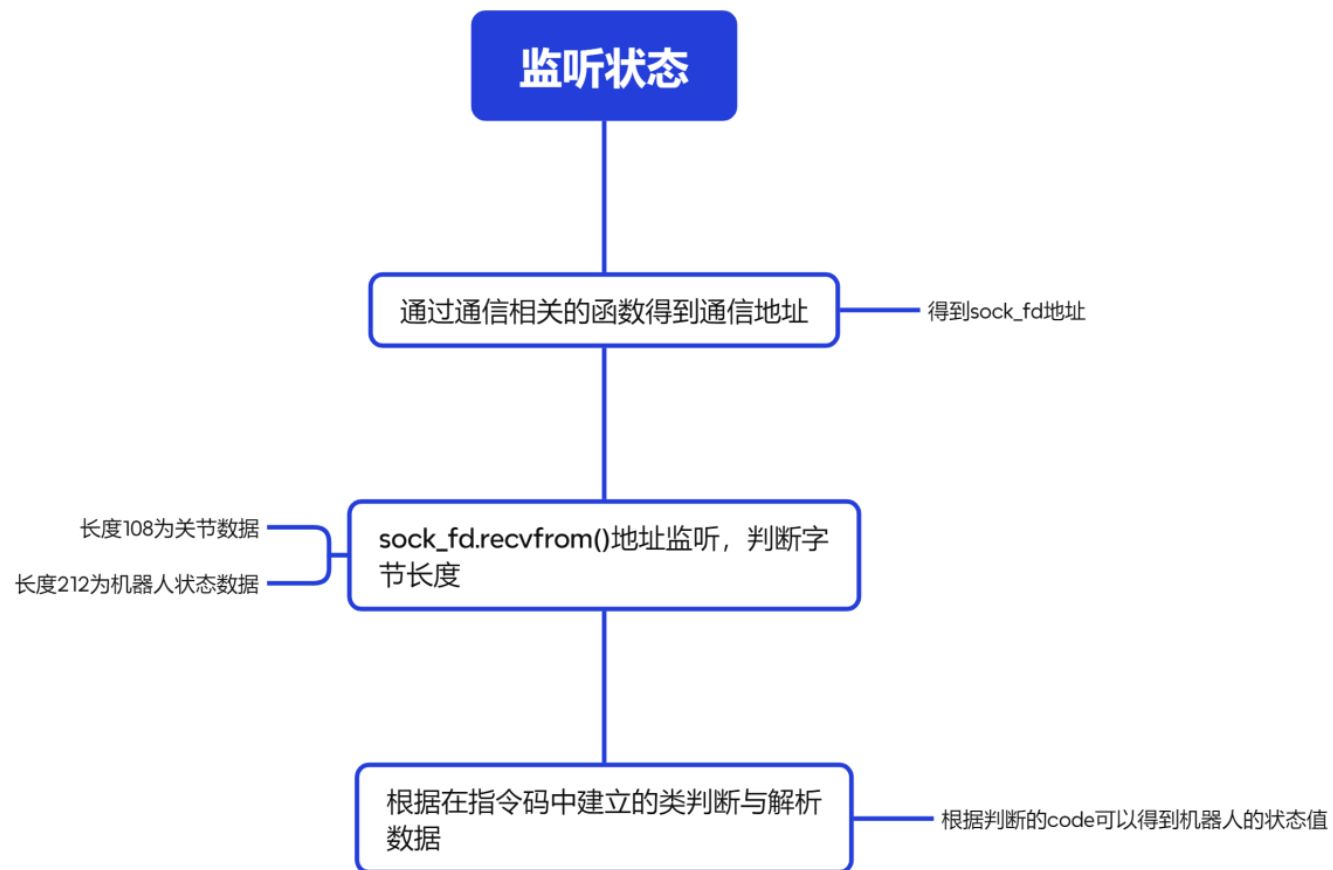
1. *# 建立通讯*
2. `sfd, target_address = network_utils.setup_socket_and_address()`
- 3.
4. *# 定义发送心跳包的线程执行体*
5. *# 注意*args必须以tuple形式传入, 包括sfd和target_address*
6. `heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None, sfd, target_address)`
7. `heartbeat_thread.start()`

`heartbeat_thread.start()` 启动了心跳线程, 它将按照设定的时间间隔 (这里是0.25秒) 发送心跳包 (MyRepeatThread在后文中会解释)。

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 模型初始化
 - 8.8 模型推理
 - 8.9 基本信息的初始化
 - 8.10 识别结果判定
 - 8.11 动作执行
 - 8.12 异常处理
- 九、运行示例

8.5 监听模块



8.5 监听模块

机器狗的状态值有独有的编码列表，见下表。

	值1	值2	值3	状态描述
	1	0	0	趴下状态
	1	0	11	正在执行向前跳
	4	0	0	准备起立状态
	5	0	0	正在起立状态
	6	0	0	力控状态（静止站立）且步态为平地低速步态
	6	0	1	正在以平地低速步态踏步或正在根据轴指令扭动身体
	6	0	2	正在执行扭身体
	6	0	4	正在执行扭身跳
	6	2	0	力控状态（静止站立）且步态为通用越障步态
	6	2	1	正在以通用越障步态踏步
	6	4	0	力控状态（静止站立）且步态为平地中速步态
	6	4	1	正在以平地中速步态踏步
	6	5	0	力控状态（静止站立）且步态为平地高速步态
	6	5	1	正在以平地高速步态踏步
	6	6	0	力控状态（静止站立）且步态为抓地越障步态
	6	6	1	正在以抓地越障步态踏步
	6	12	1	正在执行太空步
	6	13	0	力控状态（静止站立）且步态为高踏步越障步态
	6	13	1	正在以高踏步越障步态踏步
	7	0	0	正在趴下状态
	8	0	0	失控保护状态
	9	0	0	姿态调整状态
	11	0	0	正在执行翻身
	17	0	0	回零状态
	18	0	0	正在执行后空翻
	20	0	0	正在执行打招呼

8.5 监听模块

```
# 将上级目录添加到模块搜索路径中
sys.path.append('../')
# 从相对路径的command包中导入udp_command模块
from command.udp_command import *
# 从socketnetwork模块导入network_utils
from socketnetwork import network_utils
# 使用network_utils中的set_up_recvfrom_socket_and_address函数设置并返回套接字和接收地址
sock_fd = network_utils.set_up_recvfrom_socket_and_address()
def status_listener():
    # 无限循环，持续监听
    while True:
        # 使用sock_fd接收数据，假设接收的数据不会超过1024字节
        recv_data, _ = sock_fd.recvfrom(1024)
        # 计算接收到的数据长度
        recv_num = len(recv_data)
        # 如果接收到的数据长度为108，认为数据完整
        if recv_num == 108:
            dr = JointStateReceived(recv_data)
            # 如果解析后的code属性为2306，表示是关机角度的指令
            if dr.code == 2306:
                # 创建JointAngle实例，访问JointStateReceived实例的data属性
                joint_angle = JointAngle(dr)
                # 打印接收到的关节角度数据
                # print("Received Joint Angles:", joint_angle.joint_angles)
```

→ 用于监听接收到的数据

→ 解析接收到的数据

8.5 监听模块

```
# 如果解析后的code属性为2307，表示是关节角速度的指令
elif dr.code == 2307:
    # 创建JointSpeed实例
    joint_speed = JointSpeed(dr)
    # 打印接收到的关节速度数据
    # print("Received Joint Speed:", joint_speed.joint_speeds)

# 如果接收到的数据长度为212，认为数据是机器人状态数据
elif recv_num == 212:
    # 使用RobotState类解析接收到的数据
    dr, status_list_temp = RobotState(recv_data), []
    # 如果解析后的code属性为2305，表示是机器状态数据
    if dr.code == 2305:
        # 如果机器人基本运动状态不为0，添加状态到列表
        if dr.robot_basic_state != 0:
            status_list_temp.append(dr.robot_basic_state)
            status_list_temp.append(dr.robot_gait_state)
            status_list_temp.append(dr.robot_motion_state)
        # 返回状态列表
        return status_list_temp
```

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 模型初始化
 - 8.8 模型推理
 - 8.9 基本信息的初始化
 - 8.10 识别结果判定
 - 8.11 动作执行
 - 8.12 异常处理
- 九、运行示例

8.6 图像处理函数

letterbox: 这个函数用于将图像调整到指定的新尺寸，同时保持长宽比，通过添加边框来填充图像。它还计算了缩放比例和填充尺寸。

```
1. def letterbox(img, new_shape=(640, 640), color=(114, 114, 114), auto=False, scaleFill=False, scaleup=True):
2.     # 获取当前图像的尺寸 [height, width]
3.     shape = img.shape[:2]
4.     # 如果 new_shape 是一个整数，则将其转换为一个元组 (new_shape, new_shape)
5.     if isinstance(new_shape, int):
6.         new_shape = (new_shape, new_shape)
7.     # 计算新旧尺寸的比例，取宽度和高度比例的最小值作为缩放比例
8.     r = min(new_shape[0] / shape[0], new_shape[1] / shape[1])
9.     # 如果 scaleup 为 False，则限制 r 的值不超过 1，即不放大图像
10.    if not scaleup:
11.        r = min(r, 1.0)
12.    # 计算缩放后的图像尺寸
13.    new_unpad = int(round(shape[1] * r)), int(round(shape[0] * r))
14.    # 计算宽度和高度的填充量
15.    dw, dh = new_shape[1] - new_unpad[0], new_shape[0] - new_unpad[1] # wh padding
16.    # 如果 auto 为 True，则使用最小矩形，填充量应为 64 的倍数
17.    if auto:
18.        dw, dh = np.mod(dw, 64), np.mod(dh, 64) # wh padding
19.    # 如果 scaleFill 为 True，则不进行缩放，直接使用新尺寸填充
20.    elif scaleFill:
21.        dw, dh = 0.0, 0.0
22.        new_unpad = (new_shape[1], new_shape[0]) # 使用新尺寸
23.        ratio = new_shape[1] / shape[1], new_shape[0] / shape[0] # 重新计算宽高比例
24.
25.    .....
48.    return img, ratio, (dw, dh)
```

8.6 图像处理函数

`non_max_suppression`: 这个函数实现了非极大值抑制（NMS），用于从预测结果中去除重叠的检测框。它接受多个参数，包括置信度阈值、IoU阈值等，并返回经过NMS处理的检测结果。

```
1. def non_max_suppression(  
2.     prediction, # 预测结果，是模型输出的张量  
3.     conf_thres=0.25, # 置信度阈值，默认为0.25  
4.     iou_thres=0.45, # 交并比（IoU）阈值，默认为0.45  
5.     classes=None, # 指定要检测的类别列表，如果为None，则检测所有类别  
6.     agnostic=False, # 是否进行类别不可知的NMS  
7.     multi_label=False, # 是否允许单个实例有多个标签  
8.     labels=(), # 可选的标签列表，用于某些模型的自动标签注入  
9.     max_det=300, # 每张图像上最多检测的目标数量  
10.    nm=0, # 掩码数量，当前未使用  
11.):
```

8.6 图像处理函数

`xywh2xyxy`: 将坐标格式从(x, y, w, h)转换为(x1, y1, x2, y2), 即从中心点和宽高转换为左上角和右下角的坐标。

```
1. # 定义 xywh2xyxy 函数, 用于将 nx4 边界框从 [x, y, w, h] 格式转换为 [x1, y1, x2, y2] 格式
2. # 其中 xy1 表示左上角坐标, xy2 表示右下角坐标
3. def xywh2xyxy(x):
4.     # 如果输入是 torch.Tensor 类型, 则克隆数据; 如果是 numpy 数组, 则复制数据
5.     y = x.clone() if isinstance(x, torch.Tensor) else np.copy(x)
6.     # 计算左上角 x 坐标
7.     y[:, 0] = x[:, 0] - x[:, 2] / 2
8.     # 计算左上角 y 坐标
9.     y[:, 1] = x[:, 1] - x[:, 3] / 2
10.    # 计算右下角 x 坐标
11.    y[:, 2] = x[:, 0] + x[:, 2] / 2
12.    # 计算右下角 y 坐标
13.    y[:, 3] = x[:, 1] + x[:, 3] / 2
    return y
```

8.6 图像处理函数

`get_labels_from_txt`: 从文本文件中读取类别标签，并将它们存储在字典中。

```
1. # 定义 get_labels_from_txt 函数，用于从文本文件中读取标签并创建字典
2. def get_labels_from_txt(path):
3.     labels_dict = dict() # 初始化一个空字典来存储标签
4.     with open(path) as f: # 打开文件
5.         for cat_id, label in enumerate(f.readlines()): # 遍历文件中的每一行
6.             labels_dict[cat_id] = label.strip() # 将读取的标签去除两端空白后存入字典
7.     return labels_dict # 返回标签字典
```

8.6 图像处理函数

`scale_coords`: 将坐标从一个图像尺寸调整到另一个图像尺寸，考虑了缩放和平移。

```
1. # 定义 scale_coords 函数，用于调整坐标点以适应不同尺寸的图像
2. def scale_coords(img1_shape, coords, img0_shape, ratio_pad=None):
3.     # 根据 img1_shape 和 img0_shape 调整坐标
4.     if ratio_pad is None: # 如果没有提供 ratio_pad，则从 img0_shape 计算
5.         gain = min(img1_shape[0] / img0_shape[0], img1_shape[1] / img0_shape[1]) # 计算缩放比例
6.         pad = (img1_shape[1] - img0_shape[1] * gain) / 2, (img1_shape[0] - img0_shape[0] * gain) / 2 # 计算填充量
7.     else: # 如果提供了 ratio_pad，则使用提供的值
8.         gain = ratio_pad[0][0]
9.         pad = ratio_pad[1]
10.
11.     # 应用 x 方向的填充
12.     coords[:, [0, 2]] -= pad[0]
13.     # 应用 y 方向的填充
14.     coords[:, [1, 3]] -= pad[1]
15.     # 应用缩放比例
16.     coords[:, :4] /= gain
17.     # 裁剪坐标点，确保它们在有效范围内
18.     clip_coords(coords, img0_shape)
    return coords # 返回调整后的坐标点
```


8.6 图像处理函数

`clip_coords`: 将坐标限制在给定图像尺寸的范围內，确保检测框不会超出图像边界。

```
1. def clip_coords(bboxes, shape):
2.     # 如果 bboxes 是 torch.Tensor 类型，则逐个快速裁剪
3.     if isinstance(bboxes, torch.Tensor):
4.         bboxes[:, 0].clamp_(0, shape[1]) # 裁剪 x1 坐标，确保在 0 到图像宽度之间
5.         bboxes[:, 1].clamp_(0, shape[0]) # 裁剪 y1 坐标，确保在 0 到图像高度之间
6.         bboxes[:, 2].clamp_(0, shape[1]) # 裁剪 x2 坐标，确保在 0 到图像宽度之间
7.         bboxes[:, 3].clamp_(0, shape[0]) # 裁剪 y2 坐标，确保在 0 到图像高度之间
8.     # 如果 bboxes 是 numpy 数组，则整体快速裁剪
9.     else:
10.         bboxes[:, [0, 2]] = bboxes[:, [0, 2]].clip(0, shape[1]) # 裁剪 x1 和 x2
            bboxes[:, [1, 3]] = bboxes[:, [1, 3]].clip(0, shape[0]) # 裁剪 y1 和 y2
```

8.6 图像处理函数

nms: 一个包装函数，用于调用non_max_suppression函数并返回NMS处理后的检测框。

```
1. # 定义 nms 函数，作为非极大值抑制（NMS）的封装，调用 non_max_suppression 函数
2. def nms(box_out, conf_thres=0.4, iou_thres=0.5):
3.     try:
4.         # 尝试使用 multi_label=True 的 NMS
5.         boxout = non_max_suppression(box_out, conf_thres=conf_thres, iou_thres=iou_thres, multi_label=True)
6.     except:
7.         # 如果出错，则使用默认的 NMS
8.         boxout = non_max_suppression(box_out, conf_thres=conf_thres, iou_thres=iou_thres)
9.     return boxout # 返回 NMS 处理后的结果
```

8.6 图像处理函数

`draw_bbox`: 在图像上绘制检测框，并在框旁边显示类别名称和置信度。

```
1. def draw_bbox(bbox, img0, color, wt, names):
2.     det_result_str = '' # 初始化检测结果字符串
3.     for idx, class_id in enumerate(bbox[:, 5]): # 遍历边界框和类别 ID
4.         if float(bbox[idx][4]) < float(0.05): # 如果置信度小于 0.05, 则跳过
5.             continue
6.         print(str(idx) + ' ' + names[int(class_id)]) # 打印边界框索引和类别名称
7.         # 在图像上绘制矩形边界框
8.         img0 = cv2.rectangle(img0, (int(bbox[idx][0]), int(bbox[idx][1])), (int(bbox[idx][2]), i
           nt(bbox[idx][3])), color, wt)
9.         # 在边界框左上角绘制类别名称
10.        img0 = cv2.putText(img0, str(idx) + ' ' + names[int(class_id)], (int(bbox[idx][0]), int(
           bbox[idx][1] + 16)), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
11.        # 在边界框下方绘制置信度
12.        img0 = cv2.putText(img0, '{:.2f}'.format(bbox[idx][4]), (int(bbox[idx][0]), int(bbox[idx][
           1] + 32)), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 0, 255), 1)
13.        # 将检测结果添加到字符串
14.        det_result_str += '{} {} {} {} {} {} \n'.format(names[bbox[idx][5]], str(bbox[idx][4]), bbox
           [idx][0], bbox[idx][1], bbox[idx][2], bbox[idx][3])
15.    return img0 # 返回绘制了边界框和标签的图像
```

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 速度函数
 - 8.8 模型初始化
 - 8.9 模型推理
 - 8.10 基本信息的初始化
 - 8.11 识别结果判定
 - 8.12 动作执行
 - 8.13 异常处理

8.7 速度函数

```
1. #!/usr/bin/env python
2. # -*- coding: utf-8 -*-
3.
4. from command.udp_command import *           # 存放各种结构体与状态数据、指令
5.
6. def go_straight(long, speedgear=3):
7.     # 档位速度只是参考值，具体速度按照实际来判断，默认速度是3档
8.     speedgear_ditc = {
9.         1:7000,
10.        2:7500,
11.        3:8000,
12.        4:8700,
13.        5:9000,
14.        6:10500,
15.    }
16.
17.    val = speedgear_ditc[speedgear]
18.
19.    if long < 0:
20.        val = -val
21.        speed_per_second_meter = (-1.5059e-08) * val ** 2 - (4.1944e-04)
22.        * val - 2.1804
23.    else:
24.        speed_per_second_meter = (-7.237e-09) * val ** 2 + 0.0002933 *
25.        val - 1.643
26.    times = long / speed_per_second_meter
27.    return [times, val]
```

go_straight(long, speedgear=3):

这个函数用于控制设备直线行驶。

long: 表示要行驶的距离，如果为负值，表示反向行驶。

speedgear: 表示设备的档位，影响速度，其默认值为3。

函数首先定义了一个字典speedgear_ditc，将档位映射到一个速度值。

根据long的正负，选择相应的速度计算公式来计算所需的时间times。

如果long为负，速度值val取反，并使用一个二次方程来计算速度（每秒钟行驶的距离）。

如果long为正，使用另一个二次方程来计算速度。

最终，函数返回所需的时间和速度值。

8.7 速度函数

translate_left_and_right(long, speedgear=3): 这个函数用于控制设备左右平移。参数和go_straight函数类似，但速度值是不同的字典speedgear_ditc。根据long的正负，选择相应的线性方程来计算时间。如果long为负，速度值val取反，并使用一个线性方程来计算速度。如果long为正，使用另一个线性方程来计算速度。最终，函数返回平移所需的时间和速度值。

```
1. def translate_left_and_right(long, speedgear=3):
2.     # 档位速度只是参考值，具体速度按照实际来判断，默认速度是3档
3.     speedgear_ditc = {
4.         1: 14000,
5.         2: 18000,
6.         3: 21000,
7.         4: 24000,
8.         5: 27000,
9.         6: 34000,
10.    }
11.    val = speedgear_ditc[speedgear]
12.    if long < 0:
13.        val = -val
14.        speed_per_second_meter = -(1.6e-05) * val - 0.2256
15.        times = abs(long / speed_per_second_meter)
16.    else:
17.        speed_per_second_meter = (1.517e-05) * val - 0.1748
18.        times = long / speed_per_second_meter
19.    return [times, val]
```

8.7 速度函数

revolve_left_and_right(angle): 这个函数用于控制设备左右旋转。angle: 表示要旋转的角度，如果为负值，表示逆时针旋转。函数内部定义了一个辅助函数find_closest_output，它将输入的角度映射到一个最接近的输出值，这个映射是通过一个预定义的字典output_to_input_map实现的。如果输入的角度超过了360度，会通过取余数的方式来调整角度。根据角度的绝对值，找到最接近的角度对应的输出值，这个输出值代表了旋转的时间times。如果角度为负，速度值val设为-10000，表示逆时针旋转；如果为正，则设为10000，表示顺时针旋转。最终，函数返回旋转所需的时间和速度值。

```
1. def revolve_left_and_right(angle):
2.     def find_closest_output(input_value):
3.         # 判断角度是否超过了360度，超过了则需要则算掉
4.         if input_value > 360:
5.             input_value = input_value / (input_value % 360)
6.
7.         # 输出到输入的映射字典
8.         output_to_input_map = {
9.             0: 0,
10.            35.         }
11.
12.         # 获取所有输出键并将其转换为列表
13.         keys = list(output_to_input_map.keys())
14.
15.         # 寻找与输入值最接近的输出键
16.         closest_key = min(keys, key=lambda x: abs(x - input_value))
17.
18.         # 返回与最接近键关联的输入值
19.         return output_to_input_map[closest_key]
20.
21.
22.
23.
24.
25.
26.
27.
28.
29.
30.
31.
32.
33.
34.
35.
36.
37.
38.
39.
40.
41.
42.
43.
44.     times = find_closest_output(abs(angle)) # 获取时间与角度的关系
45.     if angle < 0:
46.         val = -10000
47.     else:
48.         val = 10000
49.
50.     return [times, val]
```

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 速度函数
 - 8.8 模型初始化
 - 8.9 模型推理
 - 8.10 基本信息的初始化
 - 8.11 识别结果判定
 - 8.12 动作执行
 - 8.13 异常处理
- 九、运行示例

8.8 模型推理

定义一个初始化模型的函数。主要作用是初始化深度学习模型并加载相关的标签信息，以便进行模型推理和结果解释。

以下是对文件的总体总结和作用的描述：

```
1. def model_init(model_path, device_id, label_path) -> Tuple['base.model', Dict[int, str]]:
2.     # 初始化资源和变量
3.     base.mx_init() # 初始化 mxVision 资源
4.     model = base.model(modelPath=model_path, deviceId=device_id) # 初始化 base.model 类
5.     labels_dict = get_labels_from_txt(label_path) # 得到类别信息，返回序号与类别对应的字典
6.     return model, labels_dict
```



函数返回两个对象：初始化的模型实例和标签字典。这样，调用者就可以使用这些对象来进行模型推理和结果的解释。

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 速度函数
 - 8.8 模型初始化
 - 8.9 模型推理
 - 8.10 基本信息的初始化
 - 8.11 识别结果判定
 - 8.12 动作执行
 - 8.13 异常处理
- 九、运行示例

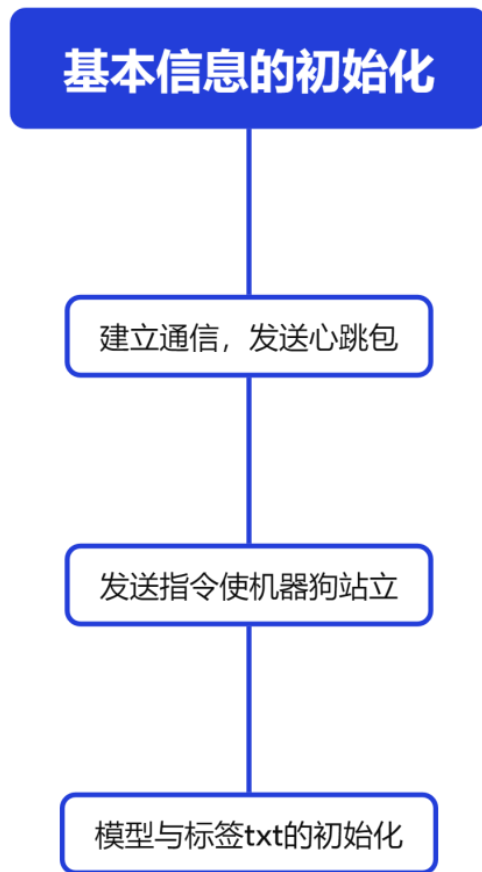
8.9 模型推理

```
1. def Image_inference(model, labels_dict) -> list:
3.     try:
4.         while True:
5.             img = getImage()
6.             frame = img.copy()
7.             # 数据前处理
8.             img, scale_ratio, pad_size = letterbox(frame, new_shape=[640, 640]) # 对图像进行缩放与填充, 保持长宽比
9.             # 用来存放推理结构, 三个元素, [idx: 识别到的序列号 (指的是一次性识别到4个东西, 那么序列号就是0,1,2,3), class_id: 识别到的类型, pred_all[idx][4]: 识别率]
10.            ill_sets = []
11.            img = img[:, :, ::-1].transpose(2, 0, 1) # BGR to RGB, HWC to CHW
12.            img = np.expand_dims(img, 0).astype(np.float32) # 将形状转换为 channel first (1, 3, 640, 640), 即扩展第一维为 batchsize
13.            img = np.ascontiguousarray(img) / 255.0 # 转换为内存连续存储的数组
14.            img = Tensor(img) # 将numpy转为Tensor类
15.
16.            # 模型推理, 得到模型输出
17.            # model = base.model(modelPath=model_path, deviceId=DEVICE_ID) # 初始化 base.model 类
18.            output = model.infer([img])[0] # 执行推理。输入数据类型: List[base.Tensor], 返回模型推理输出的 List[base.Tensor]
19.            # 后处理
20.            output.to_host() # 将 Tensor 数据转移到 Host 侧
21.            output = np.array(output) # 将数据转为 numpy array 类型
22.            boxout = nms(torch.tensor(output), conf_thres=0.4, iou_thres=0.5) # 利用非极大值抑制处理模型输出, conf_thres 为置信度阈值, iou_thres 为iou阈值
23.            pred_all = boxout[0].numpy() # 转换为numpy数组
24.            scale_coords([640, 640], pred_all[:, :4], frame.shape, ratio_pad=(scale_ratio, pad_size)) # 将推理结果缩放到原始图片大小
25.            # labels_dict = get_labels_from_txt('ceshi/demo_labels.txt') # 得到类别信息, 返回序号与类别对应的字典
26.            # img_dw = draw_bbox(pred_all, frame, (0, 255, 0), 2, labels_dict) # 画出检测框、类别、概率
27.
28.            for idx, class_id in enumerate(pred_all[:, 5]):
29.                logging.info(str(idx) + ' ' + labels_dict[int(class_id)])
30.                # 识别率保留两位小数
31.                ill_sets.append([idx, int(class_id), round(pred_all[idx][4], 2)])
32.            if ill_sets == []:
33.                pass
```

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 速度函数
 - 8.8 模型初始化
 - 8.9 模型推理
 - 8.10 基本信息的初始化
 - 8.11 识别结果判定
 - 8.12 动作执行
 - 8.13 异常处理

8.10 基本信息的初始化



8.10 基本信息的初始化

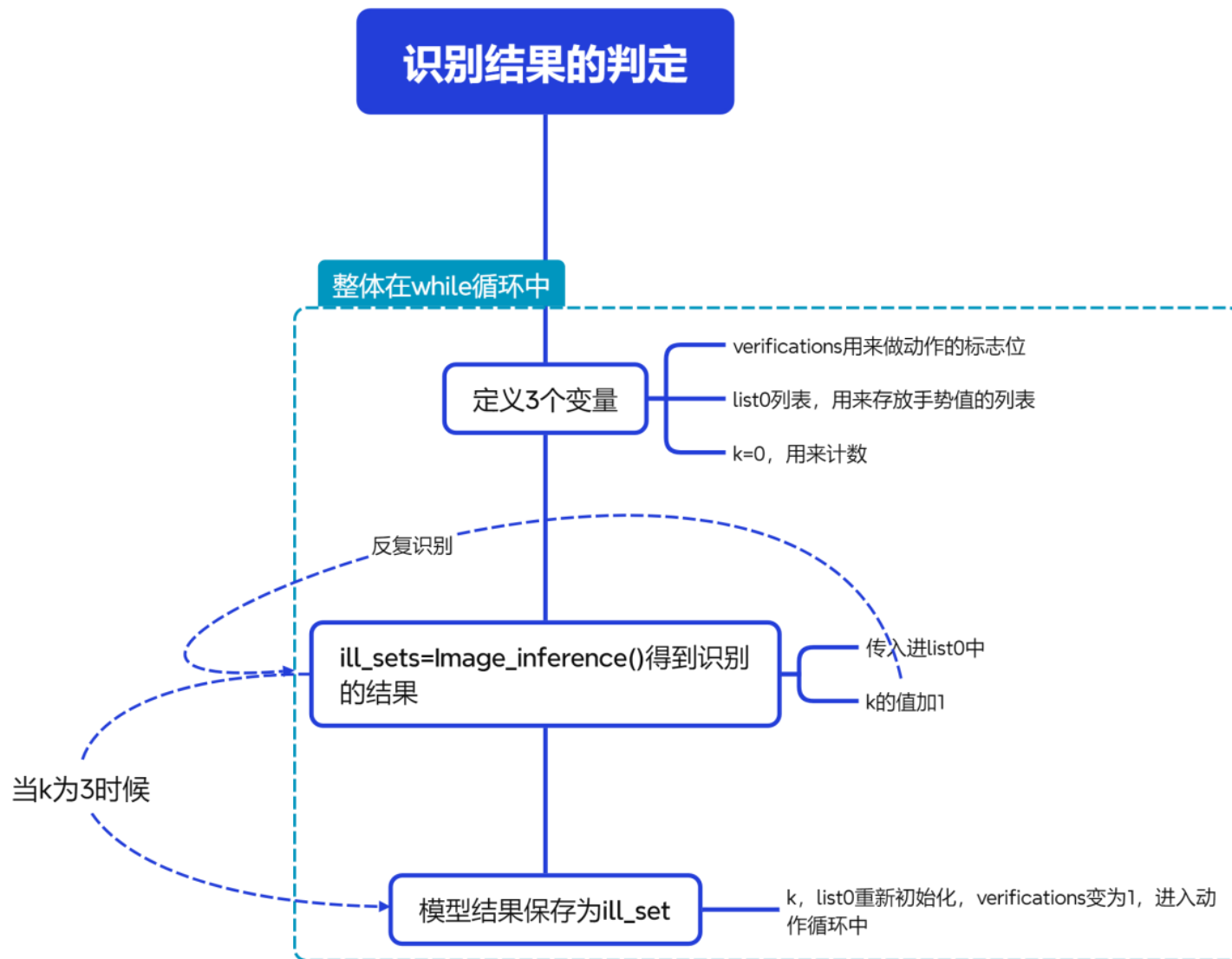
主要是对机器狗的信息初始化与目标设备的通信，下面是详细的代码解释：

```
1. if __name__ == "__main__":
2.
3.     # 建立通讯
4.     sfd, target_address = network_utils.setup_socket_and_address()
5.
6.     # 建立心跳包线程，按照心跳函数中的频率发送心跳包
7.     heartbeat_thread = MyRepeatThread("HeartbeatThread", heartbeat.send_udp_heartbeat, 0.25, None, sfd, target_address)
8.     heartbeat_thread.start()
9.
10.    # 站立指令
11.    SendToCommand.perform_action(sfd, target_address, 0x21010202, 0, 0)
12.
13.    # 变量初始化
14.    DEVICE_ID = 0                                # 设备id
15.    model_path = 'gesture_models/gesture.om'      # 模型路径
16.    label_path = 'gesture_models/predefined_classes.txt' # 标签地址
17.
18.    # 模型与标签txt的初始化
19.    model, labels_dict = model_init(model_path, DEVICE_ID, label_path)
20.
21.    # 创建一个简单指令的结构体实例
22.    action_CommandHead = CommandHead()
```

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 速度函数
 - 8.8 模型初始化
 - 8.9 模型推理
 - 8.10 基本信息的初始化
 - 8.11 识别结果判定
 - 8.12 动作执行
 - 8.13 异常处理

8.11 识别结果判定



8.11 识别结果判定

主要实现实现一个基于手势识别的判定逻辑，用于确定是否连续三次检测到相同的手势，如果是，则触发机器狗的动作执行循环。

```
1.     verifications = 0  # 进入狗动作的标志位
2.     list0 = []        # 存放手势值的列表
3.     k = 0             # 计数
4.     while True:
5.         ill_sets = Image_inference(model, labels_dict)
6.         print("ill_sets: ", ill_sets)
7.         # print('ill_sets的长度: ', len(ill_sets))
8.         list0.append(ill_sets[0][1])
9.         k += 1
10.        if k == 3:
11.            # 将列表转换为集合，集合中的元素都是唯一的
12.            unique_elements = set(list0)
13.
14.            # 如果集合中的元素个数为1，则表示列表中的三个元素都相同
15.            if len(unique_elements) == 1:
16.                verifications = 1 # 当连续三次都是统一个手势，标志位归1，进入到动作循环中
17.                k, list0 = 0, [] # 归0，重新计数
18.            else:
19.                k, list0 = 0, []
20.            pass
```

8.11 识别结果判定

当verifications 为1时，根据机器狗识别到的手势，进入动作执行循环中做出相应的动作。

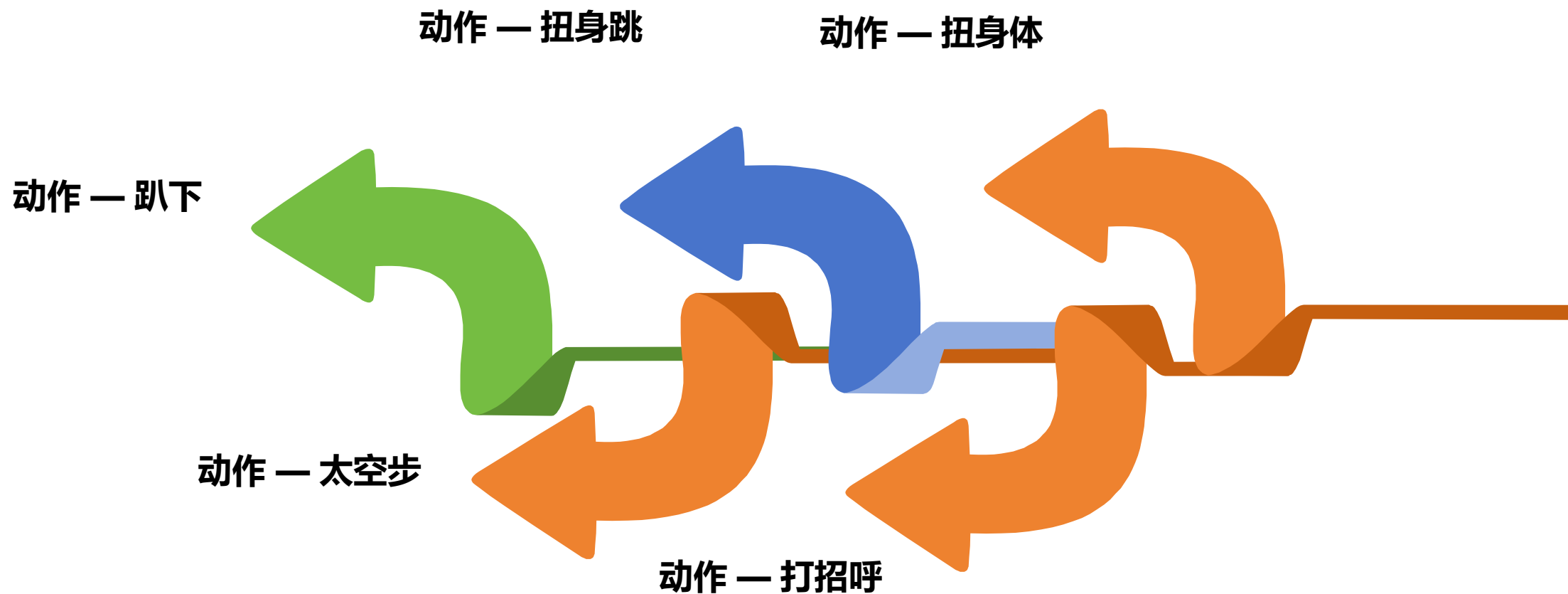
```
1. while verifications:
2.     for ill_set in ill_sets:
3.         # 只执行接受到的第一个动
           作手势，不接受其他的同时执行两个
4. ....
```

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 速度函数
 - 8.8 模型初始化
 - 8.9 模型推理
 - 8.10 基本信息的初始化
 - 8.11 识别结果判定
 - 8.12 动作执行
 - 8.13 异常处理

- 九、运行示例

8.12 动作执行



8.12 动作执行

动作执行—趴下

```
1.  if ill_set[1] == 2: # 'fist' 趴下 ACTION_STAND_DOWN
2.          if status_listener() == [6, 0, 0]: # 监听当前是否是站立状态, 站立状态满足趴下动作
3.              action_CommandHead.code = dog_actions.get("ACTION_STAND_DOWN")
4.              SendToCommand.perform_action(sfd, target_address, action_CommandHead.code, action_CommandHead.parameters_size, action_CommandHead.type_)
5.
6.          while 1:
7.              if status_listener() == [1, 0, 0]: # 监听当前是否是趴下动作
8.                  print("趴下成功! ")
9.                  time.sleep(5)
10.                 SendToCommand.perform_action(sfd, target_address, 0x21010202, 0, 0)
11.                 verifications = 0 # 动作结束, 跳出循环, 重新开始识别
12.                 break
```

8.12 动作执行

动作执行—扭身跳

```
1. elif ill_set[1] == 7: # 'one' 扭身跳 ACTION_TWIST_JUMP
2.         if status_listener() == [6, 0, 0]: # 监听当前是否是站立状态, 站立状态满足扭身
           跳动作
3.             action_CommandHead.code = dog_actions.get("ACTION_TWIST_JUMP")
4.             SendToCommand.perform_action(sfd, target_address, action_CommandHead.
           code, action_CommandHead.parameters_size, action_CommandHead.type_)
5.             while 1:
6.                 if status_listener() == [6, 0, 4]:# 监听当前是否是扭身跳动作
7.                     print("扭身跳! ")
8.                     verifications = 0 # 动作结束, 跳出循环, 重新开始识别
9.                     break
```

8.12 动作执行

动作执行—扭身跳

```
1. elif ill_set[1] == 7: # 'one' 扭身跳 ACTION_TWIST_JUMP
2.             if status_listener() == [6, 0, 0]: # 监听当前是否是站立状态，站立状态满足扭身跳动作
3.                 action_CommandHead.code = dog_actions.get("ACTION_TWIST_JUMP")
4.                 SendToCommand.perform_action(sfd, target_address, action_CommandHead.code, action_Comm
andHead.parameters_size, action_CommandHead.type_)
5.             while 1:
6.                 if status_listener() == [6, 0, 4]:# 监听当前是否是扭身跳动作
7.                     print("扭身跳！")
8.                     verifications = 0 # 动作结束，跳出循环，重新开始识别
9.                     break
```

条件判断

`elif ill_set[1] == 7:` 这一行是if语句的一个elif（即“else if”）分支。它检查`ill_set`列表中索引为1的元素是否等于7，如果是，表示识别到的手势是“one”，设备应该执行扭身跳的动作。

状态监听

`if status_listener() == [6, 0, 0]:` 这一行调用`status_listener()`函数来检查设备是否处于站立状态。站立状态（用`[6, 0, 0]`表示）是执行扭身跳动作的前提条件。

8.12 动作执行

动作执行—扭身体

```
1. elif ill_set[1] == 9: # 'peace' 扭身体 ACTION_TWIST_BODY
2.             if status_listener() == [6, 0, 0]: # 监听当前是否是站立状态，站立状态满足扭身体动作
3.                 action_CommandHead.code = dog_actions.get("ACTION_TWIST_BODY")
4.                 SendToCommand.perform_action(sfd, target_address, action_CommandHead.code, action_CommandH
ead.parameters_size,action_CommandHead.type_)
5.             while 1:
6.                 if status_listener() == [6, 0, 2]:# 监听当前是否是扭身体动作
7.                     print("扭身体！")
8.                     verifications = 0 # 动作结束，跳出循环，重新开始识别
9.                     break
```

条件判断

elif ill_set[1] == 9: 这是一个elif（else if）分支，它在前一个if或elif条件不满足时检查ill_set列表中索引为1的元素是否等于9。如果是，表示识别到的手势是“peace”，设备应该执行扭身体的动。

状态监听

if status_listener() == [6, 0, 0]: 这一行调用status_listener()函数来检查设备是否处于站立状态。站立状态（用[6, 0, 0]表示）是执行扭身体动作的前提条件。

8.12 动作执行

动作执行—太空步

```
1. elif ill_set[1] == 10: # 'rock' 太空步 ACTION_MOONWALK
2.             if status_listener() == [6, 0, 0]: # 监听当前是否是站立状态，站立状态满足太空步动作
3.                 action_CommandHead.code = dog_actions.get("ACTION_MOONWALK")
4.                 SendToCommand.perform_action(sfd, target_address, action_CommandHead.code, action_CommandHead.parameters_size, action_CommandHead.type_)
5.                 while 1:
6.                     if status_listener() == [6, 12, 1]:# 监听当前是否是扭身跳动作
7.                         print("太空步")
8.                         time.sleep(4)
9.                         SendToCommand.perform_action(sfd, target_address, 0x21010202, 0, 0)
10.                    verifications = 0 # 动作结束，跳出循环，重新开始识别
11.                    break
```

条件判断

`elif ill_set[1] == 10`: 这是另一个`elif`（else if）分支，它在前一个`if`或`elif`条件不满足时检查`ill_set`列表中索引为1的元素是否等于10。如果是，表示识别到的手势是“rock”，设备应该执行太空步的动作。

状态监听

`if status_listener() == [6, 0, 0]`: 这一行调用`status_listener()`函数来检查设备是否处于站立状态。站立状态（用`[6, 0, 0]`表示）是执行太空步动作的前提条件。

8.12 动作执行

动作执行—打招呼

```
1. elif ill_set[1] == 16: # 'three2' 打招呼 ACTION_GREET
2.             if status_listener() == [6, 0, 0]: # 监听当前是否是站立状态，站立状态满足打招呼动作
3.                 action_CommandHead.code = dog_actions.get("ACTION_GREET")
4.                 SendToCommand.perform_action(sfd, target_address, action_CommandHead.code,
5.                 action_CommandHead.parameters_size, action_CommandHead.type_)
6.             while 1:
7.                 if status_listener() == [20, 0, 0]:# 监听当前是否是打招呼动作
8.                     print("打招呼")
9.                     verifications = 0 # 动作结束，跳出循环，重新开始识别
10.                    break
```

条件判断

`elif ill_set[1] == 16`: 这是另一个`elif`（`else if`）分支，它在前一个`if`或`elif`条件不满足时检查`ill_set`列表中索引为1的元素是否等于16。如果是，表示识别到的手势是“three2”，设备应该执行打招呼的动作。

状态监听

`if status_listener() == [6, 0, 0]`: 这一行调用`status_listener()`函数来检查设备是否处于站立状态。站立状态（用`[6, 0, 0]`表示）是执行打招呼动作的前提条件。

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
 - 8.1 导入必备的包
 - 8.2 建立通信
 - 8.3 创建指令头
 - 8.4 发送心跳包
 - 8.5 监听模块
 - 8.6 图像处理函数
 - 8.7 速度函数
 - 8.8 模型初始化
 - 8.9 模型推理
 - 8.10 基本信息的初始化
 - 8.11 识别结果判定
 - 8.12 动作执行
 - 8.13 异常处理

九、运行示例

8.13 异常处理

这段代码是整个手势识别流程的“清理”或“重置”部分。它确保如果程序识别到一个无效的手势，它会通知用户并准备重新开始手势识别过程，而不是继续执行错误的或不期望的动作。

```
1. elif ill_set[1] not in [2, 7, 9, 10, 16]:  
2.         logging.info("手势不在动作集合中，请重新做动作")  
3.         verifications = 0 # 动作结束，跳出循环，重新开始识别
```

目录

- 一、实验前置准备 (0.5学时)
- 二、实验过程描述
- 三、实验环境准备 (0.5学时)
- 四、工程文件目录
- 五、数据工程 (1学时)
- 六、模型本地化训练与推理 (1学时)
- 七、在线ModelArts训练与模型转换及应用 (1学时)
- 八、逻辑实现 (4学时)
- 九、运行示例

运行示例

首先在文件夹的目录下执行先执行这个指令

```
. /usr/local/Ascend/mxVision-5.0.RC3/set_env.sh && . /usr/local/Ascend/ascend-toolkit/set_env.sh
```

然后再输入

```
python gesture_main.py
```

在对机器狗做出相对应的手势即可。

```
(base) root@davinci-mini:/opt/lab/GestureRecognition# . /usr/local/Ascend/mxVision-5.0.RC3/set_env.sh && . /usr/local/Ascend/ascend-toolkit/set_env.sh
(base) root@davinci-mini:/opt/lab/GestureRecognition# python gesture_main.py
Begin to initialize Log.
The output directory of logs file exist.
WARNING: Logging before InitGoogleLogging() is written to STDERR
I20240703 14:55:57.105859 62864 FileUtils.cpp:331] The input file is empty
I20240703 14:55:57.105924 62864 FileUtils.cpp:475] Check Other group permission: Current permission is 4, but required no greater than 0.
Save logs information to specified directory.
Find 1 devices!

u3v device: [0]
device model name: MV-CS060-10UC-PRO
user serial number: DA0275184
/usr/local/miniconda3/lib/python3.9/site-packages/torchvision/io/image.py:13: UserWarning: Failed to load image Python extension:
  warn(f"Failed to load image Python extension: {e}")
成功连接到ip: 192.168.1.100, port: 43897
Starting HeartbeatThread
INFO:root:0 three2
ill_sets:  [[0, 16, 0.89]]
INFO:root:0 three2
ill_sets:  [[0, 16, 0.9]]
INFO:root:0 three2
ill_sets:  [[0, 16, 0.93]]
打招呼
```

感谢

版权所有©2024，华为技术有限公司，保留所有权利。

本资料是华为的保密信息，所有内容仅供华为授权的培训客户内部使用，禁止用于任何其他用途。未经许可，任何人不得对本资料进行复制、修改、改编、也不得将本资料或其任何部分或基于本资料的衍生作品提供给他人。

华为开发者创新中心

<https://connect.huaweicloud.com>